

技術解説書

OpenMX 4.0 GPU 版 実装解説書

NVIDIA GPU オフロード実装の全体像と
オリジナル OpenMX 4.0 からの全変更点
— cuBLAS・GEMMu8・cuSOLVER の使い方と注意点 —

対象リビジョン: OpenMX 4.0 GPU 版(2026-07-23 時点、コミット `9fe8ce13`)

比較基準: オリジナル OpenMX 4.0 ソース一式

検証環境: NVHPC SDK 26.3 / CUDA 13.1 / compute capability 12.0(Blackwell 世代 GeForce)

発行日: 2026 年 7 月 23 日(第 3 版)

注記: 本書は第一原理計算コード [OpenMX](#)(GPL v3)を NVIDIA GPU 向けに移植した**非公式版**の技術解説書であり、OpenMX 開発チームによる公式文書ではありません。記載のファイル名・行番号は上記リビジョン時点のものであり、開発の進行に伴って変わる可能性があります。

目次

1. はじめに

2. 全体アーキテクチャ

設計方針 / レイヤ構成 / 有効化とデイスパッチ / GPU 化対象マップ / 複数ランク共有 GPU 対策

3. cuBLAS の使い方

使用ルーチン / ハンドルとストリーム / メモリ管理 / ソルバ別 GEMM エンジンの実像 / エラー処理 / 注意点

4. GEMMul8 の使い方

原理と出自 / 統合アーキテクチャ / 精度パラメータ / ワークスペース管理 / 複素対応 / ビルド / 注意点

5. cuSOLVER(gpusolver 層)の使い方

層の構成と命名意図 / 使用 API / 一般化固有値問題 / devInfo 処理 / 適応並列度制御 / ELPA フォールバック / デバイス割当 / 注意点

6. OpenACC と生 CUDA の使い分け

データ常駐の 3 パターン / host_data 連携 / set_hamiltonian_gpu.cu / NVHPC 固有の注意

7. 計算経路別の GPU 実装詳細

Band / Cluster / DC / DC-LNO / Krylov / 行列要素構築 / VNA 射影子 / 密度グリッド / 全エネルギー / 力 / 混合スキーム / 派生ソルバ群

8. GPU 化以外の変更点

OpenMP バグ修正 / メモリ管理の堅牢化 / その他の修正

9. ビルドシステム

ツールチェーン / フラグ / リンク構成 / third_party 自動ビルド / ELPA / 派生 Makefile / 手順

10. 実行時制御リファレンス

入力キーワード / 環境変数一覧 / 標準出力の GPU 診断行 / チューニング指針

11. 制約・注意点の総まとめ

付録 A. 変更ファイル一覧(47 ファイル)

付録 B. 新規追加ファイル一覧

付録 C. 開発履歴(全 91 コミット)

参考文献

1. はじめに

1.1 本書の目的と対象読者

本書は、密度汎関数理論(DFT)に基づく第一原理計算コード **OpenMX 4.0** を NVIDIA GPU 向けに移植した「OpenMX 4.0 GPU 版」について、(1) 現在の実装の全体像、(2) オリジナル OpenMX 4.0 からの**すべての変更点**、(3) 中核となる 3 つの GPU ライブラリ — **cuBLAS・GEMM_{ul}8・cuSOLVER** — の使い方と注意点、を一次情報(ソースコードと diff)に基づいて解説する公開ドキュメントです。想定読者は次のとおりです。

- OpenMX を GPU で高速に実行したいユーザー(第 2・10・11 章が中心)
- 本実装を理解・改修したい開発者(第 3～9 章)
- AMD GPU など他ベンダーへの移植を検討する開発者(第 5 章の gpusolver 層、第 11 章)

前提知識として、DFT・LCAO 法の初步、C 言語、MPI、CUDA プログラミングの基礎を仮定します。

1.2 OpenMX と本移植の位置づけ

OpenMX(Open source package for Material eXplorer)は、局在擬原子軌道(LCAO)基底とノルム保存擬ポテンシャルを用いた DFT コードで、クラスタ計算・バンド計算に加え、発散重畳(DC)法・DC-LNO 法・Krylov 部分空間法などの $O(N)$ 法、非平衡グリーン関数(NEGF)輸送計算などを備えます。ライセンスは GPL v3 です。

本 GPU 版は OpenMX 4.0 の公式ソースを出発点とし、**91 コミット**の開発で GPU オフロードを段階的に追加したものです。初期の開発(～第 2 版の対象であった 51 コミット)では SCF 計算で支配的な「密行列の固有値問題」と「行列積(GEMM)」を GPU 化し、その後の 40 コミットで対象が大きく広がりました。現在は、行列要素構築(Set_Hamiltonian の格子積分・Set_Nonlocal・Set_ProExpn_VNA)、密度グリッド構築(Set_Density_Grid)、力の主要トレース(#2/#3/#4/#4B/#5/HNL)、全エネルギーの格子縮約、電荷/DM 混合(RMM-DIISH の GPU 化ほか全スキームの高速パス)までが GPU またはそれに準ずる高速経路で実行されます。疎行列処理、通信、入出力などはオリジナルの構造を保っています。

1.3 変更規模の概要

項目	値	備考
変更されたソースファイル	47 ファイル(+ <code>makefile</code> → <code>Makefile</code> の改名・全面改修)	削除されたソースは <code>my_cublasDgemm.c</code> / <code>my_cublasZgemm.c</code> の 2 件(休眠ラップの撤去)
変更行数(生の diff)	約 +47,000 / -21,200 行	全面再インデント(<code>Force.c</code> 等)を含む。 <code>Force.c</code> 単体で +15,123/-11,241
新規ソースファイル	14 ファイル、計約 4,000 行	gpusolver ラップ、GEMM _{ul} 8 ブリッジ、GPU 用エネルギー/密度グリッドカーネル、Set_Hamiltonian 用 CUDA カーネルなど
新規ビルドファイル	3 ファイル	<code>Makefile</code> / <code>Makefile.bunshiken</code> / <code>makefile.cpu_intel.bak</code> (原本の同一保存)
third_party(git submodule)	2 件	GEMM _{ul} 8(コミット固定)、FFTW 3.3.11(自前ビルド)。どちらも <code>make</code> が自動取得・自動ビルド

1.4 対象環境

- **コンパイラ:** NVIDIA HPC SDK(NVHPC)26.3 — `mpicc` (nvc)+ OpenACC、`mpif90` (nvfortran)、`nvcc`(CUDA 13.1)
- **GPU:** CUDA 対応 NVIDIA GPU。GEMM_{ul}8 は INT8 Tensor Core を使用。開発・検証は compute capability 12.0(Blackwell 世代 GeForce)で実施
- **並列モデル:** MPI プロセス並列。ノード内 local rank によるラウンドロビンで GPU を割当て、**複数ランクが 1 GPU を共有**することを前提に設計。共有状況は物理 GPU の UUID 単位で検出され、必要 VRAM の見積り・実予約に基づいて並

列度が自動調整される(§ 2.5、§ 5.5)

- **OpenMP:** ビルドは `-fopenmp` 付きだが、GPU 経路の一部(`Set_OLP_Kin` の動径積分オフロード)は 1 スレッド実行を要求する。GPU 利用時は MPI のみ(スレッド数 1)の運用が想定されている

1.5 表記法

- ソース位置は `ファイル名:行番号` で示す(すべて `source/` ディレクトリ基準、2026-07-23 時点)。
- **GPU** 既定で GPU 実行、**条件付き** 環境変数などによるオプトイン/メモリ状況依存、**CPU** CPU 実行(意図的に GPU 化しない)、**休眠** 実装済みだが現行経路では呼ばれないコード。
- 「オリジナル」は OpenMX 4.0 公式配布のソースを指す。

本書の読み方: ライブラリの使い方だけを知りたい場合は第 3～5 章、運用パラメータは第 10 章、既知の落とし穴は第 11 章にまとまっています。第 7 章は計算経路(ソルバ)別の縦割り解説で、第 3～6 章(ライブラリ・技法別の横割り)と相互参照になっています。第 2 版(2026-07-05)からの主な差分は、行列要素構築・密度グリッド・力・混合の GPU 化(§ 7.7～7.12)、全経路の適応並列度制御と ELPA フォールバック(§ 5.5～5.6)、GPU フェーズ間のメモリ協調機構(§ 2.5)です。

2. 全体アーキテクチャ

2.1 設計方針

1. **実行時切替・CPU 経路の完全温存** — GPU 化は `#ifdef` による一括置換ではなく、既存コードへの**分岐追加**で実現している。入力ファイルの 1 行 `scf.eigen.lib gpusolver` だけで有効化され、指定しなければオリジナルと同じ ELPA/ScaLAPACK/LAPACK 経路が走る。ゲートは実行時フラグ `scf_eigen_lib_flag == GPUSOLVER` (enum 値 3)の一点に集約されている。
2. **大規模な密行列問題のみ GPU 化** — 行列次元が大きい値 `GPU_CPU_SWITCH_NUM = 1000` (`openmx_common.h:4135`、DC-LNO のみ既定 800)未満の場合は転送コストが勝るため CPU にフォールバックする。
3. **MPI ランク単位の GPU 利用と「共有」前提の設計** — ノード内 local rank % GPU 数のラウンドロビン割当。1 GPU を複数ランクが共有する状況を正面から想定し、物理 GPU の **UUID 単位でデバイス共有ランク群を検出**したうえで、VRAM 見積り/実予約に基づく適応並列度制御・実行時 ELPA フォールバック・失敗時リトライ・GPU フェーズ間のメモリ協調(需要公開レジストリ)という多層の制御を持つ (§ 2.5)。
4. **GEMM は GEMMu8 を第一選択、cuBLAS FP64 へ自動フォールバック** — 大規模 GEMM は INT8 Tensor Core による FP64 エミュレーション(Ozaki Scheme II)で実行し、ワークスペースが確保できない場合のみ native cuBLAS に切り替える(第 4 章)。
5. **対角化は cuSOLVER の 64-bit generic API** — `cusolverDnXsyevdx` (部分固有値)を中心に使い、一般化固有値問題 $Hc = \varepsilon Sc$ は S の固有分解によるレイディン(対称)直交化を GPU 上で自前実装する(第 5 章)。固有値のみ必要な場面では `jobz=NOVECTOR` で解く。
6. **ベンダー中立の命名(gpusolver)** — 将来の AMD GPU 対応を見据え、ラッパ層の識別子・入力キーワードを `cusolver` から `gpusolver` へ一括改名済み(コミット 2302f965, d8219384)。cuSOLVER API の呼び出し自体は NVIDIA ビルドの実装詳細として残る。なお開発初期に MAGMA も評価されたが、現在はソースツリーから完全に削除されている(現行ソースに MAGMA への参照は 0 件)。
7. **数値再現性への配慮** — GPU 経路は原則として「CPU ループと同一の加算順序」を保つように書かれている: 密度行列構築や格子トレースの縮約は `#pragma acc loop seq` または要素単位の逐次累積、力の GPU バッチは float 積 → double 加算のキャストの位置まで CPU を再現、混合の高速パスはポインタ回転でビット単位の同値性を保証する。GEMMu8 自体も bitwise 再現性を持つ(一方で atomic 加算による順序非決定箇所も存在する。§ 11 参照)。
8. **フォールバックは「静かに正しく」** — VRAM が足りない場合、各フェーズは事前見積り/実予約で発動可否を判定し、可なら CPU/ELPA の従来経路へ落ちる。通知はノードあたり 1 回(または値が変わったときだけ)に抑制され、結果の正しさは変わらない。abort するのは「回復不能なエラー」と「明示強制(環境変数)との矛盾」だけに縮小された。

2.2 レイヤ構成

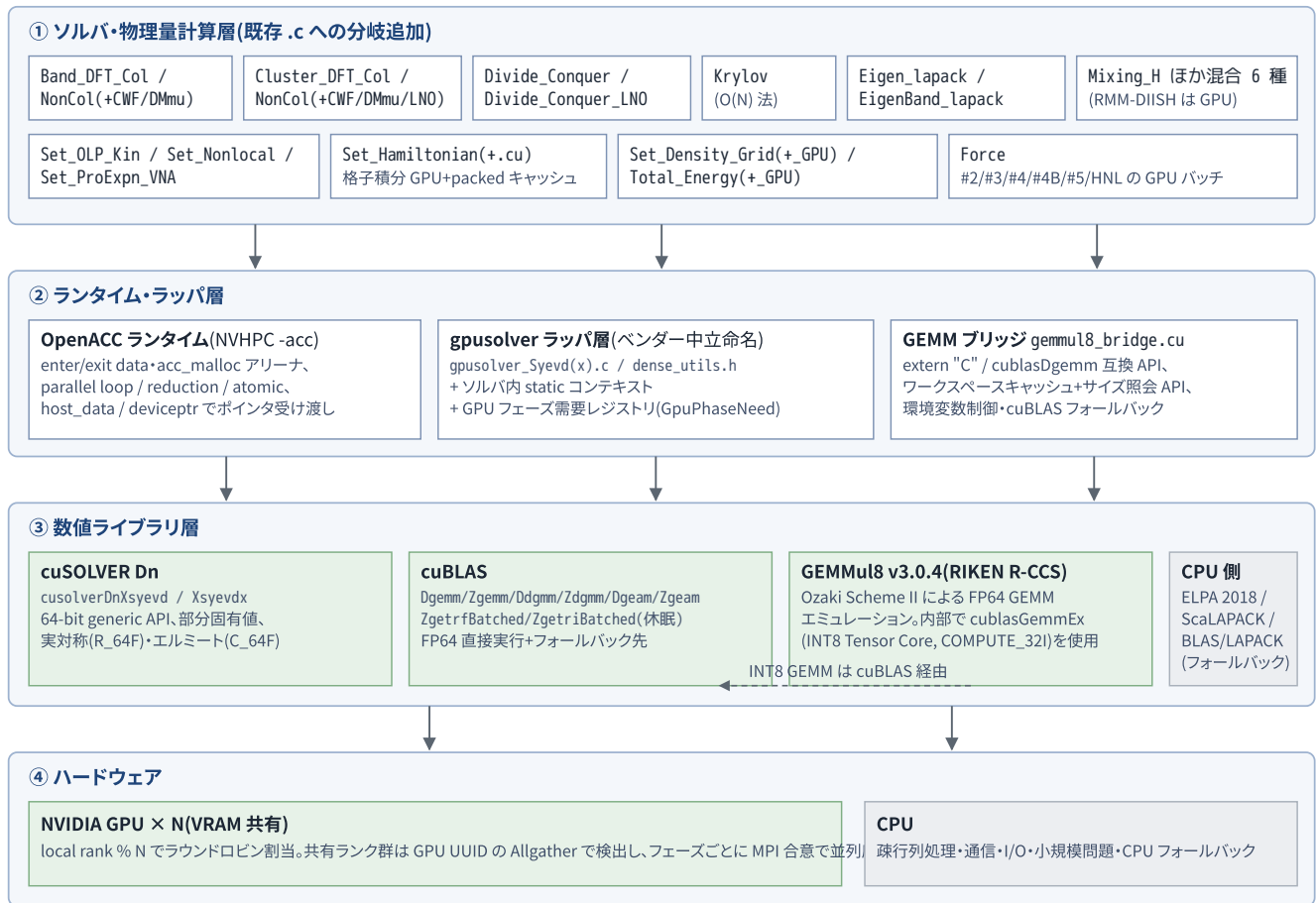


図 1: OpenMX 4.0 GPU 版のレイヤ構成。緑は GPU 実行、灰は CPU 実行。

GPU 実行の形態は 3 種類が併用されている。

- OpenACC ディレクティブ** — 格子積分・縮約・行列組立などのループ並列。データ常駐は `enter data / exit data` に加え、力計算・混合・クラスタ系では `acc_malloc / cudaMalloc` による**単一確保のアリーナ**+ `deviceptr / acc_map_data` が主流になった(第 6 章)。
- ソルバ内 static コンテキスト+生 CUDA API** — `cudaMalloc` による永続デバイスバッファと cuBLAS/cuSOLVER ハンドルをファイルスコープの構造体に保持し、ライブラリを直接呼ぶ(`BandColGpuSolverCtx` など。第 3・5 章)。
- extern "C" ブリッジ経由の CUDA C++** — GEMMv8(第 4 章)と、Set_Hamiltonian の共有メモリアル型 行列要素カーネル `set_hamiltonian_gpu.cu` (§ 7.7)。C コードから C++/CUDA を呼ぶための単一翻訳単位方式。

2.3 有効化の流れと SCF 内の GPU 化ポイント

GPU 経路が有効になるまでの流れは次のとおり。

- 入力解釈**(`Input_std.c`) : `scf.eigen.lib gpusolver` で `scf_eigen_lib_flag = GPUSOLVER`。旧名 `cusolver` は非推奨エイリアスとして警告付きで受理される。ユーザー指定値は `scf_eigen_lib_flag_input` に保存。
- デバイス初期化**(`DFT.c` の `DFT_GPU_DeviceInit()`) : `MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)` でノード内 local rank を求め、`cudaSetDevice` と `acc_set_device_num` を**対**で設定する。CUDA/OpenACC デバイスが見つからなければ警告を出して `scf_eigen_lib_flag = ELPA2` に自動降格する。

3. 各フェーズ入口での条件判定: GPUSOLVER フラグ && サイズ条件 && メモリ条件(見積り or 実予約) を満たすときだけ GPU 経路に入り、満たさなければ従来経路(ELPA2/CPU)がそのまま走る。判定はデバイス共有ランク群または `mpi_comm_level1` 全体の MPI 合意 (Allreduce(MIN))で全ランク一致に揃えられる。

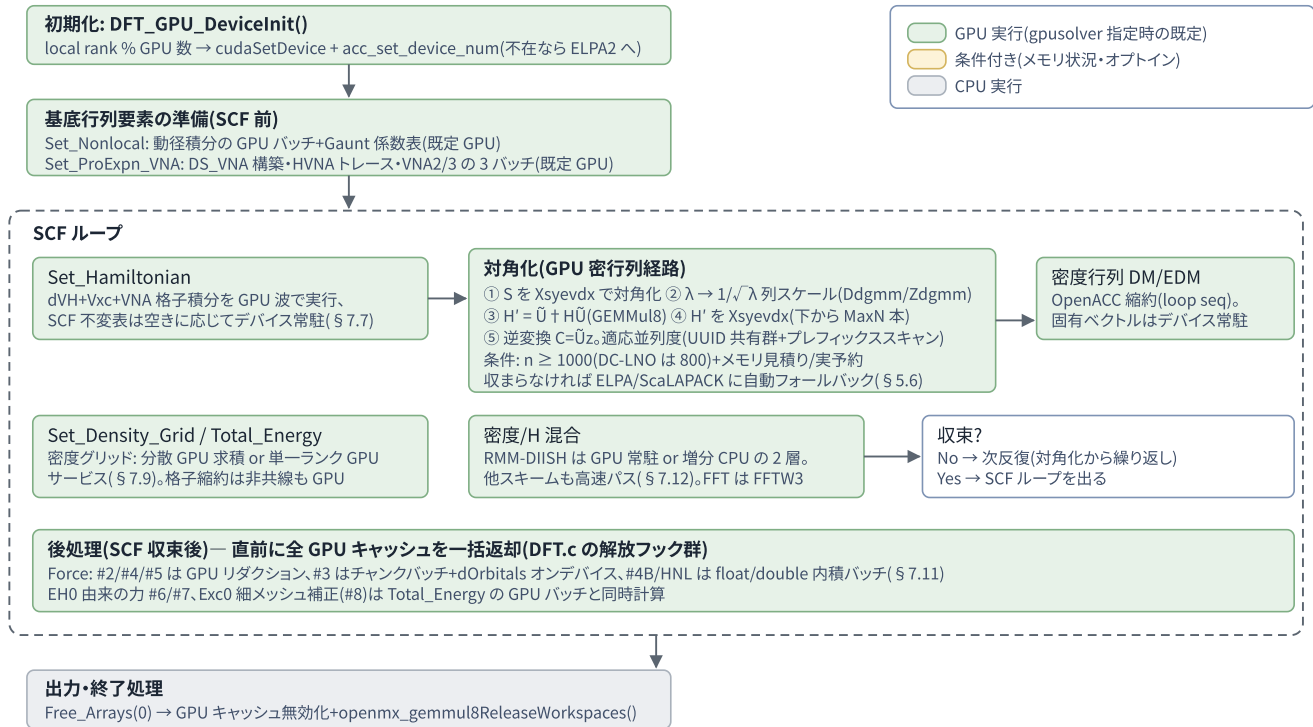


図 2: SCF 計算 1 サイクルの流れと GPU 化ポイント(コリニア・バンド/クラスタ計算の場合)。

2.4 GPU 化対象マップ

計算経路(`scf.EigenvalueSolver`)別・補助計算別の GPU 化状況を表 2-1 に示す。詳細は第 7 章。

経路 / 計算	主担当ファイル	GPU 化内容	発動条件
クラスタ(Col)	<code>Cluster_DFT_Col.c</code>	root-dense 対角化一式(S 対角化 → レイディン直交化 → H' 対角化 → 逆変換)+ DM/EDM の GPU 蓄積。固有ベクトルはデバイス常駐のまま DM へ。2 スピンが同居できない場合はスピンワールド 0 のオーナーが両スピンを直列に解く	GPUSOLVER かつ $n \geq 1000$ かつ実予約成功(不成立なら ELPA へ)
クラスタ(NonCol)	<code>Cluster_DFT_NonCol.c</code>	単一 cudaMalloc アリーナ上の root-dense 対角化 (Dsyevx/Zheevx)+ $U + HU$ (GEMMul8 $\times 12$)+ DM/EDM 縮約。S/Ss2・疎 → 密 index+ cublas ハンドル+ zheevdx ワークスペースを反復間キャッシュ	GPUSOLVER かつ $2n \geq 1000$ かつアリーナ予約成功(不成立なら ELPA へ)
バンド(Col)	<code>Band_DFT_Col.c</code>	k 点ごとの密行列対角化一式をデバイス常駐で実行。GEMM は GEMMul8 Zgemm。S 変換行列をキャッシュ。第 1 パスは NOVECTOR 対角化。UUID 共有群+降順プレフィックススキャンの適応並列度	GPUSOLVER かつ $n \geq 1000$ かつ 1 ターン分の VRAM 見積りが成立(不成立なら ELPA へ)
バンド(NonCol)	<code>Band_DFT_NonCol.c</code>	OpenACC 常駐型 root-dense 経路。固有値のみ/固有ベクトル/DM の 3 ステージ別 VRAM 見積りで並列度を独立に決定	GPUSOLVER かつ $2n \geq 1000$ かつ見積り成立(不成立なら ELPA へ)
DC 法(O(N))	<code>Divide_Conquer.c</code>	原子ごとの部分クラスタ行列を Xsyevdx で対角化。GEMMul8 → cuBLAS → CPU の多段フォールバック。S12 パネルのデバイス常駐キャッシュ(S-cache)+ cublasDgeam デバイス転置	GPUSOLVER かつ局所次元 ≥ 1000 (<code>OPENMX_DC_GPU_THRESHOLD</code>)

経路 / 計算	主担当ファイル	GPU 化内容	発動条件
DC-LNO 法	<code>Divide_Conquer_LNO.c</code>	direct per-rank dispatch。 非コリニアも GPU 化済み (スピン倍化変換+GEMM $\times 8$ Zgemm $\times 3$)。デバイス共有ランク群の合算メモリ判定で CPU フォールバック	GPUSOLVER かつ局所次元 ≥ 800 (<code>OPENMX_DCLNO_GPU_THRESHOLD</code>)
Krylov 法($O(N)$)	<code>Krylov.c</code>	原子ごとの射影解パイプラインを 1 回のデバイス訪問に融合 (GEMM $\times 3$ + Xsyevd)。Krylov 基底のデバイス常駐 KU キャッシュ	GPUSOLVER かつ 3 GEMM とも $m \cdot n \cdot k \geq 5 \times 10^7$ (対角化は $n \geq 1000$)
汎用固有値ルーチン	<code>Eigen_lapack.c</code> <code>EigenBand_lapack.c</code>	$n \geq 1000$ で <code>gpuserver_Syevdx(_Complex)</code> へ分岐。これらと呼ぶ全 20 以上のファイルが自動的に GPU 化の恩恵を受ける	GPUSOLVER かつ $n \geq 1000$
分極・DMmu・CWF・LNO 系	11 ファイル	ELPA2 分岐の手前に定型の GPUSOLVER 分岐(<code>dense_utils</code> 経由の <code>Dsyevx/Zheevx</code>)を挿入	GPUSOLVER かつ $n \geq 1000$ かつ <code>na_rows==n</code> 等
行列要素構築	<code>Set_Hamiltonian.c</code> <code>set_hamiltonian_gpu.cu</code> <code>Set_Nonlocal.c</code> <code>Set_OLP_Kin.c</code>	<code>Set_Hamiltonian</code> の <code>dVH+Vxc+VNA</code> 格子積分は 既定で GPU (共有メモリアイルの CUDA カーネル+適応波スケジューリング+SCF 不変表の常駐キャッシュ)。 <code>Set_Nonlocal</code> は動径積分のデバイス生成球ベッセル+バッチ縮約。 <code>Set_OLP_Kin</code> はオプション	GPUSOLVER(+メモリ判定、§ 7.7)
VNA 射影子	<code>Set_ProExpn_VNA.c</code>	DS_VNA 構築・HVNA トレース・VNA2/VNA3 の 3 バッチを GPU 実行(球ベッセルのデバイス漸化、Gaunt/変換表のホスト表引き化)	GPUSOLVER(+バッチ別メモリ判定)
密度グリッド	<code>Set_Density_Grid.c</code> <code>Set_Density_Grid_GPU.c</code>	分散 GPU 求積(各ランクが自原子を積分)→ 単一ランク GPU サービス(オーナーが全域を構築し scatter)→ CPU の 3 段フォールバック	GPUSOLVER かつ Cluster/Band かつ <code>Cnt_switch==0</code>
全エネルギー	<code>Total_Energy.c</code> <code>Total_Energy_GPU.c</code>	EXC/EH1・双極子・CWF-Dc・EH0 二中心項・Exc0 細メッシュ補正の格子縮約を OpenACC 実行。 非共線(NC)でも有効	GPUSOLVER(<code>OPENMX_TOTALENERGY_GPU=0</code> で無効化)
力	<code>Force.c</code>	#2/#4/#5 の GPU リダクション、#3 のチャンクバッチ(dOrbitals オンデバイス再計算つき)、 #4B・HNL の GPU 内積バッチ (旧版で CPU 維持とされていた #4B も GPU 化)。アリーナ+ピア待ち機構で共有デバイスのメモリ競合に耐性	GPUSOLVER(+項別メモリ判定、§ 7.11)
電荷/DM 混合	<code>Mixing_H.c</code> ほかに 5 ファイル	RMM-DIISH は GPU 常駐 Gram(Tier A)/増分 CPU(Tier B)の 2 層。RMM-DIIS/DIISK/DIISV/GR-Pulay/Kerker はバッチ Gram・増分 Gram・ポインタ回転シフトの CPU 高速パス	GPUSOLVER(<code>OPENMX_MIX_FAST=0</code> で無効化)
NEGF・複素逆行列	<code>Lapack_LU_inverses.c</code>	休眠 <code>cublasZgetrf/ZgetriBatched</code> による GPU 版 LU 逆行列を実装済みだが呼び出しはコメントアウト	—(将来の布石)

表 2-1: GPU 化対象マップ

2.5 複数ランク共有 GPU への対策(概観)

本実装の特徴は、「16 GB 級の GPU 1 台を 8~36 MPI ランクで共有する」ような構成を明示的に想定した多層防御にある。第 2 版以降、この防御は「固定ノブによる直列化」から「**実測に基づく適応制御+実行時フォールバック**」へ全面的に進化した。

機構	内容	実装箇所
デバイス共有ランク群の検出	<code>MPI_Comm_split_type(SHARED)</code> + <code>cudaGetDeviceProperties</code> の GPU UUID を Allgather し、同一物理 GPU を共有するランクのコミュニケーターを作る。空き VRAM は <code>acc_clear_freelists()</code> でプールを返却してから <code>cudaMemGetInfo</code> を全員で測り、MIN で共有	<code>Band_DFT_Col.c</code> 、 <code>Band_DFT_NoCol.c</code> 、 <code>Set_Hamiltonian.c</code> 、 <code>Mixing_H.c</code>

機構	内容	実装箇所
適応並列度(プレフィックススキャン)	各ランクの必要 VRAM を Allgather して降順ソートし、「上位 c ランクの合計+予約が空きに収まる最大の c」を 1 本のスキャンで決定。全ランクが同時に収まる場合はバリアを畳み込み、収まらなければ波(ターン)に分割。初期ウェーブはデバイス共有ランク数(旧版の固定はしご 32→……→1 は撤廃)	<code>BandCol_AutoGpuTurnLimit</code> 、 <code>BandNonCol_AutoGpuTurnLimit</code> 、 <code>Set_Hamiltonian_CreateGpuTurnPlan</code>
実予約プローブ+ELPA フォールバック	クラスタ経路は「見積り」ではなく 実際に <code>cudaMalloc</code> を試して可否を判定 (8 回上限のリトライ、失敗時は全解放)。バンド経路は 1 ターン分の見積りで判定。どちらも不成立なら ELPA/ScaLAPACK に自動フォールバックし、判定は <code>MPI_Allreduce(MIN)</code> で全ランク一致	<code>ClusterCol_GpuDiagFits</code> 、 <code>ClusterNonCol_GpuDiagFits</code> 、 <code>BandCol_GpuDenseFits</code> 、 <code>BandNonCol_GpuDiagFits</code> (§ 5.6)
GPU フェーズ需要レジストリ	対角化・密度グリッドが「このデバイスでフル並列時に必要な総量」を <code>OpenMX_GpuPhaseNeed_Register()</code> で公開し、 <code>Set_Hamiltonian</code> の常駐キャッシュが その分を空けてから入場 ・不足時は自主退去する(フェーズ間のメモリ協調)	<code>openmx_common.c</code> のレジストリ、 <code>Set_Hamiltonian.c</code> (§ 7.7)
カバッチのアリーナ+ピア待ち	力計算の GPU バッチは全確保を 512 B 整列の単一アリーナにまとめ、必須確保は「50 ms 刻みで最大 180 秒ピアの解放を待つ」、任意確保(リダクションバッファ)は失敗時その場でホスト計算。 abort し得るデバイス確保はカステージに存在しない	<code>Force_gpu_arena_wait/try</code> (§ 7.11)
SCF フェーズ間の一括返却	毎 SCF サイクルの力計算直前に、 <code>Set_Hamiltonian</code> 常駐表・DC S-cache・Krylov KU キャッシュ・クラスタソルバコンテキスト・混合履歴アリーナをすべて解放し、カバッチが真の空き VRAM を測れるようにする	<code>DFT.c:2312-2319</code> の解放フック群
アロケーション失敗リトライ	<code>wait_cudafunc</code> マクロが CUDA/cuBLAS/cuSOLVER 呼び出しを成功までランダム待ち($\leq 10 \mu\text{s}$)付きで再試行し、失敗ごとにエラーラッチを掃除。恒久的不足があり得る確保は有界リトライの <code>TryDeviceMalloc</code> 系へ移行済み	<code>openmx_common.h:4148-4162</code>
ワークスペースのターン毎解放	約 1 GiB の GEMMu8 ワークスペースをターン/ソルブ終了ごとに返却 (Band・Cluster・DC-LNO、既定有効)	<code>OPENMX_*_GEMMU8_TURN_RELEASE</code> 系

休眠コードの扱い: 本書では、定義は存在するが現行経路では呼ばれないコード (`gpusolver_Syevd`、`LU_Zinverse_GPU`、一部の GEMMu8 ヘルパなど) を **休眠** と明示して区別する。一覧は § 11.7 を参照。なお第 2 版で休眠扱いだった `openmx_cuda_kernels.cu` (未結線の生 CUDA カーネル集) はソースツリーから削除され、代わりに**結線済みの** `set_hamiltonian_gpu.cu` が加わった。

3. cuBLAS の使い方

3.1 使用ルーチンと役割

cuBLAS は本実装で 3 つの役割を持つ: (1) 対角スケーリング(`Ddgm` / `Zdgm`)や デバイス内転置(`Dgeam` / `Zgeam`)など GEMM 以外の密行列演算の直接実行、(2) GEMM_{ul}8 が使えないときの FP64 フォールバック先、(3) GEMM_{ul}8 内部の INT8 Tensor Core GEMM(`cublasGemmEx`)の実行基盤。かつて一部ソルバに残っていた `cublasDgemm/Zgemm/Dsymm` の直接呼び出しは、コミット 66ce1e62 (Use GEMM_{ul}8 for remaining GPU GEMM paths)で**すべて GEMM_{ul}8 経由に統一**された。使用ルーチンの全体像を表 3-1 に示す。

cuBLAS ルーチン	用途	主な使用箇所
<code>cublasDgemm</code> / <code>cublasZgemm</code>	GEMM _{ul} 8 ブリッジ内の FP64 フォールバック先、および DC 法の多段フォールバック 2 段目(直接呼び出しはこの 2 箇所のみ)	<code>gemmul8_bridge.cu</code> 、 <code>Divide_Conquer.c</code>
<code>cublasDdgm</code> / <code>cublasZdgm</code>	レイディン直交化の $1/\sqrt{\lambda}$ 列スケーリング(対角行列との積)。DC-LNO 非コリニアではスピン倍化変換行列 $T = \text{diag}(U \cdot s^{-1/2}, U \cdot s^{-1/2})$ の左上ブロック生成にも使用	<code>Cluster_DFT_Col.c</code> 、 <code>Band_DFT_Col.c</code> 、 <code>Divide_Conquer_LNO.c</code>
<code>cublasDgeam</code> / <code>cublasZgeam</code>	デバイス内転置(純粋なデータ移動なので後段への入力はビット一致)。DC 法はホスト側 $O(n^2)$ ストライド転置を <code>DC_gpuSolver_DeviceTranspose</code> (<code>Divide_Conquer.c:1175</code>)に置換。DC-LNO 非コリニアは H' の共役行列生成に使用	<code>Divide_Conquer.c</code> 、 <code>Divide_Conquer_LNO.c</code>
<code>cublasZgetrfBatched</code> / <code>cublasZgetriBatched</code>	休眠 複素 LU 分解・逆行列(batch=1)。NEGF 表面グリーン関数向けの布石(§ 5.9)	<code>Lapack_LU_inverse.c:384,400</code>
<code>cublasGemmEx</code> (INT8, COMPUTE_32I)	GEMM _{ul} 8 内部の法別 INT8 行列積(OpenMX からは直接呼ばない)	<code>third_party/GEMM_{ul}8/src/oz2/common/matmult.hpp</code>

表 3-1: 使用 cuBLAS ルーチン一覧

3.2 ハンドルと CUDA ストリームのライフサイクル

現行実装の主経路では、**ソルバごとのファイルスコープ static コンテキスト**が cuBLAS/cuSOLVER ハンドルを 遅延初期化で 1 個ずつ保持する(MPI プロセスあたりソルバ種別ごとに 1 個)。デバイスが変わった場合のみ破棄・再生成される。かつて反復ごとに create/destroy していた箇所も、現在はキャッシュに統一されている (例: 非コリニアクラスタは反復あたり 13 回のハンドル生成を `ClusterNonCol_CublasHandle()` の デバイス別キャッシュに置換、`Cluster_DFT_NonCol.c:762`。DC 非コリニア/DC-LNO 非コリニアの毎原子対角化は `gpusolver_Syevdx_Complex_openacc_cached` がハンドル・ストリーム・ワークスペースを保持、§ 5.2)。

`Cluster_DFT_Col.c` — コンテキストの遅延初期化(典型形)

```
wait_cudafunc(cudaGetDevice(&current_device));

if (ctx->initialized && ctx->device_id == current_device) return;
if (ctx->initialized) ClusterCol_GpuSolver_Destroy();

wait_cudafunc(cudaStreamCreateWithFlags(&ctx->stream, cudaStreamNonBlocking));
wait_cudafunc(cublasCreate(&ctx->cublas));
wait_cudafunc(cusolverDnCreate(&ctx->gpusolver));
wait_cudafunc(cublasSetStream(ctx->cublas, ctx->stream));
wait_cudafunc(cusolverDnSetStream(ctx->gpusolver, ctx->stream));

ctx->initialized = 1;
ctx->device_id = current_device;
```

ストリームの扱いはソルバ間で**非対称**である点に注意が必要である。

- **Cluster_DFT_Col / Cluster_DFT_NonCol / Band_DFT_NonCol(DM 用) / Krylov / DC / DC-LNO:** 専用の non-blocking ストリームを生成し、`cublasSetStream` / `cusolverDnSetStream` で束ね、転送は `cudaMemcpyAsync` + 要所の `cudaStreamSynchronize` で順序を保証する。
- **Band_DFT_Col:** ソルバコンテキスト構造体に stream メンバがなく、`cublasSetStream` も呼ばれない。cuBLAS はデフォルトストリームで動き、転送も同期 `cudaMemcpy` (syevdx ワークスペース照会専用の一時ハンドルのみ一時ストリームを使う)。
- **OpenACC との順序制御:** OpenACC の既定キューとライブラリ用ストリームは束ねていない (`acc_get_cuda_stream / async` 節は全コードで不使用)。OpenACC 管理データをライブラリに渡す直前に `#pragma acc wait`、直後に `cudaStreamSynchronize` を置いて明示同期する。

注意: ソルバ間でストリーム前提が異なるため、あるソルバのヘルパ関数を別ソルバに移植する際に同期規約を混ぜると競合の恐れがある。各コンテキストのハンドルは**プログラム正常終了時に明示破棄されない**が、デバイス常駐の行列バッファ・ワークスペース・キャッシュは、コミット `ae3dd189` 以降 **毎 SCF サイクルの力計算直前に一括返却される**ようになった (`DFT.c:2312-2319` の解放フック群: `Set_Hamiltonian_Release_OpenACC_DeviceCache` / `Divide_Conquer_Release_GPU_SCache` / `Krylov_Release_GPU_KUCache` / `Cluster_DFT_Col_Release_GPU_Solver` / `Cluster_DFT_NonCol_Release_GPU_Solver` / `Mixing_H_Release_GPU`)。これにより力計算の GPU バッチが「真の空き VRAM」を測って発動可否を判断できる。

3.3 デバイスメモリ管理

- **grow-only の永続バッファ:** `if (n <= ctx->matrix_dim) return;` の形で、必要サイズが既存以下なら再利用し、不足時のみ free→malloc する。cuSOLVER ワークスペースも同様。
- **有界リトライの導入:** 必須でない確保、および失敗時に CPU/ELPA フォールバックが用意されている確保は、無限リトライの `wait_cudafunc` ではなく **8 回上限のリトライ付き `TryDeviceMalloc` 系ヘルパ** (`ClusterCol_TryDeviceMalloc`、`ClusterNonCol_TryDeviceMalloc`、`MixH_TryDeviceMalloc` など)を使う。恒久的にメモリが足りない場合に「無言のフリーズ」ではなく失敗を返してフォールバックできる(コミット `ae3dd189` 以降の設計。§ 5.6)。
- **単一 `cudaMalloc` のアリーナ:** 非コリニアクラスタは対角化サイクル全体のバッファ群を 1 回の `cudaMalloc` で確保し (512 バイト整列の区画割り)、`acc_map_data` で OpenACC 側に見せる (`ClusterNonCol_DenseArena_TryEnsure`、`Cluster_DFT_NonCol.c:1613`)。予約が通れば**サイクル途中で他フェーズがデバイスを埋めても GO 判定が無効化されない**。DC 法の S キャッシュ、Krylov の KU キャッシュ、Mixing_H の履歴リングも同じアリーナ方式。
- **pinned メモリ:** ほぼ不使用。唯一 DC 法がホスト側に `cudaMallocHost` を使う。
- **共有 GPU への配慮:** Band 系の SCF 反復間バッファ常駐は既定 OFF (`OPENMX_BAND_GPU_PERSISTENT`)。コード内コメントは「16 GB を 8 ランクで共有すると GEMMu8 が FP64 フォールバックに追い込まれ、反復あたり 0.2 秒の節約より高かつ

く」と明記する。一方 Set_Hamiltonian の SCF 不変テーブルは、空き VRAM と他フェーズの公開需要を見て入るときだけ常駐する (§ 7.7)。

3.4 ソルバ別 GEMM エンジンの実像

コミット 66ce1e62 以降、GPU 上で実行される GEMM は全ソルバとも GEMMu18 ブリッジ経由に統一されている (cuBLAS FP64 が使われるのはブリッジ内フォールバックと DC 法の多段フォールバックのみ)。ただし定義だけ存在して呼ばれないヘルパが複数残るため、grep の見かけに惑わされないよう、実際に呼ばれる経路(ライブ経路)を表 3-2 に整理する。

ソルバ	ライブ経路の GEMM エンジン	備考
Band_DFT_Col	GEMMu18(<code>openmx_gemm18Zgemm</code> × 3、三重積変換と逆変換)	第 1 パス(all_knum≠1)は固有値のみ必要なため <code>jobz=NOVECTOR</code> で対角化し逆変換 GEMM を省く (§ 7.1)
Band_DFT_NonCol	GEMMu18(<code>BandNonCol_GEMMu18Zgemm_OpenACC</code> 経由 × 6)	永続ワークスペースの cublas ハンドル+ストリームを使用
Cluster_DFT_Col	GEMMu18(<code>ClusterCol_GEMMu18Dgemm_Device</code> 。H' 構成 × 2 + 逆変換 × 1)	DM/EDM 蓄積は GEMM ではなく OpenACC <code>device ptr</code> カーネル (§ 7.3)
Cluster_DFT_NonCol	GEMMu18(D/Z 合計 12 回の U + HU 変換 + 逆変換)	逆変換は明示 leading dimension 版 <code>ClusterNonCol_GEMMu18ZgemmLd_OpenACC</code> で m=MaxN に縮小(<code>f5de630e</code>)
Divide_Conquer(DC)	GEMMu18 → cuBLAS → CPU の多段(<code>DC_GpuSolver_TryGpuDgemm</code>)	GEMMu18 失敗時は以後そのランクで GEMMu18 を恒久無効化。承認済み GEMM 形状以下では preflight を省略(<code>6f7f31b2</code>)
Divide_Conquer_LNO(Col)	GEMMu18(<code>openmx_gemm18Dgemm</code> × 3)	—
Divide_Conquer_LNO(NonCol)	GEMMu18(<code>openmx_gemm18Zgemm</code> × 3: H・T、T†(HT)、逆変換)	<code>d62cac60</code> で新設。スピン倍化変換 T は Zdgmm + <code>cudaMemcpy2DAsync</code> のブロックコピーで構成 (§ 7.5)
Krylov	GEMMu18(<code>openmx_gemm18Dgemm</code> 。融合パスでは原子あたり 3 本: H・u1、u1†Hu1、逆変換)	3 本すべてが $m \cdot n \cdot k \geq 5 \times 10^7$ のときのみ融合パス発動(<code>3ee45228</code> 、 § 7.6)
Mixing_H(RMM-DIISH)	GEMM 不使用(OpenACC の Gram/最適残差カーネル)	GPU を使う混合は RMM-DIISH の Tier A のみ (§ 7.12)
DMmu/CWF/LNO 系 11 ファイル	GPU は対角化のみ(GEMM は従来どおり CPU)	各ファイルの GEMMu18 ヘルパは定義のみ(準備扱い)

表 3-2: ソルバ別 GEMM エンジン(ライブ経路と休眠コードの区別)

3.5 撤去された汎用ラッパと簡約ヘルパの注意

初期開発で使われた自己完結型 GEMM ラッパ `my_cublasDgemm.c` / `my_cublasZgemm.c` (呼び出しごとに `cublasCreate` → `cudaMalloc` / `cudaMemcpy` → GEMM → 破棄)は、長らく呼び出し箇所 0 件の休眠コードだったのち、コミット a7f8cba6 (Remove obsolete cuBLAS wrapper files)でソース・宣言・リンク対象とも完全に削除された。古いリビジョンを読む際のみ登場する名前である。

関連して注意すべき簡約が現行ヘルパにも残る: ソルバ内 GEMM ヘルパの多くは **alpha=1.0 / beta=0.0 固定**であり、lda/ldb の扱いは 2 系統ある。ライブヘルパの大半は転置に応じた lda/ldb 計算に対応済みで、非コリニアクラスタには明示 leading dimension 引数を持つ `ClusterNonCol_GEMMu18ZgemmLd_OpenACC` (`Cluster_DFT_NonCol.c:993`)が追加されたが、正方行列でのみ使われる旧型ヘルパや DMmu/CWF 系の休眠ヘルパには **lda=m・ldb=k のハードコード**が残っており、転置指定時に正しいのは正方行列(m=n=k)の場合だけである。非正方・転置で再利用する場合は ld の扱いを必ず確認すること。

3.6 エラー処理 — wait_cudafunc マクロ

CUDA/cuBLAS/cuSOLVER 呼び出しの大半が次のマクロで包まれる。

openmx_common.h:4135-4162(抜粋)

```
#define GPU_CPU_SWITCH_NUM 1000
#define WAITTIME (10.0 * 1.0E-6)

#define wait_cudafunc(call) \
    while (true) { \
        if (cudaSuccess == call) { \
            break; \
        } \
        /* pop the CUDA error latched by the failed attempt so it cannot be \
        misattributed by the next cudaGetLastError() caller once a retry \
        succeeds */ \
        (void)cudaGetLastError(); \
        double wait_time = drand48() * WAITTIME; \
        double start_time = MPI_Wtime(); \
        double current_time = start_time; \
        while ((current_time - start_time) < wait_time) { \
            current_time = MPI_Wtime(); \
        } \
    }
```

失敗時は 0~10 μ s のランダム待ち(ビジーウェイト)を挟んで**成功するまで同じ呼び出しを無限に再試行**する。成功コード 0 は `cudaError_t` / `cublasStatus_t` / `cusolverStatus_t` に共通なので 3 種の API を同一マクロで包める。これは複数ランクが同一 GPU を取り合う際の一時的な `cudaMalloc` 失敗を吸収するための設計である。

コミット `c5eb6845` で、失敗した試行ごとに `cudaGetLastError()` を呼んで **CUDA ランタイムのスレッド別エラーラッチを掃除**する行が追加された。これがないと、リトライで回復した確保失敗の エラーコードがラッチに残り、後続の無関係な `cudaGetLastError()` 利用者(Set_Hamiltonian の CUDA カーネルなど)が 自分の失敗と誤認して abort する「幻の失敗」が起こる(実際に SCF 316 反復目で発症した事例への対処。§ 7.7)。同じ理由で、DC 法の CPU フォールバック (`DC_GpuSolver_DisableGemmPath(Cuda)`)もフォールバック前にラッチを pop する。

計算失敗(固有値の収束失敗など)への対応はソルバ層で行われ、経路により `MPI_Abort` / `exit` か CPU フォールバックかが異なる(詳細は § 5.4 の表)。

3.7 cuBLAS 使用上の注意点

1. **wait_cudafunc は無限リトライ** — 引数の呼び出し式は毎周再評価される。恒久的エラー (不正な leading dimension、デバイス喪失など)を渡すとハング+副作用の反復になる。GPU 実行が「止まったように見える」ときは まずここを疑う。ただし主要な「満杯になり得る」確保は有界リトライの `TryDeviceMalloc` 系+フォールバックに 移行済みで、恒久的な VRAM 不足でハングする箇所は大幅に減った(§ 5.6)。
2. **簡約ヘルパの ld/alpha/beta 固定** — § 3.5 のとおり。一部ヘルパに固定が残り、転置 + 非正方の組み合わせで静かに誤結果になる。
3. **ストリーム前提の非対称性** — Band_DFT_Col はデフォルトストリーム+同期 memcpy、他は専用ストリーム+明示同期。コードを移植・共通化の際は同期規約を確認すること。
4. **エラーラッチの規約** — `cudaGetLastError()` を「自分のカーネルの検査」に使うコードを追加する場合は、直前に stale エラーを pop してから launch する(`set_hamiltonian_gpu.cu` の作法に倣う)。回復可能な失敗を握りつぶす他コンポーネントとの共存が前提のツリーである。

5. **行列サイズ 0 のガード** — GEMMu8 ブリッジは $m, n, k \leq 0$ で即成功を返す(全 GEMM がブリッジ経由のため、この一箇所で守られる)。
6. **cuBLAS ワークスペース未設定** — GEMMu8 が推奨する `cublasSetWorkspace` (32 MiB) は どのハンドルにも設定されていない(性能改修の候補。`gemmu8.hpp` のパフォーマンスノート参照)。
7. **列優先規約** — cuBLAS は Fortran BLAS と同じ列優先。OpenMX の実対称行列は対称性を利用して 行優先フラット配列をそのまま渡す箇所がある(転置と等価なため)。エルミート行列は明示的に列優先へ詰め替えるか、`cublasDgeam/Zgeam` によるデバイス内転置で整える(DC 法、DC-LNO 非コリニア)。

4. GEMMu8 の使い方

4.1 GEMMu8 とは — 原理と出自

GEMMu8(GEMMulate)は理化学研究所 計算科学研究センター(RIKEN R-CCS)が開発する「INT8/FP8 行列エンジンを用いた GEMM エミュレーション」ライブラリである(責任開発者: Yuki Uchino 氏、MIT ライセンス)。本実装が取り込んでいるのは **v3.0.4**(上流コミット `884a2875...` に固定)。上流リポジトリ: <https://github.com/RIKEN-RCCS/GEMMu8>。

原理は **Ozaki Scheme II**(整数モジュラー技法/中国剰余定理 CRT に基づく浮動小数点行列積エミュレーション)である。

1. **スケーリング+量子化**: FP64 行列 A, B を指数調整し、互いに素な法 $p = 256, 255, 253, 251, 247, 241, \dots$ (8 bit に収まる整数、最大 20 個)ごとの剰余をとって `num_moduli` 枚の INT8 行列に変換する。
2. **誤差なし整数行列積**: 法ごとに INT8 GEMM を実行する。実体は cuBLAS の `cublasGemmEx(..., CUDA_R_8I, ..., CUBLAS_COMPUTE_32I, ...)`、すなわち **INT8 Tensor Core(IMMA)**である。整数演算なので丸め誤差が入らない。
3. **CRT 復元**: 法別の結果から中国剰余定理で整数積を復元し、スケーリングを戻して $\alpha AB + \beta C$ を得る。

`num_moduli` (法の個数)を増やすほど高精度・低速になる。手順が決定的であるため、**同一設定なら結果は bitwise に再現する**(上流 README が明記)。GEMMu8 は、INT8 整数行列エンジンで DGEMM を行う先行研究 ozIMMU(Ootomo, Ozaki, Yokota 2024)系の Ozaki Scheme I を発展させた Scheme II の実装という位置づけである。引用すべき文献は巻末の参考文献を参照。

4.2 統合アーキテクチャ — 2 ファイル方式

OpenMX 本体は NVHPC `mpicc -std=c11` でビルドされる C コードであり、C++ テンプレート API を直接呼べない。また上流のビルドシステムは「ルーチン × 型 × バックエンド」ごとに大量のオブジェクトを生成する。そこで本実装は次の 2 ファイルで統合している。

(a) `gemmul8_openmx.cu` — 単一翻訳単位での明示的テンプレート実体化

上流のテンプレート定義ヘッダ(`gemm_impl.hpp`、`worksize_impl.hpp` と、上流では別オブジェクトに分かれるカーネル定義ヘッダ 7 本)を 1 つの翻訳単位に include し、OpenMX が必要とする **4 シンボルだけ**を明示的実体化する(67 行の小さなファイル)。

`gemmul8_openmx.cu:31-44(double 版、cuDoubleComplex 版と workSize 2 種も同様)`

```
template std::vector<double> gemm<double, Backend::INT8, double, double>(
    cublasHandle_t,
    cublasOperation_t, cublasOperation_t,
    size_t, size_t, size_t,
    const double *,
    const double *const, size_t,
    const double *const, size_t,
    const double *,
    double *const, size_t,
    int, bool,
    void *const,
    void *const,
    void *const,
    bool, bool, bool, bool);
```

実体化されるのは `gemm<double,INT8>`、`gemm<cuDoubleComplex,INT8>`、`workSize<false,INT8,Func::gemm>`、`workSize<true,INT8,Func::gemm>` の 4 本。このオブジェクトだけを `ar` で固めて `third_party/GEMMu8/lib/libgemmul8.a` を自

前生成する (上流の同名ライブラリとは中身が異なる)。この方式は上流の内部ヘッダ構成に依存するため、**上流コミットのピン留めが必須**である (Makefile のコメントに明記。§ 4.7)。

(b) `gemmul8_bridge.cu` — `extern "C"` ブリッジ

C 側から呼べる 5 関数を輸出する (プロトタイプは `openmx_common.h:4164-4173`)。

- `cublasStatus_t openmx_gemmul8Dgemm(handle, transa, transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc)` — **`cublasDgemm` と完全同型のシグネチャ**($\alpha/\beta \cdot ld$ とも任意)
- `openmx_gemmul8Zgemm(...)` — 複素版 (同型)
- `size_t openmx_gemmul8DWorkspaceSize(m, n, k) / openmx_gemmul8ZWorkspaceSize(m, n, k)` — **ワークスペース必要量の照会 API**。現在有効な `num_moduli` 設定で `gemmul8::workSize` を評価して返す (環境変数で当該精度が無効化されていれば 0)。各ソルバの VRAM 見積り (Band の k ターン計画、DC-LNO のグループ適合判定など) が GEMM ワークスペース分を正確に算入するために使う (コミット `58ea6471`, `d62cac60` で追加)
- `openmx_gemmul8ReleaseWorkspaces()` — 全ワークスペースの解放

ブリッジは (i) 環境変数の解釈、(ii) ワークスペースの照会・確保・キャッシュ、(iii) 確保できない場合の native cuBLAS へのフォールバック、(iv) `gemmul8::gemm` 呼び出し、を担う。

`gemmul8_bridge.cu` (`Dgemm` 本体の骨格。`Zgemm` も同構造)

```
if (gemmul8_disabled("OPENMX_GEMMUL8_DISABLE_D", "GEMMUL8_DISABLE_D")) {
    report.reason = "environment disable";
    log_workspace_fallback_once<false>(report);
    return cublasDgemm(handle, gemmul8_transa, gemmul8_transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc);
}

cublasStatus_t status =
    ensure_workspace<false>(handle, static_cast<size_t>(m), static_cast<size_t>(n), static_cast<size_t>(k),
                           num_moduli, &work, &report);
if (status == CUBLAS_STATUS_ALLOC_FAILED) {
    log_workspace_fallback_once<false>(report);
    return cublasDgemm(handle, gemmul8_transa, gemmul8_transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc);
}
if (status != CUBLAS_STATUS_SUCCESS) {
    return status;
}

(void)gemmul8::gemm<double, gemmul8::Backend::INT8>(handle, gemmul8_transa, gemmul8_transb, ..., num_moduli, fastmode, work);
```

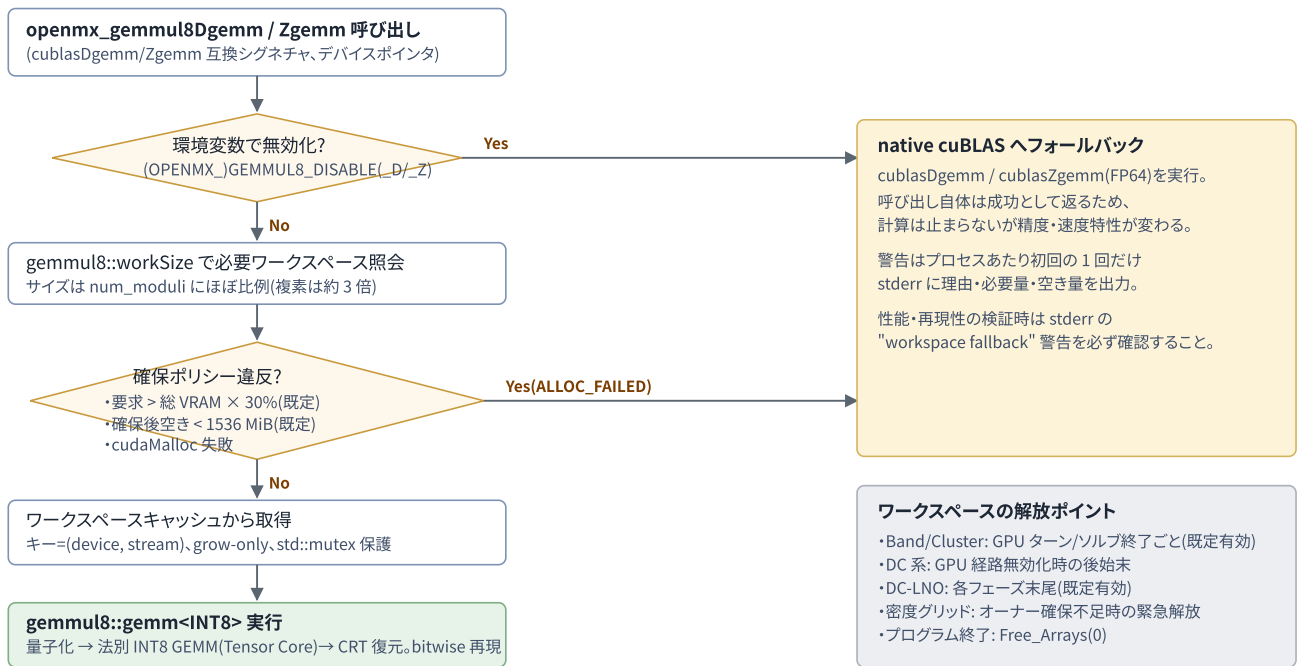


図 3: GEMM8 ブリッジのディスパッチフロー(gemmul8_bridge.cu)。

4.3 呼び出しパラメータと精度設定

パラメータ	既定値	環境変数(OPENMX_ 名が優先)	意味・挙動
<code>num_moduli</code>	15	<code>(OPENMX_)GEMMUL8_NUM_MOD_D / _Z</code>	法の個数(精度)。有効範囲 2~20、範囲外は黙って 15 に戻る。D/Z 独立に設定可
<code>fastmode</code>	false(accurate)	<code>(OPENMX_)GEMMUL8_FASTMODE_D / _Z</code> ("1" で有効)	高速スケーリング経路。精度が下がる
無効化	off	<code>(OPENMX_)GEMMUL8_DISABLE_ _D / _Z</code>	native cuBLAS FP64 に固定
WS 上限比率	30%	<code>(OPENMX_)GEMMUL8_MAX_WORKSPACE_PERCENT</code>	要求が総 VRAM のこの割合を超えたら拒否(Band_DFT_NonCol は未設定時に 50 を setenv する)
確保後最低空き	1536 MiB	<code>(OPENMX_)GEMMUL8_MIN_FREE_AFTER_MB</code>	確保後の空き VRAM がこの値を割るなら拒否

- 実数(D)経路は `CUBLAS_OP_C` を `CUBLAS_OP_T` に静かに正規化する(実数では共役転置=転置)。
- `m, n, k ≤ 0` は何もせず成功を返す。
- OpenMX 側に GEMM8 の結果を FP64 直接計算と比較・検証するコードは存在しない。フォールバックはすべてメモリ/環境変数条件によるもので、精度条件によるものではない。
- skip-scaling 等の上流拡張機能は未使用。フェーズ別時間の戻り値も捨てられる。

4.4 ワークスペース管理

- 照会:** 呼び出しごとに `gemmul8::workSize<is_complex, INT8>(m, n, k, num_moduli)` で必要量を計算。ソルバ側からは `openmx_gemmul8D/ZWorkspaceSize` で事前照会でき、各経路の VRAM 見積りに算入される。
- キャッシュ:** `(device, cudaStream_t)` をキーとする `std::unordered_map` でプロセス全体共有、`std::mutex` 保護、grow-only。cuBLAS ハンドルのストリームがキーになるため、**ハンドル(ストリーム)ごとに別領域**になる点に注意。
- サイズ感:** A 側 = num_moduli 枚の INT8 行列+シフトベクトル、B 側同様、C 側 = (num_moduli-1) 枚の中間行列 + 32 MiB の内部 BLAS 領域 + INT32 アキュムレータ。複素はさらに約 3 倍。n ≈ 1 万規模で約 **1 GiB/ランク**。

- **解放:** `openmx_gemm8ReleaseWorkspaces()` のみ。呼び出し箇所は (1) `Free_Arrays(0)` (プログラム終了時)、(2) `Band_DFT_Col` の GPU ターン終了ごと (`OPENMX_BAND_GEMM8_TURN_RELEASE=0` でオプトアウト)、(3) `Cluster_DFT_Col/NonCol` のソルブ終了ごと (`OPENMX_CLUSTER_GEMM8_TURN_RELEASE`、未設定時は `BAND` 設定を継承)、(4) DC 法の GPU 経路無効化時、(5) DC-LNO の各フェーズ末尾 (`OPENMX_DCLNO_GPU_TURN_RELEASE`)、(6) 密度グリッド GPU サービスがオーナー確保に失敗しそうなときの緊急解放。

4.5 複素数(Z)対応

OpenMX 側では複素行列を実行列に分解せず、`cuDoubleComplex` のまま `gemm<cuDoubleComplex, INT8>` に渡す。GEMM8 内部のネイティブ複素パスは、入力 1 個あたり低精度表現を 3 セット持ち、1 法あたり **実 INT8 GEMM 3 本** で複素積を構成する (4M 法ではなく 3M/Karatsuba 系。理論的背景は CRT に基づく複素行列積エミュレーションの論文、参考文献参照)。このため複素のワークスペースと演算量は実数の約 3 倍になる。

4.6 どの GEMM が GEMM8 で実行されるか

ブリッジ内に行列サイズによる振り分けは **ない** (GEMM8 が既定、フォールバックはメモリ/環境変数条件のみ)。コミット `66ce1e62` 以降は **GPU 上の全 GEMM がブリッジを経由する** (表 3-2)。代表例は `Band_DFT_Col` の一般化固有値問題変換 $H' = \tilde{U} + H\tilde{U}$ である (ここが最大の GEMM 負荷)。

`Band_DFT_Col.c` — 三重積変換 (2 本目までを抜粋。逆変換 1 本が続く)

```
wait_cudafunc(openmx_gemm8Zgemm(ctx->cublas, CUBLAS_OP_N, CUBLAS_OP_N, n, n, n, &alpha,
                                (cuDoubleComplex *)ctx->d_H, n, (cuDoubleComplex *)ctx->d_S, n, &beta,
                                (cuDoubleComplex *)ctx->d_tmp, n));

wait_cudafunc(openmx_gemm8Zgemm(ctx->cublas, CUBLAS_OP_C, CUBLAS_OP_N, n, n, n, &alpha,
                                (cuDoubleComplex *)ctx->d_S, n, (cuDoubleComplex *)ctx->d_tmp, n, &beta,
                                (cuDoubleComplex *)ctx->d_H, n));
```

4.7 ビルド

- **取得:** `make` が git submodule ディレクトリで `git fetch --depth 1 origin $(GEMM8_COMMIT)` → `checkout` を実行し、コミット `884a2875...` に固定する。チェックアウトが無い新規クローンでも `gemm8.hpp` の order-only ルール経由で自動取得される (コミット `9fe8ce13`)。ピン留めの理由は Makefile コメントに明記: 「gemm8_openmx.cu が GEMM8 内部を直接実体化するため、チェックアウトは内部レイアウトが一致するコミットに固定し続けなければならない」。
- **コンパイル:** NVHPC 同梱 CUDA 13.1 の `nvcc`、ホストコンパイラ `-ccbin nvcc++`、`-std=c++20 -O3 -diag-suppress 177 -DGPU_ARCH=$(arch) -gencode arch=compute_$(arch),code=sm_$(arch)`。アーキは `nvidia-smi --query-gpu=compute_cap` から自動検出 (本開発環境では `compute_120/sm_120`)。 `GEMM8_GPU_ARCH=90` のように手動指定も可能。
- **環境サニタイズ:** `NVCC_GEMM8_ENV` が `CPATH` 等を空にして NVHPC のヘッダ・フラグ混入を遮断。
- **リンク追加:** `-lcublasLt -lcuda -lnvidia-ml -lstdc++`。

4.8 GEMM8 使用上の注意点

1. **メモリ使用量** — ワークスペースは `num_moduli` にほぼ比例、複素は約 3 倍、 $n \approx 1$ 万で約 1 GiB/ランク。複数ランク共有 GPU では「30% 上限・1536 MiB 予約・ターン毎解放」の 3 段階で枯渇を防いでいる。さらに各ソルバの並列度プランナが `openmx_gemm8D/ZWorkspaceSize` でワークスペース分を事前算入するため、既定値を変える場合は見積りとの整合を保つこと。
2. **静かな FP64 フォールバック** — ワークスペース拒否時は警告 1 回だけで `cuBLAS FP64` に切り替わり、**速度と数値結果 (ビットパターン) の両方が変わる**。「なぜか遅い/結果が揺れる」ときは `stderr` の "GEMM8 workspace fallback" を確認

- 認する。決定的な再現が必要な検証では `OPENMX_GEMMUL8_DISABLE=1` で全無効化するか、メモリに十分な余裕を持たせる。
3. **精度設定** — 既定 15 moduli・accurate モード。範囲外の指定は黙って 15 に戻る。fastmode は高速だが スケーリング精度が落ちる。D と Z は独立に調整できる。
 4. **初回呼び出しコスト** — 初回およびサイズ増加時に GiB 級の `cudaMalloc` が同期的に走る。ベンチマークは 2 回目以降の反復で評価すること。
 5. **ビルドのハマりどころ** — (a) コミットピンを外すと内部ヘッダ構成の変更で `gemmul8_openmx.cu` が壊れる。(b) NVHPC 環境変数(CPATH 等)を掃除しないと nvcc が NVHPC ヘッダを拾って失敗し得る。(c) `-lcublasLt`・`-lstdc++` のリンク忘れ。(d) アーキ自動検出は `nvidia-smi` が使える環境前提。(e) 単一アーキ `-gencode` のため別世代 GPU ではバイナリ非互換(要再ビルド)。
 6. **上流 API 追従** — 旧 API から現行構造(884a287)への移行(コミット `cb4d348d`)で `gemmul8_openmx.cu` は 149 行 → 67 行に簡素化された。ブリッジの公開シグネチャは不変で、C 側への影響はない。今後の上流更新時も「単一 TU が include する定義ヘッダの構成」を最初に確認するとよい。
 7. **ハードウェア要件** — INT8 Tensor Core(IMMA)必須。C++20 対応 nvcc(CUDA 12 系以降)が事実上必要 (検証済みは CUDA 13.1 のみ)。

5. cuSOLVER(gpuserver 層)の使い方

5.1 層の構成と命名の意図

固有値計算は cuSOLVER を **gpuserver** というベンダー中立名のラッパ層経由で使う。層は次の 3 階層で構成される。

1. **汎用ラッパ関数**(1 呼び出し完結型): `gpuserver_Syevd.c` / `gpuserver_Syevdx.c` の 6 関数。ホスト配列版と、OpenACC デバイス常駐配列を直接処理する `_openacc` 版、さらにハンドル・ワークスペースを呼び出し間でキャッシュする `_cached` 版がある(表 5-1)。冒頭のライセンスヘッダが示すとおり、NVIDIA CUDA Library Samples の `syevd/syevdx` サンプルを改変した出自を持つ。
2. **共通ユーティリティヘッダ**: `openmx_gpuserver_dense_utils.h` (79 行)。`OpenMX_GpuSolver_DenseDsyevx/DenseZheevx` の 1-based/0-based 版 4 関数を static 定義し、`info != 0` なら `MPI_Abort` する。分極・DMmu・CWF・LNO 系の 12 ファイルが include する。
3. **ソルバ内常駐コンテキスト**: `BandColGpuSolverCtx`、`ClusterColGpuSolverCtx`、`DCGpuSolverCtx`、`DCLNO_GpuSolverZCtx`、Krylov のスレッド別ワークスペースプールなど。ハンドル・デバイスバッファをキャッシュし、cuSOLVER API を直接呼ぶ(第 3 章と共通の構造)。

命名の意図: ベンダー中立化は「命名規約による抽象化」であり、マクロや関数ポインタによる切替機構はまだ存在しない。関数名・構造体メンバ名・入力キーワードを gpuserver に統一し(例: `cusolverDnHandle_t gpuserver;`)、cuSOLVER の型名・API 名は NVIDIA ビルドの実装詳細としてそのまま残す。AMD(hipSOLVER/rocSOLVER)対応は 同一の命名規則の下で別途実装する想定である。入力キーワードの旧名 `cusolver` は警告付きの非推奨エイリアスとして残る。

関数(gpuserver_Syevd(x).c)	cuSOLVER API	データ所在	現在の呼び出し元
<code>gpuserver_Syevd(A,W,m)</code>	Xsyevd(全固有値)	ホスト⇄デバイス 転送内蔵	休眠 呼び出し元なし
<code>gpuserver_Syevdx(A,W,m,MaxN)</code>	Xsyevdx(下から MaxN 本)	同上	<code>Eigen_lapack.c</code> 、 <code>dense_utils</code> 経由多数
<code>gpuserver_Syevdx_openacc(...)</code>	Xsyevdx	OpenACC 常駐 (present + host_data)	<code>Eigen_lapack.c</code>
<code>gpuserver_Syevdx_Complex(...)</code>	Xsyevdx(CUDA_C_64F)	ホスト⇄デバイス 転送内蔵	<code>EigenBand_lapack.c</code> 、 <code>dense_utils</code> 経由 多数
<code>gpuserver_Syevdx_Complex_openacc(...)</code>	Xsyevdx(CUDA_C_64F)	OpenACC 常駐	<code>EigenBand_lapack.c</code>
<code>gpuserver_Syevdx_Complex_openacc_cached(...)</code>	Xsyevdx(CUDA_C_64F)	OpenACC 常駐	DC 非コリニア・DC-LNO 非コリニアの毎 原子対角化(コミット <code>6f7f31b2</code> で新設。 static コンテキストにハンドル・ストリー ム・grow-only ワークスペース・d_info を 保持し、原子ごとの create/destroy を排 除)

表 5-1: 汎用ラッパの公開 API

5.2 使用 API の詳細

- すべて **generic(X)API・64-bit 整数版** を使用: `cusolverDnXsyevd(_bufferSize)` / `cusolverDnXsyevdx(_bufferSize)`。legacy API(`cusolverDnDsyevd` 等)や `Zheevd` 系の別名 API は一切使わず、複素も Xsyevdx に `CUDA_C_64F` を渡す統一形。一般化固有値ソルバ `cusolverDnXsygvd` も使わない(§ 5.3)。
- 共通パラメータ: `params = NULL`、`uplo = CUBLAS_FILL_MODE_LOWER`、範囲指定は `CUSOLVER_EIG_RANGE_I` (`il=1`, `iu=MaxN`)で、`m == MaxN` のとき `RANGE_ALL` に切り替える実装と常に `RANGE_I` の実装が混在する。`jobz` は原則

CUSOLVER_EIG_MODE_VECTOR だが、Band(Col)の第 1 パス(固有値のみ必要)は CUSOLVER_EIG_MODE_NOVECTOR で解いて逆変換 GEMM ごと省略する (BandCol_EigenvaluesOnlyNoVectors()、OPENMX_BAND_SYEVDX_NOVECTOR=0 で無効化)。

- ハンドル: 汎用ラッパは呼び出しごとに cusolverDnCreate → non-blocking ストリーム生成 → 計算 → 全解放。_cached 版と常駐コンテキストはキャッシュする(§ 3.2)。
- ワークスペース: 全経路で _bufferSize によりデバイス/ホスト両方の必要量を照会してから確保する。常駐側は grow-only キャッシュ。さらに並列度プランナも **実際に _bufferSize を照会して** syevdx ワークスペース分を VRAM 見積りに算入する(照会失敗時は $4n^2$ 複素要素で代用。§ 5.5)。

gpusolver_Syevdx.c — 中核の呼び出し形(64-bit generic API)

```
wait_cudafunc(cusolverDnXsyevdx_bufferSize(cusolverH, NULL, jobz, range, uplo, m, CUDA_R_64F, d_A, lda, &vl, &vu,
                                            1L, MaxN, &h_meig, CUDA_R_64F, d_W, CUDA_R_64F,
                                            &workspaceInBytesOnDevice, &workspaceInBytesOnHost));

wait_cudafunc(cudaMalloc((void **>(&d_work), workspaceInBytesOnDevice));
h_work = (workspaceInBytesOnHost == 0) ? NULL : malloc(workspaceInBytesOnHost);
...
wait_cudafunc(cusolverDnXsyevdx(cusolverH, NULL, jobz, range, uplo, m, CUDA_R_64F, d_A, lda, &vl, &vu, 1L, MaxN,
                                &h_meig, CUDA_R_64F, d_W, CUDA_R_64F, d_work, workspaceInBytesOnDevice, h_work,
                                workspaceInBytesOnHost, d_info));
```

5.3 一般化固有値問題 $Hc = \epsilon Sc$ の処理

cuSOLVER の一般化ソルバは使わず、OpenMX 従来の **S の固有分解によるレイディン(対称)直交化**を各経路が GPU 上で自前実装する。Cholesky 分解は一切使わない。手順は共通して次の 6 ステップである。

1. S を Xsyevdx(全固有値)で対角化 → 固有ベクトル U、固有値 λ
2. 小さい/負の固有値の処置(**経路により異なる**: Band/Cluster は 1×10^{-10} に切上げ、DC は OLP_eigen_cut 未満を切り捨て、DC-LNO は fabs)
3. U の列を $1/\sqrt{\lambda}$ でスケール(cublasDdgm/Zdgm) → $\tilde{U}$ 。DC-LNO 非コリニアはさらに cudaMemcpy2DAsync のブロックコピーでスピン倍化変換 $T = \text{diag}(\tilde{U}, \tilde{U})$ を構成する
4. $H' = \tilde{U}^\dagger H \tilde{U}$ を GEMM 2 回で構成(GEMM_{ul8}。表 3-2)
5. H' を Xsyevdx(il=1..MaxN)で対角化 → 固有値 ϵ 、固有ベクトル z
6. 逆変換 $c = \tilde{U}z$ を GEMM で実行(固有値のみ必要な場面では省略)

Cluster_DFT_Col.c — S 対角化と $1/\sqrt{\lambda}$ スケーリング(GPU 上のレイディン直交化)

```
if (rebuild || !(ctx->transformed_s_valid && ctx->transformed_s_dim==n)){
    ClusterCol_GpuSolver_EigenDevice(ctx->d_S, n, ko0+1);

    for (int l=1; l<=n; l++){
        if (ko0[l]<1.0e-10) ko0[l] = 1.0e-10;
        ko0[l] = 1.0/sqrt(ko0[l]);
    }

    wait_cudafunc(cudaMemcpyAsync(ctx->d_W, ko0+1, sizeof(double)*(size_t)n, cudaMemcpyHostToDevice, ctx->stream));

    old_s = ctx->d_S;
    new_s = ctx->d_tmp;
    wait_cudafunc(cublasDdgm(ctx->cublas, CUBLAS_SIDE_RIGHT, n, n,
                            old_s, n, ctx->d_W, 1, new_s, n));
```

注意(数値的挙動の差): S の小さい固有値の処置(切上げ / 切捨て / fabs)が経路間で統一されていないため、悪条件の重なり行列を持つ系では、同じ物理系でもソルバ経路によって数値挙動が微妙に異なり得る。

部分固有値数 MaxN の決め方: Band(Col)は $\text{MaxN} = (\text{TZ-charge})/2 + \max(0.4 \cdot \text{Ne}/2, 100)$ 、Cluster は SCF 第 1 反復で全数、以後 $(\text{TZ-charge})/2 + \max(0.1n \sim 0.2n, 400)$ 。DC-LNO は SCF 2 反復まで全数、以後 $\text{HO_TC} + 100$ 。占有帯+余裕分だけを解くことで syevdx の計算量を抑えている。

5.4 devInfo の検査と失敗時処理(経路別)

`d_info` は全経路でデバイスに確保し、計算後にホストへコピーして検査するが、**失敗時の方針は経路ごとに異なる**。なお VRAM 不足による発動不可は info 失敗より前の段階 (事前見積り/実予約)で検出され、ELPA/CPU にフォールバックする (§ 5.6)。ここでの表は「GPU 経路に入った後の計算失敗」の扱いである。

経路	info 検査	失敗時の挙動
汎用ラッパ(gpusolver_Syevdx 等)	呼び出し元に返すのみ	呼び出し元依存
dense_utils ヘッダ経由(11 ファイル)	あり	即 <code>MPI_Abort</code>
<code>Eigen_lapack.c</code>	info<0 のみ exit(10)	info>0(収束失敗)は素通り。ただし上位の <code>Eigen_lapack()</code> に固有ベクトル直交性検査つきリトライがあり、失敗時はソルバを CPU 系へ巡回して再計算 (最大 3 回)
<code>EigenBand_lapack.c</code>	あり	CPU の Householder 法 <code>Eigen_HH</code> へフォールバック
Band_DFT_Col / Cluster_DFT_Col / Cluster_DFT_NonCol	あり(Band は h_meig も)	即 abort(exit / <code>MPI_Abort</code>)。ただし VRAM 起因の発動不可は事前に ELPA へ逃げるため、ここに来るのは数値的失敗のみ
Divide_Conquer(DC)	あり(h_meig も)	sticky 無効化フラグ を立てて以後そのランクは CPU の DC ソルバへ。GEMMu8 ワークスペースも解放
Divide_Conquer_LNO(Col/NonCol)	あり	exit / abort(フォールバックなし。ただし発動前にデバイス共有ランク群の合算メモリ判定があり、不足時は最初から CPU 固有値ソルバを使う)
Krylov	あり	CPU LAPACK(<code>Eigen_lapack2</code>)へその場でフォールバック(融合パスでは結果を再アップロードして続行)

表 5-2: devInfo 検査と失敗時処理の経路差

5.5 適応並列度制御 — UUID 共有群とプレフィックススキャン

第 2 版時点の並列度制御は「固定グループ幅(既定 4)の GPU ターン直列化」と「固定はしご(32→16→…→1)」だったが、現在は**全経路が実測ベースの適応制御**に統一されている(コミット `58ea6471` , `7c414f91` , `48921b36` , `99f4014b` , `6b84a1f9`)。共通の骨格は次のとおり。

- 共有群の検出:** `MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)` でノード内コミュニケータを作り、`cudaGetDeviceProperties` の GPU UUID(16 バイト)を Allgather して、同一物理 GPU を共有するランク群のコミュニケータ(device_comm)に分割する。`cudaSetDevice` の番号ではなく UUID で照合するため、`CUDA_VISIBLE_DEVICES` の設定差があっても正しく共有を検出できる。
- 空き VRAM の合意:** 計測前に `acc_wait_all(); cudaDeviceSynchronize(); acc_clear_freelists();` で各プロセスの OpenACC フリーリストを CUDA に返却してから `cudaMemGetInfo` を全員で実行し、`MPI_Allreduce(MIN)` でグループ共通の空き(group_free)を得る。
- per-rank 所要量モデル:** 密行列・ベクトル・syevdx ワークスペース(`_bufferSize` の実照会値)・GEMMu8 ワークスペース(`openmx_gemmul8D/ZWorkspaceSize`)・疎→密構築/DM 転送域・ランタイム オーバーヘッド(既定 256 MiB、`OPENMX_BAND_GPU_RANK_OVERHEAD_MB`)を合算する。
- プレフィックススキャン:** 各ランクの所要量を Allgather して降順ソートし、上位 c ランクの合計 peak が `peak ≤ group_free` かつ `reserve ≤ group_free - peak` を満たす**最大の c** を選ぶ (reserve = max(総 VRAM の 1/10, 1536 MiB)、`OPENMX_BAND_GPU_RESERVE_MB` で引き上げ可)。初期候補はデバイス共有ランク数(旧版の固定はしごと暗黙の上限 32 は撤廃され、環境変数は「明示上限」の意味だけになった)。

5. **合意と縮退:** 結果を `MPI_Allreduce(MIN)` で全ランク一致に揃える。全ランクが同時に収まる場合は ターン分割そのものを畳み込み(バリアと GEMM_{ul8} 解放点が消える)、1 ランクも収まらない場合は ELPA フォールバック(§ 5.6)。
6. **需要の公開:** フル並列時の必要総量を `OpenMX_GpuPhaseNeed_Register("band_col" など)` で公開し、`Set_Hamiltonian` の常駐キャッシュがこの分を空けて動く(§ 7.7)。

5.5.1 Band(Col) — BandCol_AutoGpuTurnLimit

所要量モデルは `BandCol_GpuTurnRequiredBytes(n, maxn, size_H1)` (`Band_DFT_Col.c:429`): 密行列 3 枚($3 \cdot n^2 \cdot 16 \text{ B}$) + ベクトル + syevdx ワークスペース($(n,n)/(n,maxn)$ の 2 形状を実照会してメモ化) + GEMM_{ul8} Zgemm ワークスペース + max(構築転送域, DM 転送域) + オーバーヘッド。決定された並列度は次の 1 行で報告される(値が変わったときだけ、デバイス代表ランクが出力):

```
<Band_DFT_Col> GPU device %d: %d k-owner rank(s), GPU concurrency=%d, turn group=%d,
per-rank=%.3f GiB, peak=%.3f GiB, free=%.3f GiB, reserve=%.3f GiB
```

`all_knum==1` 側の GPU ターン直列化(`OPENMX_GPUSOLVER_SERIAL_GPU_TURNS`、既定 1)は温存されており、コード内コメントは「8 オーナーランク/1 GPU では並行投入より約 27% 高速」のまま。明示上限は `OPENMX_BAND_GPU_MAX_CONCURRENT_K` → `OPENMX_BAND_GPU_TURN_GROUP` → `OPENMX_BAND_GPU_MAX_RANKS_PER_DEVICE` の優先順で読まれる。

5.5.2 Band(NonCol) — ステージ別の 3 モード見積り

非コリニアバンドは 1 ターンの中でメモリピークが大きく変わるため、見積りを **固有値のみ(EIGVALS)/固有ベクトル(EIGVECS)/密度行列(DM)** の 3 モードに分けて別々に並列度を定める (`BandNonCol_RootDenseDeviceBytes` / `BandNonCol_RootDenseDMDeviceBytes`、`Band_DFT_NonCol.c:803,871`)。DM 蓄積の所要量はソルブ段よりはるかに小さいため、たとえば「ソルブは 2 ランクずつ、DM 蓄積は 8 ランク同時」のような 運転が可能になった(コミット `7c414f91`)。報告行はモード名付き:

```
<Band_DFT_NonCol> GPU device %d: %d k-owner rank(s), GPU concurrency=%d (eigenvalues only |
eigenvectors | density matrix), turn group=%d, per-rank=%.3f GiB, ...
```

複数 k 世界の同時実行可否も同じ枠組みで判定され、収まらなければ「Serializing non-collinear dense GpuSolver k-worlds」と表示して直列化する。また `BandNonCol_SetDenseGemmul8Defaults` が `OPENMX_GEMMul8_MAX_WORKSPACE_PERCENT` 未設定時に 50 を setenv する。

5.5.3 クラスタ(Col/NonCol) — 実予約方式

クラスタ経路は k ターンを持たない(root-dense の単一オーナー)ため、並列度制御の代わりに「**見積り**」ではなく「**実予約**」で発動可否を決める。オーナーランクが行列バッファと syevdx ワークスペース(`_bufferSize` の実報告値)を **8 回上限のリトライ付き cudaMalloc で実際に確保**し、さらに `OPENMX_CLUSTER_GPU_DIAG_RESERVE_MB` (既定 1024 MiB)が空きに残ることを要求する(`ClusterCol_OwnerReserveProbe`)。実予約方式に変えた背景には、「syevdx ワークスペースの見積り $2n^2$ doubles に対し実際は約 $4n^2$ doubles だった」という実測がある(コミット `cff323e6`)。非コリニアは対角化サイクル全体を **単一 cudaMalloc のアリーナ**として予約し、`acc_map_data` で OpenACC に見せるため、予約成立後は他フェーズの動きに影響されない(`ClusterNonCol_DenseArena_TryEnsure`)。

スピン分極(SpinP_switch==1)で 2 つのスピンワールドのオーナーが同時に収まらない場合は、**スピンワールド 0 のオーナーが両スピンを直列に解く**第 3 の運転モードがある(コミット `bf08a9d8`、`OPENMX_CLUSTER_GPU_DIAG_SERIAL` 既定 1、2 で直列強制)。このときリモートスピンの固有値・固有ベクトルは MPI(タグ 997/998、64M 要素ずつ分割)で相手オーナーへ送られる。それも不可なら ELPA へ(§ 5.6)。

5.5.4 DC-LNO — デバイス共有ランク群の合算判定

DC-LNO の direct per-rank dispatch は全ランクがソルババッファを常駐させるため、`DCLNO_GpuGroupMemoryFits ( Divide_Conquer_LNO.c:1012 )`が「 $7 \cdot \text{elem} \cdot \text{Max_Msize}^2 + \text{GEMMu8}$ ワークスペース + 256 MiB」をランク合算(Allreduce SUM)し、空き MIN と比較して不足なら**グループ全体で CPU 固有値ソルバに切り替える** (18 ランク NC 実行が 16 GiB カードに 26.1 GiB を要求して 40 分スピンした事例への対処)。通知は代表ランクが 1 回:

```
<DC-LNO> GPU device %d: %d rank(s) would need %.3f GiB (free %.3f GiB, reserve %.3f GiB);
using the CPU eigensolver instead.
```

5.6 実行時 ELPA フォールバック

第 2 版までは「GPU 経路に入ってから VRAM 枯渇は abort かハング」だったが、コミット `ae3dd189` / `cff323e6` / `3ef5347a` / `a4200a04` で **バンド・クラスタの全 4 経路に実行時 ELPA/ScaLAPACK フォールバック**が入った。共通仕様:

- 判定関数: `BandCol_GpuDenseFits` / `BandNonCol_GpuDiagFits` (1 ターン分の見積り)、`ClusterCol_GpuDiagFits` / `ClusterNonCol_GpuDiagFits` (実予約)。
- 判定(verdict)は `mpi_comm_level1` の `MPI_Allreduce(MIN)` による**集団合意**。GPU/ELPA の分岐は集団通信の構造を変えるため、全ランクが必ず同じ側に落ちる。
- verdict は static にキャッシュされ、SCF リスタートが行列サイズ変化まで再判定しない(sticky)。
- フォールバック時は予約済みバッファを全解放してから従来の ELPA/ScaLAPACK コードへ落ちる。通知は 1 回だけ:

```
<Cluster_DFT_Col> Falling back to the ELPA/ScaLAPACK diagonalization. Force the GPU path with
OPENMX_CLUSTER_GPU_DIAG=1 or lower OPENMX_CLUSTER_GPU_DIAG_RESERVE_MB.

<Band_DFT_Col> A k-owner rank cannot fit even one k-point's dense GPU diagonalization
(%.1f MiB needed per rank, %.1f MiB free); falling back to the ScaLAPACK/ELPA diagonalization.
Force the GPU path with OPENMX_BAND_GPU_DIAG=1 or lower OPENMX_BAND_GPU_RESERVE_MB.
```

- 強制ノブ: `OPENMX_BAND_GPU_DIAG=1/0`、`OPENMX_CLUSTER_GPU_DIAG=1/0` (1 = 判定を無視して GPU 強制(予約失敗時は明確な診断つき abort)、0 = 常に ELPA)。
- 必須確保が本当に尽きた場合の abort は、無言のハングではなく所要量・空き量つきの診断になる: `"Cluster_DFT_Col.c: out of GPU memory while allocating %s (%.1f MiB requested, %.1f MiB free of %.1f MiB). Reduce the MPI ranks sharing the device or set OPENMX_CLUSTER_GPU_DIAG=0 ..."`

設計の意図: 旧実装では root-dense の行列確保が無限リトライ(`wait_cudafunc(cudaMalloc(...))`)だったため、満杯のデバイスでオーナーがスピンし、他の全ランクが `MPI_Bcast` で待つ「ノード全体の無言フリーズ」が起きた ($n=13,000 \cdot np18 \cdot 16$ GiB GPU で実測)。有界リトライ+実予約+集団合意+ELPA フォールバックはこのクラスの 障害を仕様上排除する(コミット `ae3dd189`)。

5.7 MPI ランク → GPU デバイス割当

- collective 版** (`set_cuda_default_device_from_local_rank.c`): `MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)` でノード内 local rank を求め、`local_rank % deviceCount` を `cudaSetDevice`。全ランクが揃って呼ぶ必要がある(片側だけ呼ぶとデッドロック)。
- noncollective 版**: SCF 内ホットパス用。環境変数 `OMPI_COMM_WORLD_LOCAL_RANK` → `MV2_COMM_WORLD_LOCAL_RANK` → `SLURM_LOCALID` → `PMI_LOCAL_RANK` の順で local rank を取得。どれも無ければ大域 rank で代用(この場合マルチノードでは割当が偏り得る)。

- **OpenACC 側も対で設定** (`set_openacc_device_from_local_rank.c`): 同じ規則で `acc_set_device_num`。CUDA と OpenACC の両ランタイムに同一デバイスを設定することが必須という設計。
- 割当後の「実際にどのランクがどの物理 GPU を共有しているか」は、各フェーズが UUID の Allgather で自律的に検出する (§ 5.5)。

5.8 LU_Zinverse_GPU — NEGF 向けの休眠実装

休眠コード: 複素 LU 逆行列の GPU 版 `LU_Zinverse_GPU` (`Lapack_LU_inverse.c:347-444`) は、cuSOLVER ではなく **cuBLAS のバッチ API** `cublasZgetrfBatched` / `cublasZgetriBatched` (`batch=1`) で実装されている。A は OpenACC `deviceptr`、特異行列検出時は `abort`、結果は `acc kernels` で書き戻す。しかし公開関数 `Lapack_LU_Zinverse` からの呼び出しは `/* LU_Zinverse_GPU(n, A, handle); */` とコメントアウトされており、既定は従来の CPU 実装 (LAPACK `zgetrf/zgetri`) のまま。呼び出し元は NEGF の表面グリーン関数計算 (`TRAN_Calc_SurfGreen.c` ほか) であり、**将来 NEGF 系の複素逆行列を GPU 化するための布石**である。

5.9 cuSOLVER(gpuserver 層)使用上の注意点

1. **ワークスペース量の見積り** — $n \times n$ 複素行列 1 本で $16n^2$ バイト、`syevdx` ワークスペースは見積りでなく `_bufferSize` の実照会値を使うこと(実測で見積り $2n^2$ に対し実際 $4n^2$ だった例がある)。各プランナのモデル (§ 5.5) を流用するのが確実。
2. **`wait_cudafunc` の無限ループリスク** — § 3.7 と同じ。恒久的エラーではハングする。「満杯になり得る」確保には `TryDeviceMalloc` 系(8 回上限)を使うのが現行の作法。
3. **`devInfo` 検査の不均一** — 表 5-2 のとおり、収束失敗からの回復可能性が経路により異なる。Band/Cluster/DC-LNO/dense_utils 系は「数値的失敗=即終了」である(VRAM 不足は事前に ELPA へ逃げる)。
4. **フォールバックの階層** — (a) デバイス不在: 起動時に全体が ELPA2 へ降格。(b) VRAM 不足: 経路別の fits 判定で ELPA/CPU へ (§ 5.6)。(c) 数値的失敗: 表 5-2。この 3 層を混同しないこと。
5. **対称性と 0/1-based の変換** — 全呼び出しが `uplo=LOWER` の対称/エルミート前提。固有値配列の 0/1-based ずらし (`W[l]=W[l-1]`) が各所で必要で、dense_utils が吸収している。
6. **S の条件数処理が経路で不統一** — § 5.3 の警告参照。
7. **再現性** — 変換 GEMM は GEMMu8 で実行されるため丸め挙動が cuBLAS FP64 と異なり、メモリ状況によるフォールバックで実行ごとに経路が変わり得る(対策は § 4.8-2)。加えて GPU $\rightleftharpoons$ ELPA のフォールバックでも結果のビットパターンは変わる。`verdict` は sticky なので 1 回の SCF 内では揺れない。
8. **fits 判定は集団操作** — 判定関数の内部で `MPI_Comm_split_type` / `Allreduce` / `MPI_Barrier` を呼ぶため、**全ランクが同じ回数だけ到達する必要がある**。経路を改修する際に片側のランクだけ判定をスキップさせるとデッドロックする。

6. OpenACC と生 CUDA の使い分け

6.1 全体像

ループ並列のオフロードは NVHPC の OpenACC(`-acc -Minfo=accel`)で記述されている。要点:

- **GPU 専用の `#ifdef` ガードは存在しない。**CUDA ヘッダ(`cublas_v2.h` 等)は `openmx_common.h` で無条件 include されるため、**このツリーは CPU 専用にはビルドできない**(CPU 用はオリジナルまたは `makefile.cpu_intel.bak` を使う)。ゲートはすべて実行時フラグである。
- `-gpu=ccXX` 指定はなく、ターゲット CC はビルドマシンの GPU 自動検出に依存する。**managed memory は不使用。**
`async` 節・`acc_get_cuda_stream` も全コードで不使用。
- `#pragma acc` を持つファイルは約 30。最厚は `Divide_Conquer.c` (66 箇所)、`Band_DFT_NonCol.c` (61)、`Force.c` (49)、`Band_DFT_Col.c` (46)、`Cluster_DFT_NonCol.c` (43)、`Set_ProExpn_VNA.c` (21)。**Krylov.c は OpenACC ゼロ**(`-acc` なしでコンパイルされ、生 CUDA のみ)。
- 基本形は `parallel loop gang` + `loop vector reduction`。密度行列構築・格子トレースなど再現性が重要な縮約は `loop seq` (逐次)や「float 積 → double 加算」のキャスト位置まで含めて CPU と加算順序を一致させる。

6.2 データ常駐の 4 パターン

1. 呼び出し毎の一時 data 領域(`copyin/copyout`) — `Total_Energy_GPU`、`Set_Nonlocal` など。OpenMX の多次元ポインタ配列はデバイスで辿れないため、**必ずホスト側で 1 次元フラット配列に詰め替えてから転送する**設計で統一されている。
2. `enter data / exit data + update` による SCF スコープ常駐 — `Band_DFT_NonCol` の root-dense 経路、`Set_Hamiltonian` の SCF 不変テーブル(`present_or_copyin` 意味論により、常駐済みなら転送されない。§ 7.7)。
3. 生 `cudaMalloc` + `deviceptr` ブリッジ — `Band_DFT_Col.c` / `Cluster_DFT_Col.c` は static コンテキストの `d_H/d_S` (`cudaMalloc` 領域)に対し、OpenACC 側から `#pragma acc parallel loop deviceptr(d_H) + acc atomic update` で疎→密の scatter-add を実行する。
4. 単一確保のアリーナ + `acc_map_data / deviceptr` — 現行実装で最も新しいパターン。複数のバッファを 512 バイト整列で 1 つの `cudaMalloc / acc_malloc` 領域に区画割りし、OpenACC には `acc_map_data` (`Cluster_DFT_NonCol` のアリーナ)または同名シャドウポインタ+ `deviceptr` 節(`Force` のバッチ、`Mixing_H` の履歴リング)で見せる。確保が 1 回になることで (a) 予約の成否が原子的に決まり、(b) 共有デバイスでの部分確保・部分失敗という中間状態が消える。

ライブラリへのポインタ受け渡しは `host_data use_device` で行う。`present` で常駐を表明したうえで、ブロック内では配列名がデバイスアドレスに置換され、`cuSOLVER/cuBLAS/GEMMv8` にそのまま渡せる(H2D/D2H 転送なし)。

`gpsolver_Syevdx.c` — OpenACC 常駐配列を `cuSOLVER` に直接渡す典型形

```
#pragma acc data      present(A[0 : m * m])
#pragma acc data      present(W[0 : MaxN])
#pragma acc host_data use_device(A, W)
{
    cusolverDnHandle_t cusolverH = NULL;
    wait_cudafunc(cusolverDnCreate(&cusolverH));
    cudaStream_t stream = NULL;
    wait_cudafunc(cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking));
    wait_cudafunc(cusolverDnSetStream(cusolverH, stream));
```

注意(present エラー): `_openacc` 系ラッパは呼び出し側が data 領域を確立していることを前提とする。確立せずに呼ぶと NVHPC ランタイムの FATAL(present table lookup failed)で落ちる。現行コードは「呼び出し側が必ず enter data を先行させる」規約で統一されている。

6.3 同期の設計と OpenACC メモリプールの扱い

OpenACC の既定キューとライブラリ用 CUDA ストリームは接続されていない。順序保証は (1) ライブラリ呼び出し直前の `#pragma acc wait`、(2) ライブラリ用ストリームの `cudaStreamSynchronize`、(3) 同期 `cudaMemcpy`、の明示同期のみで行う。性能よりも正しさと単純さを優先した設計である。

共有 GPU 運用で重要なのが NVHPC の **OpenACC メモリプール(フリーリスト)の性質**である。`exit data delete` や `acc_free` で解放したブロックはプロセス私有のプールに戻るだけで CUDA には返らず、しかも**同一サイズの要求にしか再利用されない**。この 2 つの性質が共有デバイスで実害を生む: (a) 他ランクから見ると「解放したはずのメモリが空きに現れない」、(b) サイズが微妙に異なる確保を繰り返すとプール内にブロックが座礁して実質リークになる(Force3 のチャンクバッチで実際に OOM を起こした。コミット 28361f7e)。このため現行コードは、(i) 空き VRAM を測る前に必ず `acc_wait_all(); cudaDeviceSynchronize(); acc_clear_freelists();` でプールを CUDA に返却し、(ii) サイズが変動する確保は per-chunk 最大値で 1 回だけ確保して再利用する、という作法で統一されている。唯一の例外として、Mixing_H の preflight は**意図的に `acc_clear_freelists()` を呼ばない**(SCF サイクル中にプールをドレインすると常駐 GPU モジュールの確保動態が乱れ、約 30 反復後の Set_Hamiltonian で OOM を誘発した実測に基づく。コード内コメントに明記)。

6.4 set_hamiltonian_gpu.cu — 結線済みの生 CUDA カーネル

現行ツリーで唯一の「.c から呼ばれる生 CUDA カーネル」が `set_hamiltonian_gpu.cu` (194 行、コミット 95fa51ed で追加)である。Set_Hamiltonian の dVH+Vxc+VNA 格子積分の内核 `matrix_elements_kernel` 1 本と、extern "C" ラッパ `Set_Hamiltonian_Cuda_MatrixElements()` を含む。設計上の要点:

- 1 ブロック行 = 1 原子ペア、block=256 スレッド。共有メモリに「重み付き軌道タイル」(double、GridVol・Vpot・Orbs0 を前計算)と「orbs1 タイル」(float)の 2 枚を置き、グリッド点を 32 点刻みのタイルで回して積和する。float→double のキャスト位置・積和順序は CPU/OpenACC 経路と一致(ビット同一)。
- 戻り値は 4 値: 0=成功、1=共有メモリ不足でカーネル非対応(タイル幅を 32→1 まで半減して試行した後) → OpenACC カーネルへ、2=一時的失敗だがコンテキスト健全(stale エラー対策後の再分類) → 同一バッチを OpenACC カーネルでやり直し、負値=致命的 → abort。
- 起動前に `cudaGetLastError()` で他コンポーネントが残した stale エラーを掃除する (§ 3.6 のエラーラッチ規約の実装例。コミット c5eb6845)。
- ビルドは GEMMUL8 と同じ `nvcc(NVCC_GEMMUL8_FLAGS)` でコンパイルされ、OBJJS にリンクされる。`OPENMX_SETHAM_CUDA_KERNEL=0` で無効化でき、その場合は同一計算の OpenACC カーネルが走る。

撤去された休眠カーネル集: 第 2 版で「検証済み・未結線」として記載した `openmx_cuda_kernels.cu/.h` (git 未追跡・ビルド対象外の生 CUDA カーネル集)は、現行ツリーでは**ディスクからも削除済み**である。当時のカーネルのうち Set_Hamiltonian 系の発想は `set_hamiltonian_gpu.cu` として結線された形になった。

6.5 NVHPC 固有の注意 — ミスコンパイル回避の実例

NVHPC 26.3 の OpenACC(`-acc`)には特定コードを壊す既知の不具合があり、Makefile は**ファイル単位で `-acc` を外すこと**で回避している(理由コメント付き)。これは NVHPC のバージョンを変える際に必ず再検証すべきポイントである。

ファイル	コンパイル指定	Makefile コメントの理由(要約)
Occupation_Number_LDA_U.c / Eff_Hub_Pot.c / Total_Energy.c	CC_NOACC (-O1, -acc なし)	NVHPC 26.3 + -acc が LDA+U Type2/cFLL の占有数・ポテンシャル経路をミスコンパイルする
zero_cfrac.c	CC_NOACC	-acc が DSYGV を失敗させ、ON2 法の極を壊す
RestartFileDFT.c	CC_NOACC	過分割 MPI 実行のリスタート I/O で segfault し得る
MD_pac.c / Generate_Wannier.c	CC_NOACC	SCF 後の MD 制御 / Wannier 生成経路で segfault し得る
Krylov.c	CC_NOACC_03 (-O3, -acc なし)	GPU 化は生 CUDA API のみで行う(OpenACC 不使用)
Band_DFT_NonCol.c / Divide_Conquer.c	CC3 (-O2)	最適化レベル調整
truncation.c / OutData.c	CC2 (-O1)	同上
DFTD3vdW_init.c	CC_DFTD3 (-O0)	同上

表 6-1: ファイル別コンパイル調整(NVHPC 26.3 不具合回避を含む)

このほか、`Total_Energy.c` を `-acc` なしでビルドする必要から、OpenACC カーネル部分だけを別翻訳単位 `Total_Energy_GPU.c` (`-acc` 付きでビルド)に分離するという構成が生まれている(§ 7.10)。また `Force.c` には旧 NVHPC の OpenMP バグ回避として `#if NVHPC_VERSION >= 250000` の条件付きで OpenMP 並列を再有効化する箇所がある。`Divide_Conquer.c` の `if (use_dc_openacc)` ガードには「// compiler's bug」というコメントが残る。

7. 計算経路別の GPU 実装詳細

7.1 バンド計算(コリニア) — Band_DFT_Col.c

最大規模の改修対象(対オリジナル +4,724/-2,544 行)。GPU 密行列経路の条件は `scf_eigen_lib_flag==GPUSOLVER && n ≥ 1000` に加え、コミット `a4200a04` 以降は `BandCol_GpuDenseFits` による 1 ターン分の VRAM 見積りが成立すること(不成立なら ELPA/ScaLAPACK へ。§ 5.6)。特徴:

- **密行列構築キャッシュ:** 疎行列(H, S)から k 点ごとの密行列を組み立てる部分を OpenACC 化し、ペア構造の「fingerprint」付きキャッシュ(`entries / phase_r / phase_i`)を `enter data` で常駐させる。位相のみ `update device` で更新。scatter-add は `deviceptr + acc atomic update`。Set_Hamiltonian が構築した packed H/S キャッシュ(§ 7.7)から直接行列を集約する。
- **対角化と変換:** S 変換行列($\tilde{U}$)は初回に計算してホスト側 `transformed_s` に退避し、SCF 反復間で再利用。H' の構成は `GEMMmul8 Zgemm × 3`(§ 4.6)、対角化は `cusolverDnXsyevdx`。固有ベクトルは DM フェーズまでデバイス常駐。
- **固有値のみの第 1 パス:** `all_knum != 1` の第 1 対角化ループ(化学ポテンシャル決定用に固有値だけが要る)は `jobz=CUSOLVER_EIG_MODE_NOVECTOR` で解き、固有ベクトル書き出しと逆変換 GEMM を丸ごと省く(`BandCol_GpuSolver_SolveEigenvaluesDeviceOnly`。OPENMX_BAND_SYEVDX_NOVECTOR=0 で旧動作。コミット `58ea6471`)。この経路は `na_rows==n && na_cols==n` を要求する。
- **適応並列度:** § 5.5.1 のとおり。k 点ウェーブの初期値は「そのデバイスを共有する k オーナーランク数」(コミット `48921b36`)で、VRAM に収まらないときだけ縮退する。全ランクが収まる場合はターン分割・途中バリア・GEMMmul8 解放点が消える。決定はフル並列時の必要量として `OpenMX_GpuPhaseNeed_Register("band_col", ...)` で公開され、Set_Hamiltonian の常駐キャッシュが道を空ける(§ 7.7)。
- **DM フェーズ:** OpenACC の `gang + loop seq` 縮約で、加算順序を CPU と一致させる。
- SCF 反復間のデバイスバッファ常駐は既定 OFF(`OPENMX_BAND_GPU_PERSISTENT`)、ターンごとの GEMMmul8 ワークスペース解放は既定 ON(`OPENMX_BAND_GEMMUL8_TURN_RELEASE`)。

7.2 バンド計算(非コリニア) — Band_DFT_NonCol.c

- OpenACC 常駐型(`enter data` 中心)の root-dense 経路。位相付き密行列構築(scatter-add)、S 対角化+S^{-1/2}、2n×2n スピノル H2/S2 の組立、U†HU(GEMMmul8 Zgemm)、固有ベクトル逆変換、DM/EDM(`loop seq`)までを GPU で実行。
- **ステージ別適応並列度(§ 5.5.2):** 固有値のみ/固有ベクトル/DM の 3 モードで別々に VRAM を見積み、それぞれ独立に同時ランク数を決める(コミット `7c414f91`, `99f4014b`)。DM 蓄積専用の見積み `BandNonCol_RootDenseDMDeviceBytes` はソルブ段よりはるかに小さく、「ソルブ 2 並列・DM 8 並列」のような 運転になる。フェーズ需要は `"band_noncol_ev" / "band_noncol_vec" / "band_noncol_dm"` の名前で公開。
- **ELPA フォールバック:** `BandNonCol_GpuDiagFits` (1 ランク分すら収まらない場合)。強制は `OPENMX_BAND_GPU_DIAG` (§ 5.6)。
- OpenACC キューとの順序は `#pragma acc wait` → `openmx_gemm8Zgemm` → `cudaStreamSynchronize` で明示制御。

7.3 クラスタ計算 — Cluster_DFT_Col.c / Cluster_DFT_NonCol.c

クラスタ経路は第 2 版以降で最も密度の高い改修を受けた(Col: 対オリジナル +1,534 行、NonCol: +2,424 行)。

Col — root-dense 経路の常駐化と GPU DM

- **発動判定は実予約 (§ 5.5.3、§ 5.6):** `ClusterCol_GpuDiagFits` が 3 値を返す — 0 = ELPA フォールバック、1 = 並行 GPU(各スピンワールドが自スピンを解く)、2 = 直列化 GPU (スピンワールド 0 のオーナーが両スピンを解き、リモートスピンの結果を MPI で送る。コミット `bf08a9d8`)。直列化モードは packed Set_Hamiltonian キャッシュが前提(非 packed の行列集約は level1 全体の集団通信のため片オーナーだけでは呼べない)。
- **DM/EDM の GPU 蓄積(コミット `75fc30d3`):** `ClusterCol_AccumulateDM_OpenACC` が 131,072 エントリ単位の OpenACC gang ループで DM1/EDM1(partial charge 時は PDM1 も)を蓄積する。状態内の総和順序は CPU ループと同一(`loop seq`)。 `OPENMX_CLUSTER_GPU_DM=0` で CPU ループへ。疎→密 index は幾何指紋(FNV-1a)付きでキャッシュされ、スピン×反復ごとの再構築・再アップロードを排除。
- **固有ベクトルのデバイス常駐(コミット `527ed524`):** ソルブ直後の固有ベクトルパネルを スピン別に D2D コピーで stash し(`ClusterCol_StashDeviceEvec`)、DM 蓄積カーネルが `deviceptr` で直接読む(EVec1 の D2H → 再 pack → H2D 往復を排除)。stash の確保は `TryDeviceMalloc` で、失敗時は黙ってスキップ(最適化専用)。非オーナーランクの固有ベクトルスライスのステージングは、収まるまで状態ブロックを半減(下限 128)し、最小でも不可なら CPU ループで続行する。
- ソルブ終了ごとに GEMM_{ul8} ワークスペースを解放(`OPENMX_CLUSTER_GEMMUL8_TURN_RELEASE`、未設定時は BAND 設定を継承)。旧経路 `ClusterCol_GpuSolverDensePath` (CPU dgemm 使用)と ホストバッファ経路は削除された。

NonCol — 単一アリーナと反復間キャッシュ

- **単一 cudaMalloc アリーナ(コミット `3ef5347a`):** S、Ss2、作業行列 7 枚、Hs2、密固有ベクトル、疎→密 index/staging を 512 B 整列で 1 領域に区画割りして予約し、`acc_map_data` で OpenACC に見せる。予約不成立なら ELPA へ (§ 5.6)。
- **反復あたりオーバーヘッド削減(コミット `a9b6fa31`, `f5de630e`):** cublas ハンドルのデバイス別キャッシュ(旧: 反復あたり 13 回 create/destroy)、変換済み S とスピノル展開 Ss2 の デバイス常駐(216 原子で反復あたり約 570 MiB の再アップロードを削減)、zheevdx ワークスペースの永続コンテキスト化、ホスト側スクラッチ約 1.6 GiB の反復間再利用、逆変換 GEMM の m を n2 から MaxN に縮小 (明示 leading dimension 版ヘルパ `ClusterNonCol_GEMMul8ZgemmLd_OpenACC`)。
- 密 固 有 ベ ク ト ル `dense_evec` は デ バ イ ス 常 駐 キ ャ ッ シ ュ さ れ 、 後 段 へ は `DFT.c` から `Cluster_DFT_NonCol_ScatterGpuSolverCachedEvec` で 配 布 。 メ モ リ 圧 迫 時 は `Cluster_DFT_NonCol_DemoteGpuSolverCachedEvec` がホストへ退避させる(密度グリッド GPU サービスが オーナー確保に失敗しそうなときに呼ばれる。§ 7.9)。
- MaxN(部分固有値数)は SCF 第 1 反復のみ全数、以後は占有数+マージン (§ 5.3)。

7.4 DC 法 — Divide_Conquer.c

- 原子ごとの部分クラスタ行列(局所次元 NUM)を各ランクが独立に処理する。GPU 化条件は `NUM ≥ DC_GPU_Threshold()` (既定 1000、`OPENMX_DC_GPU_THRESHOLD` で変更可)。
- **Col 側は生 CUDA 実装:** `DCGpuSolverCtx` に cudaMalloc バッファ+pinned ホストバッファ (`cudaMallocHost`、本実装唯一の使用箇所)。S 対角化(SCF 2 反復目まで)→ H' 構成 (`DC_GpuSolver_TryGpuDgemm`: GEMM_{ul8} → 失敗時 cublasDgemm → それも失敗なら CPU) → 活性部分空間を `cudaMemcpy2DAsync` でデバイス内切り出し → Xsyevdx。
- **S12 パネルのデバイス常駐キャッシュ(S-cache、コミット `6f7f31b2`):** 変換済み重なり行列 S12 = $U \cdot \lambda^{-1/2}$ は SCF 3 反復目以降不変であることを利用し、per-rank の単一 cudaMalloc アリーナに常駐させる。採用は「予算 =(空き - 予約 4096 MiB)/ ノード内ランク数」の範囲で貪欲に(他フェーズを追い出さない polite 方式)。ヒット時は「ホスト S_DC 再充填 → 再パック → PCIe アップロード」を丸ごとスキップし、GEMM が常駐コピーを直接読む。CPU フォールバックが必要になった時だけ `DCCol_FillSDCFromS12` でホスト側を遅延復元する。 `OPENMX_DC_GPU_S_CACHE=0` で無効化。採用状況は `<DC> GPU S-cache: rank %d keeps %d of %d cluster overlaps on the device (%.1f MiB)` と報告される。

- **ホスト転置の廃止:** スライドアクセスの $O(n^2)$ ホスト転置を、行連続 memcpy のパック+ `cublasDgeam` によるデバイス内転置 (`DC_GpuSolver_DeviceTranspose`) に置換。geam は純粋なデータ移動なので後段への入力はビット一致。
- **preflight の省略:** 一度承認された GEMM 形状以下の解では `cudaMemGetInfo` preflight をスキップ (GEMM8/cuBLAS のワークスペースは grow-only で、小さい形状が新規確保を起こさないため)。
- **多層の安全装置:** 確保前の VRAM preflight (`GPUSOLVER_MIN_FREE_AFTER_MB` 既定 1536 MiB)、ホスト malloc 失敗検出、固有値分解失敗検出 — いずれかで **sticky 無効化フラグ** を立て、以後そのランクは CPU の DC ソルバで継続する (GEMM8 ワークスペースも解放)。CPU フォールバック前には CUDA エラーラッチを掃除する (§ 3.6)。
- **NonCol 側は OpenACC 実装:** `S_DC/H_DC/C` を連続ストア+行ポインタ化して data 節を 1 ブロック転送にまとめ、対角化は `gpusolver_Syevdx_Complex_openacc_cached` (ハンドル・ワークスペースを原子間で再利用) 経由。64 原子 Si・16 SCF で対角化バケットが 78.4 s → 66.4 s (コミットメッセージの実測)。

7.5 DC-LNO 法 — Divide_Conquer_LNO.c

- 現行は **direct per-rank dispatch:** 全ランクが `cudaSetDevice` を実行し、自分の局所問題 ($\text{NUM} \geq 800 = \text{GPU_CPU_SWITCH_NUM2}$ 、`OPENMX_DCLNO_GPU_THRESHOLD` で変更可) を直接 GPU に投入する。旧方式の GPU プロキシ機構 (rank 0 が他ランクの対角化を MPI タグ 41001-41004 で代行) は `DCLNO_ENABLE_SHARED_GPU_PROXY 0` で温存。
- **非コリニアも GPU 化(コミット d62cac60):** `DCLNO_Solve_NonCol_GpuSolver` が クラスタごとの一般化固有値問題全体をデバイスで実行する。手順: $\text{NUMH} \times \text{NUMH}$ の S を `zheevdx` で対角化 → $ko = 1/\sqrt{|\lambda|}$ → `cublasZdggmm` + `cudaMemcpy2DAsync` のブロックコピーで スピン倍化変換 $T = \text{diag}(\tilde{U}, \tilde{U})$ を構成 → $H' = T^\dagger H T$ を GEMM8 `Zgemm 2 本` → `cublasZgeam` で 共役行列を作成 (CPU フローが対角化するのと同じ行列にして数値互換を確保) → `NUM2 本` を `zheevdx` → 逆変換を転置形 `Zgemm 1 本` で residue ステージのレイアウトのまま書き出し。有効化は `use_gpu_accel_nc && Eigen_MPI_flag==0 && NUM \geq \text{しきい値}`。
- **デバイス共有ランク群の合算メモリ判定 (§ 5.5.4):** `DCLNO_GpuGroupMemoryFits` が不成立なら グループ全体で CPU 固有値ソルバへ (コリニア・非コリニア両方)。
- **フェーズ末尾のメモリ返却:** `DCLNO_GpuTurnRelease()` (既定 1、`OPENMX_DCLNO_GPU_TURN_RELEASE=0` でオプトアウト) が各フェーズ末尾でソルババッファと GEMM8 ワークスペースを解放し、グリッド/カフェーズにメモリを返す (ハンドルは維持)。
- ノード内 GPU 台数は `scf.Gpu.Num` (既定 30 ≡ 実質無制限) で上限化。数値的失敗時のフォールバックはなく abort (表 5-2)。216 原子 Si (クラスタ次元 ~2222) の NC 実行は np=2 で約 13 分 (CPU 解は 1 時間超)。

7.6 Krylov 法 — Krylov.c

- **OpenACC 不使用・生 CUDA のみ** (`-acc` なしの `CC_NOACC_03` でコンパイル)。OpenMP スレッドごとの `Krylov_GPU_Workspace` プール (`d_A/d_B/d_C/d_W/d_work` + 専用 non-blocking ストリーム) を持つ。
- **原子別パイプラインの融合(コミット 3ee45228):** 従来は原子ごとに「基底再パック&アップロード × 3 + GEMM ごとの H_{2D}/D_{2H} 往復 + ホスト対角化」だったが、`Krylov_ProjectSolve_GPU` が 1 回のデバイス訪問に融合した: ① 基底 d_{KU} を KU キャッシュから取得 (ミス時のみ再パック&アップロード)、② H_{DC} を 1 回だけアップロード、③ $C = H \cdot u_1$ (GEMM8)、④ $u_1 + H u_1$ (GEMM8、 $m_3 \times m_3$ だけダウンロード)、⑤ ホストで ZeroNum 補正と EC_{matrix} 加算 (CPU コードと同一)、⑥ `cusolverDnXsyevd` で対角化 (固有ベクトルパネルは 逆変換が読むレイアウトのままデバイスに残す。失敗時は LAPACK にフォールバックして再アップロード)、⑦ 逆変換 $c = u_1 \cdot b$ (GEMM8) → ダウンロード。
- **発動条件:** 3 つの GEMM すべてが $m \cdot n \cdot k \geq 5 \times 10^7$ (`OPENMX_KRYLOV_GPU_DGEMM_MIN_FLOPS`) を満たすこと。これにより「per-call パスと同じ GEMM だけが GPU に行く」条件が保たれ、既定では旧コードと **ビット一致** する。対角化のデバイス実行は $m_3+1 \geq 1000$ (`OPENMX_KRYLOV_GPU_EIGEN_MIN`)。融合パス自体は `OPENMX_KRYLOV_GPU_FUSED=0` で無効化でき、その場合は従来の per-call シーケンスが走る。

- **KU キャッシュ:** 再パック済み Krylov 基底パネル(SCF 1 反復目に生成、以後不変)を per-rank アリーナに デバイス常駐させる。採用は DC の S-cache と同じ polite 方式 (予算 =(空き - 4096 MiB) / ノード内ランク数、`OPENMX_KRYLOV_GPU_KU_RESERVE_MB`)。 `OPENMX_KRYLOV_GPU_KU_CACHE=0` で無効化。採用状況は `<Krylov> GPU KU-cache: rank %d keeps %d of %d basis panels on the device (%.1f MiB)` と 報 告 。 解 放 は 力 計 算 直 前 の `Krylov_Release_GPU_KUCache()` 。
- S 変換は Krylov 基底構成(CPU の `Inverse_S_by_Cholesky`)で処理済みのため、GPU 側は標準固有値問題のみを扱う。 `OPENMX_KRYLOV_GPU_PROFILE=1` で融合パスの内訳 (`KRYFUSE n=... ku=... g1=... eig=...`)が出力される。

7.7 行列要素構築 — `Set_Hamiltonian.c` / `set_hamiltonian_gpu.cu` / `Set_Nonlocal.c` / `Set_OLP_Kin.c`

`Set_Hamiltonian.c`(対オリジナル +2,972/-1,149 行)+ `set_hamiltonian_gpu.cu`(194 行)

第 2 版時点では「格子積分の OpenACC 版は実装済みだが実測で遅く恒久無効」だったが、コミット `95fa51ed` 以降、dVH+Vxc+VNA の格子積分(`Calc_MatrixElements_dVH_Vxc_VNA`)は **既定で GPU 実行**になった(`OPENMX_SETHAM_MATRIX_GPU`、既定 1)。転換を支えたのは次の 4 機構である。

1. **SCF 不変キャッシュ:** ペアメタデータ・グリッド索引(nolg_MN/nolg_Nc)・軌道値バッファ (orbs0buf/orbs1buf、float)を SCF を通じて 1 回だけ構築するホスト側キャッシュ (`SetHamiltonianMatrixElementsCache`)。毎反復転送されるのは H ブロック(hbuf)とランク局所の Vpot スライスだけになった。
2. **オンデバイス Vpot ギャザー(コミット `bff0af37`):** 旧実装のホスト側「spin×重なり点数」展開を廃し、 `Vpot_Grid` のランク局所スライスをそのまま転送してカーネル内で間接参照する。丸め順序は旧展開と同一でビット同一。
3. **専用 CUDA カーネル(§ 6.4):** 共有メモリに「GridVol・Vpot・Orbs0 の重み付きタイル(double)」と「orbs1 タイル(float)」を置くタイル型カーネル `matrix_elements_kernel` 。 `OPENMX_SETHAM_CUDA_KERNEL=0` で OpenACC カーネルに切替可能(数値は同一)。
4. **適応 GPU ターン計画(`Set_Hamiltonian_CreateGpuTurnPlan`):** § 5.5 と同じ UUID 共有群+降順プレフィックススキャンで同時実行ランク数を決める。既定は **ハイブリッド波**: 第 1 波のランクが GPU で計算する間、残りのランクは同時に CPU で計算する (`OPENMX_SETHAM_GPU_SERIAL_WAVES=1` で全波 GPU 直列に変更可)。既定では H の局所セグメントを持つ **全ランクが GPU 作業候補**(旧 k オーナー限定ポリシーは `OPENMX_SETHAM_GPU_KOWNER_ONLY=1` で復元)。

常駐キャッシュと「需要への譲歩」(コミット `bff0af37` , `69dde552`): そのデバイスを共有する全ランクが単一波で走れる場合に限り、SCF 不変テーブル (orbs1/orbs0/nolg/meta の 4 クラス、サイズ降順)を デバイス常駐させる (`OPENMX_SETHAM_GPU_RESIDENT`、既定 1)。入場予算は「空き-(公開済みフェーズ需要)-フロア」で、フロアは対角化需要が公開済みなら 4 GiB、未公開なら 16 GiB (`OPENMX_SETHAM_GPU_RESIDENT_FLOOR_MB` で上書き)。バンド系ソルバが `OpenMX_GpuPhaseNeed_Register("band_...")` で公開した需要を毎回照会し、空きが需要を下回ると同一 **SCF 反復の対角化の直前に自主退去**して、その MD ステップの残りは transient 運転に切り替わる。 `copyin` は `present_or_copyin` 意味論なので、常駐の有無・部分常駐にかかわらず数値はビット同一である。関連の stdout:

```
<Set_Hamiltonian> Hamiltonian matrix for VNA+dVH+Vxc (GPU-accelerated)...
<Set_Hamiltonian> GPU device %d: %d Hamiltonian rank(s), GPU concurrency=%d,
    serialized waves|CPU fallback ranks=%d, peak=%.3f GiB, free=%.3f GiB, reserve=%.3f GiB
<Set_Hamiltonian> GPU resident cache: %d rank(s) hold %.3f GiB (orbs1+orbs0+nolg+meta) on device %d across SCF iterations
<Set_Hamiltonian> GPU resident cache released: a GPU phase needs %.3f GiB on device %d; staying transient for this MD step
```

幻の失敗の修正(コミット `c5eb6845`): 満杯に近いデバイスでは、正常に回復した他コンポーネントの確保失敗(GEMMu8 の FP64 フォールバック、 `wait_cudafunc` のリトライなど)が CUDA のスレッド別エラーラッチに残り、CUDA カーネルの検査コードが自分の失敗と誤認して abort していた (SCF 316 反復目で「CUDA matrix-elements kernel failed with status -2」)。現行は、カーネル起動前に stale エラーを pop し、失敗時もコンテキストが健全なら「回復可能(status 2)」に分類して **同一バツ**

チを OpenACC カーネルでやり直す(部分更新された可能性のあるデバイス H ブロックは ホストコピーから復元)。abort するのはコンテキスト汚染型のみ。

- **base 結合($H = F_{\text{Kin}} \cdot H_0 + \dots$)の GPU 版**は引き続きオプトイン(`OPENMX_SETHAM_BASE_GPU`、実測 10 ms vs CPU 2 ms のため)。
- **packed H/S キャッシュ** (GPUSOLVER 用に H と S を 1 次元 packed 形式で保持する API 群、 `Set_Hamiltonian_Build_GpuSolver_HS_Cache` ほか)は従来どおり Band/Cluster ソルバの密行列組立を支える。クラスタ 2 スピン直列化モード(§ 7.3)はこのキャッシュが前提。
- **共有テーブルの公開**: 密度グリッドの分散求積(§ 7.9)が同じ軌道値テーブルを読めるよう、読み取り専用ビュー `Set_Hamiltonian_GetMatrixElementsTables()` と、テーブルを構築せずに存在だけ確認する `Set_Hamiltonian_MatrixElementsTables_Ready()` (数 GiB の遅延構築を誘発しないプロープ。コミット `ae3dd189`)を提供する。
- プロファイル: `OPENMX_BAND_PROFILE=1` で `SETHPROF` / `SETHMEPROF` 行を出力。

Set_Nonlocal.c(対オリジナル +1,738/-305 行)

第 2 版時点は「動径 k 積分ループの OpenACC 化」だったが、コミット `211f0684` で **非局所射影子積分全体のバッチ GPU 化** に置き換わった(`SetNonlocal_CalcDSNL_OpenACC`)。構成は 3 フェーズ:

1. **ホスト準備**: 種ごとの動径テーブル(基底/射影子の Fourier 変換)と共通 k グリッドを OpenMP で構築し、バッチの単位となる「行」=(原子ペア, 微分方向 so, 角運動量 LL, 基底×射影子コンボ)を列挙する。Wigner-3j 階乗和の数千万回再評価を避けるため **Gaunt 係数を事前表化**する(選択則 $m = M_0 - M_1$ で軸を削減)。
2. **デバイス実行**: 球ベッセル関数表を**デバイス上で生成**(ホスト実装と同じ下降漸化式・リスケーリング・正規化の忠実移植 — 旧実装の「約 21 秒の直列ホスト Bessel 生成+チャンク転送」を除去)し、台形則の動径積分を `gang×vector reduction` でバッチ縮約する。球ベッセル表はチャンクあたり 96 MiB の デバイス予算に収まるようペアを分割。
3. **ホスト仕上げ**: 球面調和・Gaunt・複素→実変換は CPU パス同様の構造で OpenMP ペア並列。

フォールバックは `OPENMX_SETNL_OPENACC=0` (または非 GPUSOLVER)のみで、メモリ量による動的 フォールバックはない。実測 (216 原子 Si NC, $np=2$): DS_NL ステージ 23.7 s → 0.66 s。メタンの CPU 比較で最初の 8 SCF ステップがビット同一。あわせて固定サイズのスタック配列を オーバーフロー検査付きのヒープ確保に置き換える堅牢化も維持されている。

Set_OLP_Kin.c(+528/-180 行)と Spherical_Bessel.c

- 重なり・運動エネルギー行列要素の動径積分オフロードは、データ転送律速で CPU より遅い実測 (コメント: 約 1.2 s vs 0.3 s)のため `OPENMX_OLPKIN_RADIAL_GPU=1` の **オプトイン**のまま。さらにスレッド数 1(`Nthrds==1`)を要求する。
- CPU 側の実質的な高速化として、ペア距離 r の **bit パターン完全一致をキーとする球ベッセル関数テーブルキャッシュ** (結果は bitwise 同一、シングルスレッド時のみ)と、 `Spherical_Bessel.c` のホットパスの 毎呼び出し malloc/free 除去は従来どおり。

7.8 VNA 射影子 — Set_ProExpn_VNA.c

第 2 版時点では「一度 GPU 化されたのち CPU 版へ戻され、オリジナルと差分なし」だったが、コミット `77d2a22c` で **+2,029 行の本格的な GPU バッチ化**が入った(216 原子 NC で 160.9 s → 4.4 s)。共通ゲートは GPUSOLVER かつ `OPENMX_SETPRO_GPU` (集団判定、既定 1)。3 つの独立なバッチからなり、それぞれ独立にメモリ判定して不可なら従来 OpenMP ループに落ちる:

1. **DS_VNA 構築バッチ**: 一中心積分 $\langle \phi | P_{\text{VNA}} \rangle$ の全ペアを 256 ペアずつのチャンクで処理。4 カーネル構成 — ① 球ベッセル漸化のデバイス実行(`Spherical_Bessel12` の忠実移植)、② Gauss-Legendre 求積和、③ Gaunt × Y_{lm} 重み付き (LL, m) 和、④ 複素→実変換と float ストア。階乗ベースの Gaunt 係数と long double の複素球面調和は**ホストで表引**

き化し、デバイスは参照のみ。求積は要素単位で CPU と同一の累積順を保持。メモリ条件は(空き - 256 MiB)/ノード内ランク数 > 1 GiB。

2. **HVNA トレースバッチ:** $HVNA[Mc][j] = dmp \cdot \sum ene \cdot DS_VNA \cdot DS_VNA$ のエネルギー重み付き float 内積を、MPI パイプラインで受信した halo ブロックのアーカイブ後に全 (Mc, j) 対を 1 カーネルで評価。ダンピングと格納はホスト(CPU ループと完全同順)。
3. **Set_VNA2 / Set_VNA3 バッチ:** $\langle \Phi | VNA \rangle$ と $\langle VNA | \Phi \rangle$ は「PAO 積側の種と方向ベクトルの符号を入れ替えた鏡像」として同一カーネル群(mirror 引数)で処理。512 ペアずつのチャンク。

メモリ不足時の通知はノードごと 1 回(コミット b569f272): Set_ProExpn_VNA GPU: not enough free device memory; CPU fallback. DFT.c のステージバナーは <MD=...> Calculation of the VNA projector matrix (GPU-accelerated) となる(コミット 30148e1e)。

7.9 密度グリッド — Set_Density_Grid.c + Set_Density_Grid_GPU.c

電子密度のグリッド構築($\rho(r) = \sum DM \cdot \phi\phi$)が新規ファイル Set_Density_Grid_GPU.c (1,366 行)で GPU 化された。**3 段のフォールバック連鎖**で動く(Set_Density_Grid.c のコメントに明記): **分散 GPU 求積 → 単一ランク GPU サービス → CPU/OpenMP ループ**。共通の有効化条件は GPUSOLVER かつ Cluster/Band ソルバかつ Cnt_switch==0 (OPENMX_DENSITY_GRID_GPU=0 で全体無効)。

(a) 分散 GPU 求積(コミット d14ef297)

- 各ランクが自分の原子の原子球密度 Tmp_Den_Grid をデバイスで積分し、既存の A→B 通信に流す(オーナー直列化なし)。出力点ごとの CSR を CPU ループと同じ (h_AN, Nog) 順で辿り、加算順序も同一。
- Set_Hamiltonian のテーブルを共有:** 軌道値テーブル (orbs0/orbs1) ・ グリッド索引は Set_Hamiltonian_GetMatrixElementsTables() の読み取り専用ビューで参照し、常駐済みなら copyin (present_or_copyin)はアタッチのみ — 毎反復転送されるのは CSR・密度行列・結果だけになる。
- モード(OPENMX_DENSITY_GRID_GPU_LOCAL): 0=無効、1(既定)=**共有テーブルが全クラス常駐のときだけ** 関与(毎回ステージングするとオーナーサービスより遅いため)、2=常に関与。モード 1 では Set_Hamiltonian_MatrixElementsTables_Ready の Allreduce(MIN) で「全ランクがテーブル構築済み」のときだけ進む(CPU 実行ランクに数 GiB のホストテーブルを作らないため。コミット ae3dd189)。さらにノード共有クォータ(空き - 256 MiB をノード内ランク数で割った値)に収まることも要求。
- 参加決定は全ランクの Allreduce(MIN) で合意し、初回に <Set_Density_Grid> distributed GPU quadrature: every rank integrates its own atoms と表示。自身の一時需要は OpenMX_GpuPhaseNeed_Register("density_grid_local", ...) で公開する。

(b) 単一ランク GPU サービス(コミット bff93d08)

- オーナー(Host_ID)が幾何ごとに 1 回、全ランクの原子中心軌道グリッドと重なり位相表を MPI_Gatherv で収集してキャッシュ。以後の SCF 反復では **CDM だけ**を gather し、オーナーが正規グリッド全体の密度を CSR カーネルで構築して MPI_Scatterv で配布する。
- オーナーの確保が不足しそうなときは段階的に道を空ける: まず全ランクで GEMMu8 ワークスペース解放+ OpenACC プール返却、次に非コリニアクラスタの固有ベクトルキャッシュのホスト退避 (Cluster_DFT_NonCoL_DemoteGpuSolverCachedEVec)。それでも不足なら Set_Density_Grid GPU owner needs ... ; CPU fallback. を出してそのエポックは CPU で計算。
- 既定ではデバイス側を計算直後に解放する(OPENMX_DENSITY_GRID_GPU_PERSISTENT=0。GEMMu8 の cudaMemGetInfo 判定を狂わせないため)。予約は OPENMX_DENSITY_GRID_GPU_RESERVE_MB (既定 256 MiB)。

7.10 全エネルギー — Total_Energy.c + Total_Energy_GPU.c

ファイル分離の理由: NVHPC 26.3 の `-acc` が LDA+U 経路をミスコンパイルするため、`Total_Energy.c` 本体は `-acc` なし (`CC_NOACC`) でビルドせざるを得ない。そこで OpenACC カーネルだけを別翻訳単位 `Total_Energy_GPU.c` (`-acc` 付き、現在 1,064 行) に分離した。

ゲートは `TotalEnergyUseOpenACC()`。第 2 版時点は「非共線 (SpinP_switch==3) では無効」だったが、コミット 19b9b69c で **NC 制限が撤廃**され、現在は GPUSOLVER なら常時有効 (`OPENMX_TOTALENERGY_GPU=0` で全カーネル一括無効化)。GPU 化された格子縮約は 5 種:

1. **EXC/EH1 格子積分** (Ena・Eef・EH1・EXC を単一 `parallel loop reduction` で縮約)
2. **双極子モーメント 6 成分** (格子座標は GN→(n1,n2,n3) をデバイス内で再構成)
3. **CWF 二重計上補正** (dcEH1/dcEXC)
4. **EH0 二中心項のバッチ計算** — 全原子ペアを一括配列化し、species ごとの球面格子・VPS メッシュ・V_H 原子テーブルをフラット化して転送。gang (ペア) × `vector reduction` (格子点) でエネルギーと r 微分 (力 #6/#7 相当) を同時計算。スプライン補間は `#pragma acc routine seq` のデバイス関数として実装。
5. **Exc0 細メッシュ補正のバッチ (新設、コミット 19b9b69c)** — `Exc0_correction_flag==1` のとき、従来ホスト直列だった Lebedev 球面格子 × 粗動径メッシュの補正計算を 2 パスの GPU バッチに置換 (`TotalEnergy_Exc0_Batch_OpenACC`): ① 全ローカル原子 × 動径 × Lebedev 点をフラット化してエネルギーを縮約しつつ点ごとのカプレファクタをデバイス常駐で保存、② (原子, 近接原子) ペアカーネルでプレファクタ × 密度勾配を縮約して力 #8 相当を得る。XC 汎関数 (Ceperley-Alder) とスプライン補間 (KumoF/Dr_KumoF) はデバイス関数として再実装。

`Total_Energy_GPU.c` — 格子縮約の典型形

```
#pragma acc data copyin(Density_Grid_B0[0:grid_count], Density_Grid_B1[0:grid_count], \
    ADensity_Grid_B[0:grid_count], PCCDensity_Grid_B0[0:grid_count], \
    PCCDensity_Grid_B1[0:grid_count], dVHart_Grid_B[0:grid_count], \
    Vxc_Grid_B0[0:grid_count], Vxc_Grid_B1[0:grid_count], \
    RefVxc_Grid_B[0:grid_count], VNA_Grid_B[0:vna_grid_count], \
    VEF_Grid_B[0:vef_grid_count])
{
#pragma acc parallel loop reduction(+:local_Ena,local_Eef,local_EH1,local_EXC0,local_EXC1)
    for (BN=0; BN<grid_count; BN++){
```

7.11 力 — Force.c

力計算は第 2 版以降に最も大きく変わった領域である (対オリジナル +15,123/-11,241 行)。第 2 版時点は「#2/#4/#5 の OpenACC 縮約、#4B は VRAM 枯渇回避のため CPU 維持」だったが、現在は **#3・#4B・HNL を含む主要トレースがすべて GPU バッチ化**され、代わりに「共有デバイスで絶対に abort しない」ためのアリーナ+ピア待ち機構が整備された。親ゲートは `OPENMX_FORCE_GPU` (集団判定、既定 1)。

ステージ	実装	ゲート/フォールバック
#1(重なり補正)	ホストのみ	—
#2(運動エネルギー)	パック+GPU リダクション(<code>Force2_Kinetic_OpenACC</code>)	SO/LDA+U/拘束/Zee-man なしのとき。確保失敗時はその場でホスト計算
#3(dn/dx・Vpot 格子トレース)	チャンクバッチ <code>Force3_GpuTrace</code> : 1 ペア = 1 gang、重なり点で vector 縮約。 dOrbitals はデバイスで再計算 (実球面調和 L0≤3 の明示式+動径スプライン。約 4.5 GB/rank の PCIe 転送を排除)	予算判定 (常駐分+192 MiB が(空き-256 MiB)/ノード内ランク数に収まる)。dOrbitals 再計算のみの不成立はホスト計算 dchi の転送に縮退(<code>OPENMX_FORCE3_GPU_DORB</code>)

ステージ	実装	ゲート/フォールバック
#4(ProExpn_VNA==0)	dVNA 格子項の GPU リダクション(<code>Force4_OpenACC</code>)	確保失敗時はホスト計算
#4B(射影 VNA)	GPU 内積バッチ (コミット <code>7d38382d</code>): DS_VNA の 4 方向を float でデバイス常駐(direction-major)、MPI パイプラインで受けた halo/case-2 ブロックをアーカイブし、case-1/case-2 の融合トレースを各 1 カーネルで実行.float 積 → double 加算のキャスト位置まで CPU を再現。ダンピング尾部はホスト	<code>OPENMX_FORCE4B_GPU</code> (既定 1)+メモリ判定。不成立時は 融合 CPU トレース (<code>OPENMX_FORCE4B_FUSED</code> 、コミット <code>fe68cca6</code>)、それも無効なら従来 dHVNA 経路。特殊対(q_AN==0 等)は常にホスト
#5(重なり×EDM)	GPU リダクション(<code>Force5_OpenACC</code>)	確保失敗時はホスト計算
HNL(非局所)	GPU 内積バッチ(case-1/case-2)。DS_NL は double のまま常駐。case-2 は全対を GPU が担当	<code>OPENMX_FORCEHNL_GPU</code> (既定 1)。スカラー相対論トレースのみ (SO/LDA+U/拘束/Zeeman 付き NC,j 依存 VPS は CPU)
#6/#7(EH0)、#8(Exc0)	Total_Energy の GPU バッチと同時計算(§ 7.10)	<code>OPENMX_TOTALENERGY_GPU</code>

表 7-1: 力計算ステージ別の GPU 化状況

アリーナ+ピア待ち機構(コミット `206b7e06` , `d0e1d17a`): 共有デバイスではランク間のステージ進行がずれるため、「バッチ開始時に見た空きメモリ」は当てにならず、遅れたランクの `acc data copyin` が満杯のデバイスに当たると OpenACC ランタイムが実行全体を abort させていた。現行実装はタイミング回避ではなく**致命窓そのものを除去**した:

- 全確保を 512 B 整列の**単一アリーナ**にまとめ(`Force_gpu_arena_off`)、カーネルは同名シャドウ ポインタ+ `deviceptr` で読む(カーネル本体=数値は不変)。
- ホストフォールバックがある確保(リダクションバッファ)は `Force_gpu_arena_try` — 失敗したら NULL を返し、その場でホスト計算(通知はランクごと 1 回: `Force: a %.1f MiB GPU reduction buffer does not fit ...; using the host for this reduction.`)。
- 必須確保(バッチ本体)は `Force_gpu_arena_wait` — 50 ms 刻みでピアランクの解放を待ち (2 秒で警告 1 回)、180 秒待っても不足なら所要量・空き量つきの診断で MPI_Abort。ピアの一時確保は必ず解放されるので前進保証があり、1 領域 = 1 アリーナなので待機中のランクは 何も保持せず相互デッドロックしない。
- カステージに「abort し得るデバイス確保」は存在しない**(コミット `d0e1d17a` で最後の 2 箇所の data 節確保もアリーナ化)。Force() 入口と大アリーナ解放後には OpenACC プールを CUDA に返却 (`Force_gpu_pool_flush`)。
- チャンクバッファは per-chunk 最大値で 1 回だけ確保して再利用(NVHPC OpenACC プールの「同一サイズのみ再利用」問題への対処。 § 6.3、コミット `28361f7e`)。

そのほか: Force4B/HNL/ProExpn の「メモリ不足で CPU フォールバック」通知はノードごと 1 回 (コミット `b569f272`)。 `OPENMX_FORCE_PROFILE=1` でステージ別の [Force profile] 行(max/avg 秒)が出る。コミット `098addc` は Force3/4B のステージングを「ホスト常駐ミラー方式」から「デバイス常駐+1 原子ステージ方式」に改め、ランク数倍で効いていたホストメモリ消費(数 GB/rank)と予算計上漏れを修正した。各ステージのバナーには GPU 実行時に (GPU reduction) / (GPU-accelerated) が付く(§ 10.3)。

7.12 電荷/DM 混合の高速化 — Mixing_H.c ほか 6 ファイル

SCF の混合(`scf.Mixing.Type`)は第 2 版まで完全に CPU のままだったが、コミット `94b6ca8c` / `6e94a9d3` で全スキームに高速パスが入った。有効化条件はすべて **GPUSOLVER のときのみ**(`OPENMX_MIX_FAST=0` で一括無効化)。レガシーコードは else 分岐に 無傷で残り、Simple mixing と NEGF 変種は対象外である。

RMM-DIISH(Pulay_Mixing_H) — 2 層構造(コミット 94b6ca8c、Mixing_H.c +1,350 行)

動機: 残差 Gram 行列のためにスパース H 構造を dim^2 回掃引し、**行列要素ごとに 1 回の MPI_Allreduce** を発行していた (History 50 の Fe fcc $5 \times 5 \times 5 \cdot \text{np}18$ で DFT 時間 7,352 s 中 2,415 s)。MixH_GpuPreflight (集団判定、指紋が変わらない限り sticky) が 2 層のどちらかを選ぶ:

- **Tier A(GPU 常駐、Pulay_Mixing_H_GPU)**: per-rank の単一 cudaMalloc アリーナに 残差スロットのリング・メトリック重み・最適残差・Gram ペアバッファを配置。履歴シフトは **論理→物理スロットマップの回転(コピー 0 回)**、ホスト側の残差履歴もポインタ回転+「新残差をスロット 0 とフラット像に同時計算」で常に整合。重み付き Gram 三角全体を 1 つの OpenACC カーネルで計算し、**MPI_Allreduce は 1 回**。メトリック・ α 階段・nan 検査は レガシーと同一。デバイス共有リンク群のアリーナ合計が空きに収まるときだけ発動。
- **Tier B(増分 CPU、Pulay_Mixing_H_Inc)**: Gram 行列を(原子, 軌道行)ブロックの メトリック非依存部分和に厳密分解し、ポインタ回転で旧エントリがビット単位で有効なままスライドすることを利用して、継続反復では**新しい 1 行だけ**を $O(\text{dim} \cdot L)$ で計算(レガシーは $O(\text{dim}^2 \cdot L)$)。デバイスメモリ不要。
- 制御: OPENMX_MIXH_GPU (未設定=自動、0=レガシー、1=Tier A 強制、2=Tier B 強制)、OPENMX_MIXH_GPU_RESERVE_MB、OPENMX_MIXH_GPU_VERBOSE。不適合時は 1 回だけ「using the incremental CPU Pulay mixing instead」と通知。preflight は**意図的に acc_clear_freelists() を呼ばない**(§ 6.3)。
- 実測: Fe fcc $5 \times 5 \times 5 \cdot 100$ 反復で Mixing_DM 504.6 s $\rightarrow$ 47.3 s(10.7 倍)、フル実行の総 wall -25%。収束 Utot は全表示桁で一致。

残りのスキーム(コミット 6e94a9d3) — GPU は使わない CPU 高速パス

スキーム(ファイル)	高速化の内容
RMM-DIISK(DIIS_Mixing_Rhok.c)/ RMM-DIISV(Mixing_V.c)	増分 Gram : 格納済み残差列は事前重み付きでイテレーション間はスライドするだけなので、Gram 行列を対角方向に 1 つスライドし、新残差行だけを dgemv で計算して 短い Allreduce 1 回 。履歴シフトはポインタ回転+復元コピー 1 回(レガシーとビット一致)。継続が切れたら(SCF 巻き戻し等)レガシー計算で再シード
RMM-DIIS(DIIS_Mixing_DM.c)/ GR-Pulay(GR_Pulay_DM.c)	バッチ Gram : 全ペアの Gram を 1 回のスパース構造掃引(OpenMP)+ 1 回の Allreduce で計算(レガシーはペアごとに 1 掃引+1 Allreduce、History 50 なら最大 1,275 回/反復)。最適残差と混合掃引も 1 パスに融合。残差履歴はポインタ回転、 DM/iDM の履歴シフトは意図的にレガシーの内容コピーのまま (他の GPU モジュールが DM バッファに紐づくデバイス側状態を保持し得るため、DM のスロットポインタは回転してはならない — コード内コメントに明記)
Kerker(Kerker_Mixing_Rhok.c / Mixing_V.c)	履歴シフトのみ高速化(回転+memmove)。RMM-DIISK/V の StartPulay 前反復で毎回走るため無視できない

検証(216 原子 Si $\cdot \text{np}8$): Kerker は全出力行ビット一致、RMM-DIISK は最終 Uele ビット一致、他スキームも $1\text{e}-10 \sim 1\text{e}-12$ で一致。Fe fcc $5 \times 5 \times 5$ (rmm-diisk $\cdot 40$ 反復 $\cdot \text{np}18$)で Mixing_DM 33.5 s $\rightarrow$ 20.0 s。

7.13 派生ソルバ群 — 分極・DMmu・CWF・LNO(定型パターン 11 ファイル)

以下の 11 ファイルには同一の定型パターンが適用されている: openmx_gpuserver_dense_utils.h の include、GPUSOLVER 要求時のデバイス割当、static コンテキスト、そして **ELPA2 分岐の直前**への scf_eigen_lib_flag==GPUSOLVER && GPU_CPU_SWITCH_NUM<=n && na_rows==n && na_cols==n 分岐の挿入(OpenMX_GpuSolver_DenseDsyevx/DenseZheevx による対角化)。条件を満たさなければ従来の ELPA/ScaLAPACK に流れる。

ファイル	GPU 対角化の挿入箇所
Band_DFT_Col_CWF.c	Zheevx $\times 3$ (S $\cdot$ H $\cdot$ CWF バンド)
Band_DFT_Col_DMmu.c	Zheevx $\times 4$
Band_DFT_NonCol_CWF.c	Zheevx $\times 3$

ファイル	GPU 対角化の挿入箇所
<code>Band_DFT_NonCol_DMmu.c</code>	Zheevx $\times$ 4
<code>Cluster_DFT_Col_CWF.c</code> / <code>Cluster_DFT_Col_DMmu.c</code> / <code>Cluster_DFT_Col_LNO.c</code>	Dsyevx $\times$ 2 ずつ
<code>Cluster_DFT_NonCol_CWF.c</code> / <code>Cluster_DFT_NonCol_DMmu.c</code>	Dsyevx $\times$ 1 + Zheevx $\times$ 1 ずつ
<code>Electric_Polarization_BandCol.c</code>	Dsyevx $\times$ 1 + Zheevx $\times$ 4(DM・重なり・H・ABC)
<code>Electric_Polarization_BandNonCol.c</code>	Zheevx $\times$ 5(ELPA1 側フォールバック条件の調整も含む)

表 7-2: 定型 GPUSOLVER 分岐が挿入されたファイル群

7.14 汎用固有値ルーチン — Eigen_lapack.c / EigenBand_lapack.c

- `Eigen_lapack()` (実対称) は `flag != ELPA1/2 && n  $\geq$  1000` で `Eigen_gpusolver_x` ($\rightarrow$ `gpusolver_Syevdx`) へ分岐。**Eigen_lapack** を呼ぶ 20 以上のファイル(軌道最適化・DC・LNO など)が自動的に GPU 化の恩恵を受ける。上位に固有ベクトル直交性検査つきのソルバ巡回リトライがある。
- `EigenBand_lapack()` (エルミート) は $N \geq 1000$ で `gpusolver_zheevx` へ。失敗時は CPU の Householder 法 `Eigen_HH` にフォールバック。
- 両者とも OpenACC デバイス常駐配列を直接処理する `_openacc` エントリを持ち、非共線 DC の OpenACC 経路などから使われる。2 次元配列 $\leftrightarrow$ 1 次元デバイス配列の詰め替えも device 上で行う。

8. GPU 化以外の変更点

8.1 OpenMP バグ修正 — private 指定漏れによるヒープ破壊の解消

オリジナル OpenMX 4.0 の `Stress.c` と `Total_Energy.c` には、`#pragma omp parallel` の `private()` 節にループ変数 `i` が入っておらず、暗黙 `shared` の `i` を複数スレッドが同時に読み書きする競合があった。あるスレッドがループ境界を判定した直後に別スレッドが `i` を進めると配列の範囲外アクセスが起こり、ヒープ破壊(malloc メタデータの破損)として顕在化する。本 GPU 版の開発中に `runtest` のクラッシュ原因として特定され、修正された(コミット `d451eeb8`)。

Stress.c — 修正内容(private リスト末尾への `i` 追加。**Total_Energy.c** も同形)

```
/* 修正前(オリジナル)*/
#pragma omp parallel ... private(OMPID,Nthrds,Nprocs,Mc_AN,(中略),sumt1,sumt2,sumt3,sumt4,sumt5)

/* 修正後(GPU 版)*/
#pragma omp parallel ... private(OMPID,Nthrds,Nprocs,Mc_AN,(中略),sumt1,sumt2,sumt3,sumt4,sumt5,i)
```

- **Stress.c**: 競合箇所は並列リージョン内の `for (i=0; i<9; i++){ sumt1[i]=0.0; ... }` (原子ループ内で毎回実行)。このファイルの差分はこの 1 行のみ。
- **Total_Energy.c**: `Calc_EXC_EH1()` の Lebedev 球面グリッド積分並列区間。競合ループは `for (i=0; i<(FNAN[Gc_AN]+1); i++)` で `Dr[i][ir][ia]` 等を添字アクセスするため、競合時に確保域外を指し得る(ヒープ破壊経路)。

意義: この 2 件はオリジナル OpenMX 4.0 にも存在するバグであり、GPU と無関係に OpenMP スレッド並列(スレッド数 ≥ 2) で発症し得る。CPU 版 OpenMX を使う場合にも適用価値がある修正である。

8.2 メモリ管理・堅牢化

ファイル	修正内容
<code>openmx.c</code>	起動直後に <code>Raise_Stack_Limit()</code> を呼び、スタックのソフトリミットをハードリミットまで自動で引き上げる(<code>getrlimit/setrlimit(RLIMIT_STACK)</code>)。 <code>ulimit -s</code> 未設定環境での大きなスタック配列によるクラッシュ対策
<code>truncation.c</code> / <code>Free_Arrays.c</code>	<code>VNA_Grid</code> / <code>VNA_Grid_B</code> / <code>VEF_Grid</code> / <code>VEF_Grid_B</code> の free 後に NULL 代入を追加(二重 free 防止)。 <code>truncation</code> は <code>Set_Hamiltonian</code> / <code>Set_Density_Grid</code> の GPU キャッシュ無効化フックも呼ぶ(幾何変更で常駐テーブルが陳腐化するため)
<code>Set_Nonlocal.c</code>	固定サイズ配列をオーバーフロー検査付きヒープ確保に置換 (§ 7.7)
<code>Free_Arrays.c</code>	終了時(wherefrom==0)に GPU キャッシュの無効化(<code>Set_Density_Grid_GPU_Invalidate</code> / <code>Set_Hamiltonian_Invalidate_OpenACC_MatrixElements_Cache</code>)と <code>openmx_gemm18ReleaseWorkspaces()</code> を呼ぶ

openmx.c — `Raise_Stack_Limit`(全文)

```
static void Raise_Stack_Limit(void)
{
#ifdef RLIMIT_STACK
    struct rlimit limit;

    if (getrlimit(RLIMIT_STACK,&limit)!=0) return;
    if (limit.rlim_cur==RLIM_INFINITY) return;
    if (limit.rlim_max==0 || limit.rlim_cur==limit.rlim_max) return;

    limit.rlim_cur = limit.rlim_max;
    setrlimit(RLIMIT_STACK,&limit);
#endif
}
```

8.3 その他の修正

ファイル	修正内容
<code>Runtest.c</code>	(1) 入力ファイル判定を <code>strstr</code> による部分一致から末尾一致 <code>has_dat_suffix()</code> に厳密化(<code>foo.dat~</code> 等の誤検出防止)。(2) 各テスト実行前に旧 <code><System.Name>.out</code> を削除し、前回結果の混入を防止(MPI_Barrier 付き)
<code>Spherical_Bessel.c</code>	ホットパスの呼び出し毎 <code>malloc/free</code> (漸化式係数配列)を除去し、係数をインライン計算化(数式は同一)
<code>openmx_common.h</code>	インクルードガードを追加(オリジナルには無かった)。バージョン文字列を "4.0" → "4.0 GPU" に変更
<code>Input_std.c</code>	GPU 関連キーワードの追加(第 10 章)
<code>mpao.c</code>	<code>#include <ctype.h></code> の 1 行差のみ(ビルド対象外のユーティリティで影響なし)

9. ビルドシステム

9.1 ツールチェーンの全面刷新

オリジナルの `makefile` (Intel oneAPI + MKL)は `Makefile` (NVHPC + CUDA + 自前 FFTW + GEMMu8)に 全面書き換えされた。原本は `makefile.cpu_intel.bak` としてバイト単位で同一保存されている。

項目	オリジナル(makefile)	GPU 版(Makefile)
C コンパイラ	<code>mpiicx -O3 -xHOST -qopenmp</code>	NVHPC 同梱 OpenMPI の <code>mpicc (nvc) + -acc -Minfo=accel -std=c11 -mtune=native -march=native -fopenmp</code>
Fortran	<code>mpiifx</code>	<code>mpif90 (nvfortran) -O2 -fopenmp</code>
BLAS/LAPACK	MKL(静的)	NVHPC 同梱の静的 <code>liblapack_lp64.a</code> / <code>libblas_lp64.a</code>
ScaLAPACK	MKL ScaLAPACK	HPC-X OpenMPI 側の <code>libscalapack_lp64.a</code>
FFT	MKL FFTW ラッパ	自前ビルドの静的 FFTW 3.3.11(third_party)
GPU ライブラリ	—	<code>-lcudart -lcusolver -lcublas -lcublasLt -lcuda -lnvidia-mt -lnvjitlink -cuda + libgemmul8.a</code>
CUDA C++	—	<code>nvcc</code> で <code>gemmul8_openmx.cu</code> / <code>set_hamiltonian_gpu.cu</code> をコンパイル(ホストコンパイラ <code>nvc++</code>)
SDK 前提	Intel oneAPI	<code>NVHPC_ROOT</code> <code>?</code> <code>= /opt/nvidia/hpc_sdk/Linux_x86_64/26.3</code> 、 <code>NVHPC_CUDA_VERSION</code> <code>?</code> <code>= 13.1</code> (変数で上書き可)

ビルド環境の隔離: `Makefile` 冒頭で `PATH` を NVHPC のみに固定し、`CPATH` / `LD_LIBRARY_PATH` / `CC` など 30 個超の環境変数を `unexport` する。ユーザー環境の `CFLAGS` 等は効かない(意図的)。

Makefile — コンパイラ定義(抜粋)

```
COMMON_INCLUDES = $(FFTW3_INCLUDE) -I$(NVHPC_MATH_DIR)/include -I$(NVHPC_CUDA_TARGET)/include -I$(NVHPC_CUDA_HOME)/include
COMMON_CFLAGS   = -w -Minfo=accel -acc -std=c11 -mtune=native -march=native -fopenmp $(COMMON_INCLUDES)

CC = $(MPICC) $(COMMON_CFLAGS) -O3
CC2 = $(MPICC) $(COMMON_CFLAGS) -O1
CC3 = $(MPICC) $(COMMON_CFLAGS) -O2
CC_NOACC = $(filter-out -Minfo=accel -acc,$(CC2))
CC_NOACC_O3 = $(filter-out -Minfo=accel -acc,$(CC))
CC_DFTD3 = $(MPICC) -w -O0 -std=c11 -fopenmp $(COMMON_INCLUDES)
FC = $(MPIF90) -O2 -mtune=native -march=native -fopenmp $(FFTW3_INCLUDE) -I$(NVHPC_MATH_DIR)/include
```

オリジナルは全ファイル `$(CC)` 一本だったのに対し、GPU 版は**ファイル別の最適化・OpenACC 調整**を行う (表 6-1)。OpenACC のターゲットアーキ指定(`-gpu=ccXX`)はなく、ビルドマシンの GPU を自動検出する。クロスターゲットしたい場合は `COMMON_CFLAGS` に追加する。

9.2 リンク構成

Makefile(抜粋)


```

NVPL_SCALAPACK_LIB ?= $(NVHPC_MPI_LIBDIR)/libscalapack_lp64.a
NVPL_LAPACK_LIB    ?= $(NVHPC_COMPILER_LIB)/liblapack_lp64.a
NVPL_BLAS_LIB      ?= $(NVHPC_COMPILER_LIB)/libblas_lp64.a
NVPL_LIBS          ?= -Wl,--start-group $(NVPL_SCALAPACK_LIB) $(NVPL_LAPACK_LIB) $(NVPL_BLAS_LIB) -Wl,--end-group
CUDA_LIBS          ?= ... -Wl,--no-as-needed -lnvJitLink -Wl,--as-needed -lcudart -lcusolver -lcublas -cuda
LIB = $(FFTW3_LIBS) $(NVPL_LIBS) $(MPI_FORTRAN_LIBS) -pgf90libs -mp -lpthread -lm -ldl $(CUDA_LIBS)
...
LIB += -lcublasLt -lcuda -lnvidia-ml -lstdc++

```

リンク行は `$(CC) $(OBJ) $(GEMMUL8_LIB) $(LIB) -lm -o openmx`。OBJ への追加分: `Total_Energy_GPU.o`、`Set_Density_Grid_GPU.o`、`set_hamiltonian_gpu.o`、`gemmul8_bridge.o`、`gpuserver_Syevd.o` `gpuserver_Syevdx.o` `set_cuda_default_device_from_local_rank.o` `set_openacc_device_from_local_rank.o`、`OutData_Binary.o` (呼び出し元のないデッドリンク)。かつて含まれていた `my_cublasDgemm.o` `my_cublasZgemm.o` は `a7f8cba6` で削除された。

9.3 third_party の自動ビルドと「make 一発」保証

既定ターゲットが `openmx: $(FFTW3_LIB) $(GEMMUL8_LIB) $(OBJ)` のため、素の `make` で FFTW3 → GEMMUL8 → 本体の順に全自動でビルドされる。コミット `9fe8ce13` で **クリーンツリーからの単一 `make (-j 並列可)` で完走**ことが保証された。修正は 4 点:

- FFTW3 ソースの自動クローン:** `third_party/fftw3` が無い場合、従来はエラーで停止していたが、現在は `git clone --branch fftw-3.3.11 --depth 1` を自動実行する(GEMMUL8 と同じ扱い)。git ソースには生成済み codelets が無いため、番兵ファイルを確認してリリース tarball (`fftw-3.3.11.tar.gz`、リポジトリにもコミット済み)から `rsync --ignore-existing` で補完 → `autoreconf` → `out-of-tree configure(--enable-static --disable-shared)`、コンパイラは `nvc/nvc++/nvfortran` → `make install` の既存チェーンに接続する。
- FFTW3_USERS の補完:** `fftw3.h` を include する `Cluster_DFT_Col.o` が order-only 依存(`| $(FFTW3_LIB)`)のリストに入っておらず、`-j` ビルドで FFTW の install 前にコンパイルが走って最初の 1~2 回の `make` が失敗していた。現在は `FFTW3_USERS = Poisson.o Stress.o Poisson_ESM.o TRAN_Poisson.o TRAN_CDen_Main.o TRAN_Set_Electrode_Grid.o Cluster_DFT_Col.o`。
- バンドル ELPA の Fortran モジュール依存エッジ:** `.mod` ファイル生成前にコンパイルが始まる レースを塞ぐため、テンプレート include 経由で到達可能な全 use 文をカバーする依存 (`elpa1_compute_real/complex.o → mod_precision.o`、`elpa2_compute_complex.o → elpa1_compute_real.o` ほか 4 件、`mod_compute_hh_trafo_* → 相互のカーネルモジュール`)を追加。
- gemmul8.hpp の order-only ルール:** GEMMUL8 チェックアウトが無い新規クローンで「No rule to make target」で死ぬのを防ぐ空レシピルールを追加。

Makefile — GEMMUL8 の取得と単一 TU ビルド

```

$(GEMMUL8_DIR)/Makefile:
    mkdir -p $(GEMMUL8_REPO_DIR)
    cd $(GEMMUL8_REPO_DIR) && \
    { [ -d .git ] || git init -q .; } && \
    { git remote get-url origin >/dev/null 2>&1 || git remote add origin $(GEMMUL8_REPO_URL); } && \
    git fetch --depth 1 origin $(GEMMUL8_COMMIT) && \
    git checkout -q $(GEMMUL8_COMMIT)
$(GEMMUL8_OPENMX_OBJ): gemmul8_openmx.cu $(GEMMUL8_DIR)/Makefile
    $(NVCC_GEMMUL8_ENV) $(NVCC) $(NVCC_GEMMUL8_FLAGS) -c gemmul8_openmx.cu -o $(GEMMUL8_OPENMX_OBJ)
$(GEMMUL8_LIB): $(GEMMUL8_OPENMX_OBJ)
    mkdir -p $(GEMMUL8_DIR)/lib
    ar rcs $(GEMMUL8_LIB) $(GEMMUL8_OPENMX_OBJ)

```

検証(コミットメッセージ): `make clean` 後、単一の `make -j16 openmx` が RTX 5080 機で 177 秒で完走。2 回目の `make` は no-op。コンパイルフラグは不変のため生成バイナリも不変。`clean` は `fftw3-clean` と `gemmul8-clean` を含むため、`make clean` 後は `third_party` の再ビルドが必要になる。

9.4 ELPA(elpa-2018.05.001)

`elpa-2018.05.001/` ディレクトリのソースは**オリジナルと完全同一**(9.3 の依存エッジ追加は Makefile 側の変更)。カーネルは generic CPU 版のまま(**ELPA の GPU カーネルは使っていない**)。違いは Fortran コンパイラが `mpiifx` → `nvfortran` になった点のみ。GPU 版でも $n < 1000$ の系・GPU 条件を満たさない経路・**VRAM 不足時の実行時フォールバック先**(§ 5.6)として ELPA は現役で使われるため、この維持は以前にも増して重要である。

9.5 派生 Makefile

- `makefile.cpu_intel.bak` — オリジナル makefile のバイト同一コピー(CPU/Intel 構成の保存)。
- `Makefile.bunshiken` — 共用計算機(モジュール環境: `nvhpc/25.9-nompi` + `openmpi/5.0.8` + `gcc-toolset/15`)向けの変種。NVHPC 25.9 + 外部 OpenMPI、CUDA 13.0、`nvcc` ホストコンパイラが `g++`、GEMMul8 が旧レイアウト(コミット固定なし)、オブジェクト名が `gpuserver` リネーム前の `cusolver_*.o` のまま、という点で**主 Makefile より一世代以上古い**。9.3 の「make 一発」修正も未反映。他環境へ移植する際の参考例として扱うこと。

9.6 ビルド手順と注意

1. `git clone --recursive` (submodule: `fftw3`, `GEMMul8`)。submodule を忘れても `make` が両方とも自動 clone/fetch する(9.3)。
2. `source/` で `make` (既定ゴール `openmx`、`-j` 可)。FFTW3 の取得・補完→`configure`→`install`、`GEMMul8` の取得→コンパイル→`ar`、CUDA カーネルの `nvcc` コンパイル、本体のコンパイル→リンクまで一括実行される。
3. `make install` で `../work/openmx` にコピー。ユーティリティ群は `make all`。

ビルド時の注意: (1) NVHPC のバージョンを変えると § 6.5 のミスコンパイル回避策の再検証が必要。(2) HPC-X のパスにはバージョン文字列(`hpcx-2.25.1`)がハードコードされており、SDK 更新時は修正が要る。(3) `GEMMul8` のアーキは `make GEMMUL8_GPU_ARCH=90` のように上書き可能(既定は `nvidia-smi` 自動検出)。(4) オリジナルにあった API ライブラリ(`libopenmx_api`)・`elpa_col` などの補助ターゲットは削除されている。

10. 実行時制御リファレンス

10.1 入力ファイルキーワード

GPU 機能に関して追加・拡張された入力キーワードは次の 2 つだけである(それ以外の調整はすべて環境変数)。

Input_std.c — 追加部分(全文)

```
/* "cusolver" is a deprecated alias of "gpusolver"; ... */
s_vec[0]="elpa2"; s_vec[1]="lapack"; s_vec[2]="elpa1"; s_vec[3]="gpusolver"; s_vec[4]="cusolver";
i_vec[0]=ELPA2; i_vec[1]=0; i_vec[2]=ELPA1; i_vec[3]=GPUSOLVER; i_vec[4]=-GPUSOLVER;
input_string2int("scf.eigen.lib", &scf_eigen_lib_flag, 5, s_vec,i_vec);
if (scf_eigen_lib_flag==GPUSOLVER){
    if (myid==Host_ID){
        printf("Warning: scf.eigen.lib=cusolver is deprecated; use scf.eigen.lib=gpusolver instead.\n");
    }
    scf_eigen_lib_flag = GPUSOLVER;
}
scf_eigen_lib_flag_input = scf_eigen_lib_flag;

/* use GPU? (added by H.Kawai, February 2024) */
input_int("scf.Gpu.Num", &SCF_Gpu_Num, 30);
```

キーワード	型	既定値	意味
scf.eigen.lib	文字列選択	elpa2	従来の elpa2 / lapack / elpa1 に gpusolver (GPU 経路一式の有効化)を追加。 cusolver は警告付きの非推奨エイリアス。GPU 不在時は自動的に ELPA2 に降格
scf.Gpu.Num	int	30	ノード内で使用する GPU 台数の上限。現在は DC-LNO の GPU 台数切り詰めのみ使用(既定 30 は実質「検出された全 GPU を使う」)

10.2 環境変数一覧

GPU 実行のチューニングは環境変数で行う設計である(入力ファイルを変えずにジョブスクリプト側で調整できる)。**OPENMX_**接頭辞版と **GEMMUL8_** 接頭辞版がある変数は **OPENMX_** 版が優先される。第 2 版以降に追加された変数が多数あるため、サブシステム別に示す。**ほぼすべての GPU 経路が既定で有効**であり、環境変数は主に「無効化」「メモリ予約の調整」「強制」に使う。

環境変数	既定値	意味
GEMMUL8(gemmul8_bridge.cu)		
(OPENMX_)GEMMUL8_DISABLE / _D / _Z	off	GEMMUL8 を無効化し cuBLAS FP64 に固定(全体 / 実数のみ / 複素のみ)
(OPENMX_)GEMMUL8_NUM_MOD_D / _Z	15	法の個数(精度)。範囲 2~20、範囲外は 15 に戻る
(OPENMX_)GEMMUL8_FASTMODE_D / _Z	0	高速スケーリングモード(精度低下)
(OPENMX_)GEMMUL8_MAX_WORKSPACE_PERCENT	30	ワークスペースの総 VRAM 比上限(Band_DFT_NonCol は未設定時 50 を setenv)
(OPENMX_)GEMMUL8_MIN_FREE_AFTER_MB	1536	ワークスペース確保後に残すべき最低空き VRAM
バンド計算(Band_DFT_Col.c / Band_DFT_NonCol.c)		
OPENMX_BAND_GPU_DIAG	自動	1=見積り判定を無視して GPU 密行列経路を強制、0=常に ELPA/ScaLAPACK(§ 5.6)
OPENMX_BAND_GPU_RESERVE_MB	max(総 VRAM/10, 1536)	並列度決定・fits 判定で空けておく予約量の引き上げ

環境変数	既定値	意味
<code>OPENMX_BAND_GPU_RANK_OVERHEAD_MB</code>	256	per-rank 所要量見積りに加算するランタイムオーバーヘッド
<code>OPENMX_BAND_SYEVDX_NOVECTOR</code>	1	固有値のみ必要な第 1 パスを NOVECTOR で解く(0 で旧動作)
<code>OPENMX_BAND_GPU_MAX_CONCURRENT_K</code> / <code>OPENMX_BAND_GPU_TURN_GROUP</code> / <code>OPENMX_BAND_GPU_MAX_RANKS_PER_DEVICE</code>	制限なし	同時実行ランク数の 明示上限 (この優先順で最初に見つかったもの)。旧版の「初期値」の意味は廃止され、実際の並列度は常に適応スキャンで決まる
<code>OPENMX_GPUSOLVER_SERIAL_GPU_TURNS</code>	1	<code>all_knum==1</code> 経路の GPU ターン直列化(0 で並行投入)
<code>OPENMX_BAND_GPU_PERSISTENT</code> / <code>_MIN_FREE_MB</code>	0 / 2048	SCF 反復間のデバイスバッファ常駐(共有 GPU では非推奨)
<code>OPENMX_BAND_GEMMUL8_TURN_RELEASE</code>	1	ターン毎の GEMMUL8 ワークスペース解放(0 で保持)
<code>OPENMX_BAND_PROFILE</code>	0	性能計測カウンタの表示(Set_Hamiltonian の SETHPROF 行も共用)
クラスタ計算(Cluster_DFT_Col.c / Cluster_DFT_NonCol.c)		
<code>OPENMX_CLUSTER_GPU_DIAG</code>	自動	1 =実予約失敗でも GPU 強制(失敗時は診断つき abort)、 0 =常に ELPA(§ 5.6)
<code>OPENMX_CLUSTER_GPU_DIAG_RESERVE_MB</code>	1024	予約プローブ後に残すべき空き VRAM
<code>OPENMX_CLUSTER_GPU_DIAG_SERIAL</code>	1	2 スピンが同居できないときの直列化モード(0=無効、2=直列強制)
<code>OPENMX_CLUSTER_GPU_DM</code>	1	DM/EDM の GPU 蓄積(0 で CPU ループ)
<code>OPENMX_CLUSTER_GEMMUL8_TURN_RELEASE</code>	1	ソルブ毎の GEMMUL8 ワークスペース解放(未設定時は BAND 設定を継承)
DC 法(Divide_Conquer.c)		
<code>OPENMX_DC_GPU_THRESHOLD</code>	1000	GPU 化する局所クラスタ次元の下限
<code>OPENMX_DC_GPU_S_CACHE</code> / <code>_RESERVE_MB</code>	1 / 4096	S12 パネルのデバイス常駐キャッシュとその予算予約
<code>(OPENMX_)GPUSOLVER_MIN_FREE_AFTER_MB</code>	1536	固有値ソルバ用確保後の最低空き VRAM(preflight)
<code>OPENMX_CUBLAS_MIN_FREE_AFTER_MB</code>	—	cuBLAS 用の同上
DC-LNO 法(Divide_Conquer_LNO.c)		
<code>OPENMX_DCLNO_GPU_THRESHOLD</code>	800	GPU 化する局所次元の下限(コリニア・非コリニア共通)
<code>OPENMX_DCLNO_GPU_TURN_RELEASE</code>	1	フェーズ末尾のソルババッファ+GEMMUL8 解放(未設定時は BAND 設定を継承)
Krylov 法(Krylov.c)		
<code>OPENMX_KRYLOV_GPU_FUSED</code>	1	原子別パイプラインの融合パス(0 で per-call シーケンス)
<code>OPENMX_KRYLOV_GPU_KU_CACHE</code> / <code>_KU_RESERVE_MB</code>	1 / 4096	Krylov 基底のデバイス常駐キャッシュとその予算予約
<code>OPENMX_KRYLOV_GPU_DGEMM_MIN_FLOPS</code>	5×10^7	GEMM を GPU に投げる下限($m \cdot n \cdot k$)
<code>OPENMX_KRYLOV_GPU_EIGEN_MIN</code>	1000	部分空間対角化をデバイスで解く下限次元(下げると大幅に速くなる系がある。 § 7.6)
<code>OPENMX_KRYLOV_GPU_PROFILE</code>	0	融合パスの内訳(KRYFUSE 行)出力
Set_Hamiltonian(Set_Hamiltonian.c / set_hamiltonian_gpu.cu)		
<code>OPENMX_SETHAM_MATRIX_GPU</code>	1	dVH+Vxc+VNA 格子積分の GPU 実行(0 で CPU)
<code>OPENMX_SETHAM_CUDA_KERNEL</code>	1	専用 CUDA カーネル(0 で OpenACC カーネルのみ。数値同一)
<code>OPENMX_SETHAM_GPU_RESIDENT</code> / <code>_FLOOR_MB</code>	1 / 4096 or 16384	SCF 不変表のデバイス常駐と、常駐後に空けておくフロア(対角化需要の公開有無で既定が変わる。 § 7.7)
<code>OPENMX_SETHAM_GPU_MAX_RANKS_PER_DEVICE</code>	制限なし	波の同時ランク数の明示上限
<code>OPENMX_SETHAM_GPU_SERIAL_WAVES</code>	0	1 で全波を GPU 直列実行(既定は第 1 波 GPU+残余ランク CPU 並行のハイブリッド)

環境変数	既定値	意味
<code>OPENMX_SETHAM_GPU_RESERVE_MB</code> / <code>_RANK_OVERHEAD_MB</code>	max(総/10,1536) / 512	予約量とランクあたりオーバーヘッド見積り
<code>OPENMX_SETHAM_GPU_KOWNER_ONLY</code>	0	1 で旧来の k オーナー限定 GPU 実行に戻す
<code>OPENMX_SETHAM_BASE_GPU</code>	0	H の base 結合を GPU 実行(実測では CPU が速い)
<code>OPENMX_SETHAM_GPU_TEST_NEED_MB</code> / <code>_FROM_CALL</code>	—	偽のフェーズ需要を注入するテスト用フック
その他の行列要素構築		
<code>OPENMX_SETNL_OPENACC</code>	1	Set_Nonlocal の GPU バッチ(0 で CPU パス)
<code>OPENMX_SETPRO_GPU</code>	1	Set_ProExpn_VNA の 3 バッチ(0 で CPU)
<code>OPENMX_OLPKIN_RADIAL_GPU</code>	0	Set_OLP_Kin の動径積分を GPU 実行(スレッド数 1 必須。転送律速で通常は CPU が速い)
密度グリッド(Set_Density_Grid_GPU.c)		
<code>OPENMX_DENSITY_GRID_GPU</code> (別名 <code>OPENMX_SETDENSITY_GPU</code>)	1	密度グリッド GPU 全体のゲート
<code>OPENMX_DENSITY_GRID_GPU_LOCAL</code>	1	分散求積モード: 0=無効、1=共有テーブル常駐時のみ、2=常に
<code>OPENMX_DENSITY_GRID_GPU_RESERVE_MB</code> / <code>_PERSISTENT</code>	256 / 0	オーナーサービスの予約量と計算後のデバイス常駐
全エネルギー・力(Total_Energy.c / Force.c)		
<code>OPENMX_TOTALENERGY_GPU</code>	1	全エネルギー系デバイスカーネルの一括ゲート(NC 含む)
<code>OPENMX_FORCE_GPU</code>	1	力計算の全 GPU 経路の親ゲート(集団判定)
<code>OPENMX_FORCE3_GPU</code> / <code>OPENMX_FORCE3_GPU_DORB</code>	1 / 1	Force3 トレースの GPU 化 / dOrbitals のオンデバイス再計算
<code>OPENMX_FORCE4B_GPU</code> / <code>OPENMX_FORCE4B_FUSED</code>	1 / 1	Force4B の GPU バッチ / CPU 側融合トレース(GPU バッチの前提でもある)
<code>OPENMX_FORCEHNL_GPU</code>	1	Force_HNL の GPU バッチ
<code>OPENMX_FORCE_PROFILE</code>	0	[Force profile] 行(ステージ別 max/avg 秒)の出力
電荷/DM 混合(Mixing_H.c ほか)		
<code>OPENMX_MIX_FAST</code>	1	RMM-DIIS/DIISK/DIISV/GR-Pulay/Kerker の CPU 高速パス(0 でレガシー)
<code>OPENMX_MIXH_GPU</code>	自動	RMM-DIISH の層選択: 未設定=自動、0=レガシー、1=GPU 層強制、2=増分 CPU 層強制
<code>OPENMX_MIXH_GPU_RESERVE_MB</code> / <code>_VERBOSE</code>	max(総/10,1536) / 0	GPU 層の予約量 / 選択結果の表示

表 10-1: GPU 関連環境変数の一覧(サブシステム別)

10.3 標準出力の GPU 診断行

コミット 30148e1e ほかで、どのステージが GPU 実行されたかが標準出力から読み取れるよう整備された。代表的な行(いずれも Host_ID または各デバイスの代表ランクが出力):

- ステージバナー: `<Set_Hamiltonian> Hamiltonian matrix for VNA+dVH+Vxc (GPU-accelerated)...`、`<MD=...> Calculation of the VNA projector matrix (GPU-accelerated)`、`Force calculation #2 (GPU reduction)` / `#3 (GPU-accelerated)` / `#4 (GPU-accelerated)` / `#5 (GPU reduction)`、`Force calculation #6/#7 (GPU-accelerated)` (Total_Energy 側)、`<Set_Density_Grid> distributed GPU quadrature: every rank integrates its own atoms`

- 並列度の決定(値が変わったときのみ): `<Band_DFT_Col> GPU device 0: 8 k-owner rank(s), GPU concurrency=3, turn group=3, per-rank=1.9 GiB, peak=5.7 GiB, free=7.4 GiB, reserve=1.5 GiB` (NonCol はモード名付き、Set_Hamiltonian は Hamiltonian rank(s) 表記)
- キャッシュ採用: `<DC> GPU S-cache: rank 0 keeps 12 of 16 cluster overlaps on the device (812.5 MiB)`、`<Krylov> GPU KU-cache: ...`、`<Set_Hamiltonian> GPU resident cache: ...`
- フォールバック通知(1 回のみ/ノードごと 1 回): `Falling back to the ELPA/ScaLAPACK diagonalization...` (§ 5.6)、`Force4B GPU: not enough free device memory; CPU fallback.`、`<Mixing_H> The RMM-DIISH residual history ... does not fit on the GPU ...`、GEMMu8 の `workspace fallback (stderr)`

読み方:「GPU で走っているつもりなのに遅い」ときは、まずこれらの行を確認する。(a) ELPA フォールバック行が出ていれば VRAM 不足で対角化が CPU に落ちている、(b) GEMMu8 の workspace fallback が出ていれば GEMM が FP64 に落ちている、(c) concurrency=1 が出ていれば共有ランクが直列化されている — の 3 つが典型である。

10.4 チューニング指針

1. **まず既定値で走らせる** — 既定値は「16 GB 級 GPU を多数ランクで共有」する構成に合わせて調整されており、並列度・常駐・フォールバックはすべて自動で決まる。旧版のように環境変数で並列度を手動指定する必要は原則なくなった。
2. **stdout / stderr の診断行を確認する** (§ 10.3) — チューニングの出発点は常にここ。
3. **VRAM に余裕がある (GPU 占有) 場合** — `OPENMX_BAND_GPU_PERSISTENT=1`、`OPENMX_GPUSOLVER_SERIAL_GPU_TURNS=0`、`OPENMX_*_GEMMU8_TURN_RELEASE=0` で転送・再確保を減らせる余地がある。Krylov 系では `OPENMX_KRYLOV_GPU_EIGEN_MIN` を下げると対角化がデバイスに移り大幅に速くなる系がある(実測 117.5 s → 78.8 s。§ 7.6)。
4. **VRAM が厳しい場合** — ランク数を減らすのが最も効く。次いで予約系 (`*_RESERVE_MB`) を既定より上げて GEMMu8 フォールバックを防ぐ、`OPENMX_SETHAM_GPU_RESIDENT=0` で常駐を切る、など。
5. **再現性を最優先する場合** — `OPENMX_GEMMU8_DISABLE=1` で GEMM を FP64 に固定し、メモリ状況によるエンジン切替の影響を排除する (§ 4.8)。GPU⇄ELPA の切替も含めて固定したい場合は `OPENMX_BAND_GPU_DIAG` / `OPENMX_CLUSTER_GPU_DIAG` を明示する。
6. **問題の切り分け** — GPU 経路のどこかを疑う場合は、サブシステム別ゲート (`OPENMX_SETHAM_MATRIX_GPU=0`、`OPENMX_FORCE_GPU=0`、`OPENMX_MIX_FAST=0` など) を 1 つずつ 0 にして二分探索できる。すべての高速パスはレガシーコードを温存している。

11. 制約・注意点の総まとめ

11.1 ビルド・実行環境の前提

- このツリーは CPU 専用にビルドできない(CUDA ヘッダを無条件 include)。CPU 版が必要なら オリジナルまたは `makefile.cpu_intel.bak` を使う。
- NVHPC 26.3 + CUDA 13.1 が既定。バージョン変更時は § 6.5 のミスコンパイル回避一覧を再検証すること。
- GEMMu8 は INT8 Tensor Core 必須・単一アーキビルド。GPU 世代を変えたら再ビルド。
- GPU 実行時は MPI のみ(OpenMP スレッド 1)の運用が想定されている。`Set_OLP_Kin` の GPU 経路は `Nthrs==1` を明示的に要求する。混合の高速パスの一部(バッチ Gram)は 1 スレッドのときレガシーと加算順序が一致する。

11.2 メモリ設計の目安

- 1 ランクあたりの VRAM 需要(バンド計算・次元 n): 複素密行列 $16n^2$ バイト $\times$ 3 本 + syevdx ワークスペース (`_bufferSize` の実照会値。目安 $4n^2$ 複素要素)+ GEMMu8 ワークスペース(約 1 GiB、複素は実数の約 3 倍) + オーバーヘッド 256 MiB。この見積りは `BandCol_GpuTurnRequiredBytes` がそのまま実装している。
- 共有 GPU では「同時ランク数 $\times$ 上記」が空き VRAM に収まる必要があるが、現在は**全経路が自動で並列度を縮退・直列化・ELPA フォールバック**するため、収まらない設定でもハングや abort にはならない(性能が落ちるだけ)。旧版にあった「Cluster root-dense や DC-LNO には自動絞り込みがない」という制約は解消済み。
- ホストメモリにも注意: `Set_Hamiltonian` / 密度グリッドの共有テーブルは数 GiB 級で、CPU 実行ランクに構築させない制御(Ready プロープ)が入っている。ランク数を増やすとホスト側の ワークスペース(混合履歴・クラスタクラッチ等)がランク数倍で効く。

11.3 数値再現性

- **ビット単位の再現を壊し得る要因:** (1) GEMMu8 $\rightleftharpoons$ cuBLAS FP64 のメモリ状況による切替、(2) GPU $\rightleftharpoons$ ELPA の実行時フォールバック(verdict は SCF 内 sticky だが、実行ごとのメモリ状況で変わり得る)、(3) scatter-add の `acc atomic` (加算順非決定)、(4) S の小固有値処理の経路差、(5) GEMMu8 の環境変数設定差(`num_moduli` / `fastmode`)。
- **再現に配慮した設計:** DM 構築・格子トレースの `loop seq` 逐次縮約、カバッチの「float 積 $\rightarrow$ double 加算」のキャスト位置まで含めた CPU 再現、混合高速パスのポインタ回転(ビット単位の同値性)、cublasDgeam 転置(純データ移動)、GEMMu8 自体の bitwise 再現性、ベッセルテーブルの bit 一致キャッシュ。多くの GPU 化コミットは「Uele/力が CPU とビット一致」を検証条件にしている。
- 検証時は `OPENMX_GEMMU8_DISABLE=1` と十分な VRAM 余裕、必要なら `OPENMX_*_GPU_DIAG` の明示固定で条件を固定するのが確実。

11.4 ハング・異常終了

- `wait_cudafunc` は恒久エラーで無限ループする(§ 3.6)。ただし「VRAM が恒久的に足りない」ケースは 有界リトライ+フォールバック(`TryDeviceMalloc` 系、`Force_gpu_arena_wait` の 180 秒上限)に移行済みで、無言のフリーズになる箇所は大幅に減った。カスステージには abort し得るデバイス確保が存在しない(§ 7.11)。
- 固有値計算失敗時の挙動は経路により abort / CPU フォールバック / 素通りと異なる(表 5-2)。
- fits 判定・波計画・混合 preflight は内部で MPI 集団操作を行うため、**全ランクが同じ回数だけ到達**することが前提。経路改修時に片側のランクだけスキップさせるとデッドロックする(§ 5.9)。
- collective 版デバイス割当関数は全ランカー斉呼び出しが前提(片側だけ呼ぶとデッドロック)。

- `cudaGetLastError()` のスレッド別ラッチは「回復済み失敗」の残骸を含み得る。自前のカーネル検査を追加する場合は起動前に `pop` する作法を守ること (§ 3.6、§ 7.7)。

11.5 AMD GPU など他ベンダーへの移植観点

- `gpuserver` 命名は準備済みだが、切替はマクロ等で機構化されていない。`cuSOLVER` 依存点は `gpuserver_Syevd(x).c` と各ソルバの常駐コンテキスト(`Xsyevdx` 直接呼び出し)に集約されている。
- `cuBLAS` 依存点は `Ddgmm/Zdgmm・Dgeam/Zgeam` と `GEMMmul8` ブリッジ(内部 `INT8 GEMM` とフォールバック `GEMM`)。 `GEMMmul8` は上流に `HIP/ROCm` バックエンドがあるが、本統合(単一 TU 実体化)は `CUDA` 経路のみを実体化している。
- `OpenACC` は `NVHPC` 依存(AMD なら `OpenMP target offload` などへの書き換えが必要)。`acc_clear_freelists()` による `OpenACC` プール制御 (§ 6.3)、GPU UUID による共有検出、`wait_cudafunc` の「戻り値 0 = 成功」前提も移植時の確認点。

11.6 オリジナルから挙動が変わる非 GPU 項目

- `OpenMP` 並列時のヒープ破壊バグ 2 件が修正済み (§ 8.1) — オリジナルにも存在するバグ。
- 起動時にスタックのソフトリミットが自動で引き上げられる (§ 8.2)。
- `runtest` の判定・実行手順が厳密化されている (§ 8.3)。
- バージョン文字列は "4.0 GPU"。

11.7 休眠・準備コード一覧(改修時の誤解防止)

コード	状態と位置づけ
<code>gpuserver_Syevd</code> (<code>Xsyevdx</code> 版)	呼び出し 0 件(<code>Syevdx</code> 版に統一された)。旧式の「大域 rank % デバイス数」割当が残る
<code>LU_Zinverse_GPU</code>	実装完了・呼び出しコメントアウト。 <code>NEGF</code> 表面グリーン関数の GPU 化布石 (§ 5.8)
<code>DMmu/CWF/LNO</code> 系の <code>GEMMmul8</code> ヘルパ、 <code>BandCo</code> <code>l_GpuSolver_Zgemm_OpenACC</code> など	定義のみ(将来の <code>GEMM GPU</code> 化の準備)。 <code>my_cublasDgemm/Zgemm</code> は削除済み、 <code>openmx_cuda_kernels.cu</code> はツリーから撤去済み (§ 6.4)
<code>DC-LNO</code> の GPU プロキシ機構	<code>DCLNO_ENABLE_SHARED_GPU_PROXY 0</code> で温存(実験用)
<code>use_force4b_openacc = 0</code> (<code>Force.c</code>)	<code>Force4B</code> の旧 <code>OpenACC</code> 経路の恒久無効化定数(現行は GPU バッチ+融合 CPU トレースが実経路)
<code>OutData_Binary.o</code> / <code>getDeviceCount()</code> 宣言	デッドリンク / デッド宣言
<code>OPENMX_SETHAM_GPU_TEST_NEED_MB</code> 系フック	テスト専用(フェーズ需要の注入)。運用では使わない

表 11-1: 休眠・準備コードの一覧

付録 A. 変更ファイル一覧(47 ファイル)

+n/-m はオリジナル OpenMX 4.0 ソースとの生の diff 行数(2026-07-23 時点)。全面再整形されたファイルは raw 行数が実質変更より大きく見える点に注意。分類: **GPU**=GPU 化、**OMP**=OpenMP 修正、**FIX**=バグ修正・堅牢化、**OPT**=CPU 最適化、**BLD**=ビルド、**MISC**=その他。

ファイル	+n/-m	分類	変更概要
Force.c	+15,123/-11,241	GPU	力 #2/#3/#4/#4B/#5/HNL の GPU 化(融合トレース、チャンクバッチ、dOrbitals オンデバイス、アリーナ+ピア待ち)。raw 行数の一部は全面再インデント
Divide_Conquer.c	+4,816/-3,038	GPU	DC 法の GPU 化: 原子別クラスタの Xsyevdx、GEMMul8→cuBLAS→CPU 多段 GEMM、S12 デバイス常駐キャッシュ、cublasDgeam 転置、sticky 無効化
Band_DFT_Col.c	+4,724/-2,544	GPU	コリニアバンドの全面 GPU 化: 密行列構築キャッシュ、Xsyevdx(第 1 パス NOVECTOR)、GEMMul8 三重積、適応並列度、ELPA フォールバック
Band_DFT_NonCol.c	+3,582/-88	GPU	非コリニアバンドの GPU 化: OpenACC 常駐 root-dense 経路、ステージ別(固有値/固有ベクトル/DM)適応並列度、ELPA フォールバック
Set_Hamiltonian.c	+2,972/-1,149	GPU	dVH+Vxc+VNA 格子積分の GPU 化(既定有効): 適応波スケジューリング、SCF 不変キャッシュ、オンデバイス Vpot ギャザー、常駐キャッシュ+需要協調、packed H/S キャッシュ API
Cluster_DFT_NonCol.c	+2,424/-218	GPU	単一アリーナの root-dense 対角化、GEMMul8 × 12、S/Ss2・index・ハンドル・ワークスペースの反復間キャッシュ、固有ベクトル常駐/退避、ELPA フォールバック
Divide_Conquer_LNO.c	+2,315/-1,896	GPU	direct per-rank dispatch(コリニア)+非コリニアクラスタ解の GPU 化(スピン倍化変換+Zgemm×3)、共有ランク群の合算メモリ判定
Set_ProExpn_VNA.c	+2,029/-0	GPU	DS_VNA 構築・HVNA トレース・VNA2/3 の 3 バッチ GPU 化(球ベッセルのデバイス漸化、Gaunt/変換表のホスト表引き化)
Set_Nonlocal.c	+1,738/-305	GPU/FIX	非局所射影積分のバッチ GPU 化(デバイス生成球ベッセル+Gaunt 事前表)+固定配列のヒープ化
Cluster_DFT_Col.c	+1,534/-90	GPU	root-dense GPU 対角化、実予約ブロープ+ELPA フォールバック、2 スピン直列化、DM/EDM の GPU 蓄積、固有ベクトルのデバイス常駐
Mixing_H.c	+1,350/-0	GPU	RMM-DIISH 混合の 2 層高速パス(GPU 常駐 Gram / 増分 CPU)
Krylov.c	+840/-24	GPU	原子射影解パイプラインの融合(GEMM×3+Xsyevd を 1 回のデバイス訪問)、KU キャッシュ、スレッド別ワークスペース
Set_OLP_Kin.c	+528/-180	GPU/OPT	動径積分の OpenACC 化(オプトイン)+bit 一致ベッセルテーブルキャッシュ
Total_Energy.c	+367/-123	GPU/OMP	EH0 二中心バッチ・EXC/EH1・双極子・CWF-Dc・Exc0 細メッシュの GPU 縮約呼び出し(NC 対応)+OpenMP private 修正
DIIS_Mixing_DM.c	+265/-2	OPT	RMM-DIIS のバッチ Gram(1 掃引+1 Allreduce)+残差履歴のポインタ回転
Mixing_V.c	+246/-53	OPT	RMM-DIISV の増分 Gram、Kerker(VKSk)のシフト高速化
DFT.c	+235/-4	GPU	GPU デバイス初期化、Set_Hamiltonian 作業ランク選定、力計算直前の GPU キャッシュ一括解放フック、VNA バナー、NonCol 固有ベクトル scatter
DIIS_Mixing_Rhok.c	+228/-44	OPT	RMM-DIISK の増分 Gram(新残差行のみ dgemv+短い Allreduce)
GR_Pulay_DM.c	+204/-2	OPT	GR-Pulay のバッチ Gram+融合混合掃引
Eigen_lapack.c	+143/-8	GPU	n≥1000 で gpusolver_Syevdx へ分岐、OpenACC 直結版を追加。呼び出し元 20+ ファイルが自動恩恵

ファイル	+n/-m	分類	変更概要
<code>openmx_common.h</code>	+142/-1	GPU/MISC	CUDA ヘッダ取込、GPUSOLVER enum、wait_cudafunc(エラーラッチ 掃除つき)、GPU 系プロトタイプ、SetHamiltonianMETables 型、 include ガード、Version "4.0 GPU"
<code>Lapack_LU_inverse.c</code>	+134/-4	GPU	cublasZgetrf/ZgetriBatched による LU_Zinverse_GPU を追加(呼び 出しはコメントアウト=休眠)
<code>EigenBand_lapack.c</code>	+112/-3	GPU	複素版: gpusolver_zheevx 分岐、失敗時 Eigen_HH フォールバック
DMmu/CWF/LNO/分極系 11 ファ イル(§ 7.13)	各 +27~76	GPU	ELPA2 分岐手前への定型 GPUSOLVER 分岐挿入(dense_utils 経由 Dsyevx/Zheevx)
<code>Kerker_Mixing_Rhok.c</code>	+62/-10	OPT	Rhok 履歴シフトの高速化(回転+memmove)
<code>openmx_common.c</code>	+60/-0	GPU	GPU フェーズ需要レジストリ(OpenMX_GpuPhaseNeed_*)
<code>Set_Density_Grid.c</code>	+40/-5	GPU	分散 GPU 求積 → 単一ランク GPU サービス → CPU の 3 段ディスパッ チ
<code>Free_Arrays.c</code>	+28/-17	GPU/FIX	終了時の GPU キャッシュ無効化+GEMMu8 ワークスペース解放+free 後 NULL 代入
<code>truncation.c</code>	+23/-13	FIX/GPU	VNA/VEF グリッドの free 後 NULL 代入+GPU キャッシュ無効化フック
<code>Runtest.c</code>	+19/-7	FIX	.dat 判定の厳密化、テスト前の旧 .out 削除
<code>openmx.c</code>	+16/-0	FIX	Raise_Stack_Limit()(スタック上限自動引き上げ)
<code>Input_std.c</code>	+15/-3	GPU	scf.eigen.lib への gpusolver 追加(cusolver は非推奨エイリアス)、 scf.Gpu.Num 追加
<code>Spherical_Bessel.c</code>	+12/-24	OPT	ホットパスの毎呼び出し malloc/free を除去
<code>tran_prototypes.h</code>	+10/-0	GPU	CUDA_CHECK マクロ追加
<code>Set_Orbitals_Grid.c</code>	+8/-3	GPU	軌道グリッド再構築時の GPU キャッシュ無効化フック
<code>Stress.c</code>	+1/-1	OMP	OpenMP private リストに i を追加(ヒープ破壊修正)
<code>mpao.c</code>	+1/-0	MISC	include 1 行差のみ(ビルド対象外)
<code>makefile</code> → <code>Makefile</code>	全面改修	BLD	Intel oneAPI+MKL 構成から NVHPC+CUDA+自前 FFTW+GEMMu8 構成へ(第 9 章)。クリーンツリーから make 一発で完走

補足: 削除されたソースは `my_cublasDgemm.c` / `my_cublasZgemm.c` (休眠ラッパ) の 2 件。 `elpa-2018.05.001/` 以下のソースは完全同一(Makefile
側の依存エッジのみ追加)。第 2 版で「オリジナルと差分なし」だった `Set_ProExpn_VNA.c` と、混合 6 ファイル・密度グリッドは 本版までの 40 コミ
ットで新たに変更された。

付録 B. 新規追加ファイル一覧

B.1 新規ソース(14 ファイル、計約 4,000 行)

ファイル	行数	役割
<code>Set_Density_Grid_GPU.c</code>	1,366	密度グリッドの GPU 化(単一ランク GPU サービス+分散 GPU 求積)
<code>Total_Energy_GPU.c</code>	1,064	全エネルギー計算の OpenACC カーネル集(EXC/EH1・双極子・CWF-Dc・EH0 パッチ・Exc0 細メッシュパッチ)。-acc 分離ビルドのため別 TU
<code>gpusolver_Syevdx.c</code>	516	cusolverDnXsyevdx による部分固有値分解ラッパ(実/複素 × ホスト/OpenACC/キャッシュ付きの 5 変種)。GPU 対角化の実体
<code>gemmul8_bridge.cu</code>	419	GEMMul8 呼び出しブリッジ: ワークスペースキャッシュ、サイズ照会 API、環境変数制御、cuBLAS フォールバック
<code>set_hamiltonian_gpu.cu</code>	194	Set_Hamiltonian 格子積分の共有メモリアイランド型 CUDA カーネル(stale エラー耐性つき)
<code>gpusolver_Syevd.c</code>	131	cusolverDnXsyevd 版ラッパ(NVIDIA CUDA Library Samples 由来)。現在は未使用
<code>openmx_gpusolver_dense_utils.h</code>	79	密対角化の共通ディスパッチヘルパ(0/1-based 変換+失敗時 MPI_Abort)。11 ファイルが使用
<code>gemmul8_openmx.cu</code>	67	GEMMul8 テンプレートの単一 TU 明示的実体化(4 シンボルのみ)
<code>set_cuda_default_device_from_local_rank.c/.h</code>	73+9	ノード内 local rank による CUDA デバイスのラウンドロビン割当
<code>set_openacc_device_from_local_rank.c/.h</code>	79+12	同上の OpenACC 版(acc_set_device_num)
<code>LU_Zinverse_GPU.h</code>	9	GPU 版複素 LU 逆行列の宣言(休眠)

補足: 過去に存在した汎用 GEMM ラッパ `my_cublasDgemm.c` / `my_cublasZgemm.c` は削除された(§ 3.5)。第 2 版に記載した未結線カーネル集 `openmx_cuda_kernels.cu/.h` もソースから撤去された(§ 6.4)。

B.2 ビルド・文書ファイル

ファイル	役割
<code>Makefile</code> (1,377 行)	新ビルドシステム本体。各種スパコン向け構成例のコメント付き。NVHPC 26.3 + CUDA 13.1 既定。クリーンツリーから make 一発(-j 可)で third_party 込みの全ビルドが完走(§ 9.3)
<code>Makefile.bunshiken</code> (1,366 行)	共用計算機向け変種(NVHPC 25.9-nompi + OpenMPI 5.0.8 + gcc-toolset-15)。一世代以上古い構成
<code>makefile.cpu_intel.bak</code> (1,383 行)	オリジナル makefile のバイト同一バックアップ(Intel CPU 構成)
<code>third_party/</code>	GEMMul8(submodule、コミット 884a287 固定)、fftw3(submodule、タグ fftw-3.3.11)、fftw-3.3.11.tar.gz(codelets 補完用)。いずれも欠けていれば make が自動取得
<code>doc/</code>	本書(日本語/英語、HTML/PDF)

付録 C. 開発履歴(全 91 コミット)

#	ハッシュ	題名(原文)
1	0cf2acbc	Initial commit
2	60b6aa1e	Performed GPU acceleration
3	2f94ca06	Performed GPU acceleration on Divide_Conquer_LNO.c
4	e3fc656e	Optimized Band_DFT_NonCol.c
5	24e8b34f	Optimized Cluster_DFT_Col.c
6	33e9459b	Optimized Cluster_DFT_Col.c
7	82579d12	Bug fix
8	33a07450	Fixed bugs in Band_DFT_Col.c and Cluster_DFT_Col.c
9	b4221bba	Fixed a bug in runtestL
10	18d92700	Bug fix
11	563f69bd	Add files that were not included in the commit
12	1b020264	Optimized Band_DFT_Col.c and Band_DFT_NonCol.c
13	f4041307	Optimized Set_Hamiltonian.c
14	9f1633da	Optimized Set_OLP_Kin.c
15	d6b463a8	Explicitly indicate that the generation of overlap matrices is GPU-accelerated
16	5383544b	Optimized Total_Energy.c
17	c9e111ea	Optimized Set_ProExpn_VNA.c
18	8162b8ec	Optimized Set_ProExpn_VNA.c and Total_Energy.c
19	91416fb1	Bug fix
20	24a113a9	Add files that were not included in the commit
21	87ea7062	Bug fix
22	faf76fa2	Bug fix
23	a8224724	Add .gitignore for build artifacts and runtime output
24	d3c45cb3	Optimized Bnad_DFT_NonCol.c
25	db7b9327	Add files that were not included in the commit
26	a61f24b3	Bug fix
27	d6d19f99	Add README.md
28	a91f920f	Bug fix
29	88b135f4	Optimized Krylov.c
30	34cf8bc1	Optimized Divide_Conquer_LNO.c
31	685a2bc9	Reverted Set_ProExpn_VNA.c to the CPU version
32	2aa31a43	Add files that were not included in the commit
33	d3effce1	Bug fix of Divide_Conquer.c
34	34b52269	GPU acceleration for paths where all_knum !=1 in Band_DFT_Col.c
35	75ef67c7	GPU acceleration for paths where all_knum !=1 in Band_DFT_NonCol.c
36	1e0c777e	Remove runtest.result from Git

#	ハッシュ	題名(原文)
37	c8c1e3e1	Modify the Makefile
38	cb4d348d	Now compatible with the latest GEMMu8
39	d451eeb8	Fixed OpenMP bugs in Total_Energy.c and Stress.c
40	2302f965	The name has been changed from CuSOLVER to GPUSOLVER
41	d5b897fd	Band_DFT_Col: serialize GPU turns in pairs and release GEMMu8 workspace per turn
42	788acb4c	Delete unnecessary files, such as the MAGMA library
43	d8219384	Rename the identifier from cusolver to gpusolver
44	8647a5ed	Set large2_example and Large3_example to be processed on the GPU.
45	f508a2a6	Enable direct GPUSOLVER dispatch for DC-LNO
46	3545a9d5	Edit README.md
47	ca3d97c2	Edit README.md
48	43e30c65	Limit band GPU solver concurrency adaptively
49	66ce1e62	Use GEMMu8 for remaining GPU GEMM paths
50	a7f8cba6	Remove obsolete cuBLAS wrapper files
51	8c7801fa	Limit band GPU k-turn concurrency
52	64da9a3d	Add Implementation Manual
53	bff93d08	Add single-rank GPU density grid service
54	95fa51ed	Accelerate Hamiltonian construction with adaptive GPU scheduling
55	fe68cca6	Accelerate force evaluation with fused contractions
56	58ea6471	Adapt band GPU k-turn concurrency to device memory
57	7c414f91	Adapt non-collinear band GPU concurrency to device memory
58	75fc30d3	Accumulate cluster density matrix on the GPU
59	a9b6fa31	Trim per-iteration GPU overhead in noncollinear cluster path
60	d62cac60	Run noncollinear DC-LNO cluster solves on the GPU
61	f5de630e	Cut per-iteration overhead in the noncollinear cluster GPU path
62	527ed524	Keep collinear cluster eigenvectors on the GPU for the DM step
63	211f0684	Batch the nonlocal-projector integrals on the GPU in Set_Nonlocal.c
64	6b84a1f9	Start the Set_Hamiltonian GPU wave at the full device-sharing rank count
65	48921b36	Start the Band_DFT_Col k-point wave at the full device-sharing rank count
66	99f4014b	Start the Band_DFT_NonCol k-point wave at the full device-sharing rank count
67	697383e6	Run the Force3 grid trace on the GPU
68	7d38382d	Batch the Force4B and Force_HNL traces on the GPU, move dOrbitals on device
69	19b9b69c	Run the Total_Energy device kernels in the NC case, batch the Exc0 mesh
70	77d2a22c	Batch the Set_ProExpn_VNA construction and traces on the GPU
71	28361f7e	Reuse fixed-size device buffers across Force3 trace chunks
72	80a3a8f7	Drop the one-rank GPU service cache diagnostic line
73	b569f272	Print the GPU-fallback notice once per node, not once per rank
74	098adddc	Fixed a bug in Force.c

#	ハッシュ	題名(原文)
75	bff0af37	Reduce Set_Hamiltonian GPU staging with on-device Vpot gather and a resident cache
76	69dde552	Make the Set_Hamiltonian resident cache yield to published GPU-phase needs
77	30148e1e	Label the GPU-accelerated VNA projector and force stages on stdout
78	d14ef297	Add a distributed GPU quadrature to Set_Density_Grid
79	6f7f31b2	Cut per-atom staging overhead in the GPU divide-conquer solver
80	3ee45228	Fuse the per-atom GPU pipeline in the Krylov O(N) solver
81	ae3dd189	Keep large cluster GPU runs from freezing or exhausting memory
82	206b7e06	Make the force-stage GPU batches immune to shared-device memory races
83	cff323e6	Fall back to ELPA when the cluster GPU diagonalization cannot reserve its buffers
84	d0e1d17a	Close the last fatal allocation windows in the force-stage GPU batches
85	3ef5347a	Fall back to ELPA when the non-collinear cluster GPU diagonalization cannot reserve its buffers
86	a4200a04	Fall back to ELPA when one k-point's band GPU diagonalization cannot fit
87	94b6ca8c	Accelerate RMM-DIISH mixing with a two-tier GPU/incremental fast path
88	6e94a9d3	Add fast paths to the remaining charge/DM mixing schemes
89	c5eb6845	Fix phantom Set_Hamiltonian abort caused by stale latched CUDA errors
90	bf08a9d8	Serialize the two-spin cluster GPU diagonalization when one device cannot hold both owners
91	9fe8ce13	Make a pristine tree build with a single make invocation

参考文献

1. OpenMX 公式サイト: <http://www.openmx-square.org/> (コード本体・マニュアル・入力キーワード)
2. GEMMu8(GEMMulate)リポジトリ: <https://github.com/RIKEN-RCCS/GEMMu8> (MIT ライセンス、責任開発者 Yuki Uchino, RIKEN R-CCS)
3. Y. Uchino, K. Ozaki, T. Imamura, "High-Performance and Power-Efficient Emulation of Matrix Multiplication using INT8 Matrix Engines", SC Workshops '25, doi:10.1145/3731599.3767539
4. K. Ozaki, Y. Uchino, T. Imamura, "Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique", arXiv:2504.08009
5. Y. Uchino, S. Ma, T. Imamura, K. Ozaki, T. Gutsche, "Emulation of Complex Matrix Multiplication based on the Chinese Remainder Theorem", arXiv:2512.08321(複素対応の理論)
6. Y. Uchino, K. Ozaki, T. Imamura, "Double-Precision Matrix Multiplication Emulation via Ozaki-II Scheme with FP8 Quantization", arXiv:2603.10634
7. H. Ootomo, K. Ozaki, R. Yokota, "DGEMM on integer matrix multiplication unit", Int. J. High Perform. Comput. Appl. (2024), doi:10.1177/10943420241239588 (ozIMMU / Ozaki Scheme I 系の先行研究)
8. H. Kawai ほか、GPU 版 OpenMX の ベ ン チ マ ー ク : J. Phys. Soc. Jpn. 94, 124003 (2025), doi:10.7566/JPSJ.94.124003
9. NVIDIA cuSOLVER Documentation(cusolverDnXsyevd/XsyevdX): <https://docs.nvidia.com/cuda/cusolver/>
10. NVIDIA cuBLAS Documentation: <https://docs.nvidia.com/cuda/cublas/>
11. NVIDIA HPC SDK Documentation(OpenACC): <https://docs.nvidia.com/hpc-sdk/>
12. ELPA(Eigenvalue solvers for Petaflop Applications): <https://elpa.mpcdf.mpg.de/>
13. FFTW: <https://www.fftw.org/>

本書について: 本書は 2026-07-23 時点のソースツリー(コミット 9fe8ce13)と、オリジナル OpenMX 4.0 との全ファイル diff、および開発リポジトリの全 91 コミットの調査に基づいて作成された(第 3 版)。記載の行番号・既定値は当該時点のものである。誤りを見つけた場合はソースコードを一次情報として優先されたい。